Chapter 1: Why Math for ML? — And How to Read This Book

> *"Pure mathematicians prove theorems. Engineers ship products. ML practitioners need just enough math to know which theorem to reach for."*

1.0 What You Need Before This Book

This book assumes you are a working software engineer. Specifically:

You should be comfortable with:

- Basic algebra: solving for x in $2x + 3 = 7$
- Functions: understanding that $f(x) = x^2$ means "square the input"
- Loops, arrays, and basic Python (all code examples use Python)
- The concept of a coordinate system (x/y axes from high school)

You do NOT need:

- Calculus (we teach it from scratch in Chapters 5–6)
- Any prior statistics beyond "average" and "percentage"
- University-level mathematics of any kind
- Any ML framework experience

If you passed high school math and write code for a living, you're ready.

1.1 You Already Know More Than You Think

If you've written a loop that processes a list of numbers, you've done linear algebra.

If you've implemented a retry policy with exponential backoff, you've applied calculus concepts.
If you've A/B tested a feature and asked "is this difference real?", you've done statistics.

Machine learning doesn't invent new mathematics — it *combines* existing mathematics in specific, powerful ways. As a software engineer, your biggest advantage is that you already think algorithmically. Math notation is just another syntax.

This book translates that syntax.

1.2 Why Can't You Skip the Math?

A fair question. Many ML libraries abstract the math away — you can call `model.fit(X, y)` without knowing what happens inside. So why bother?

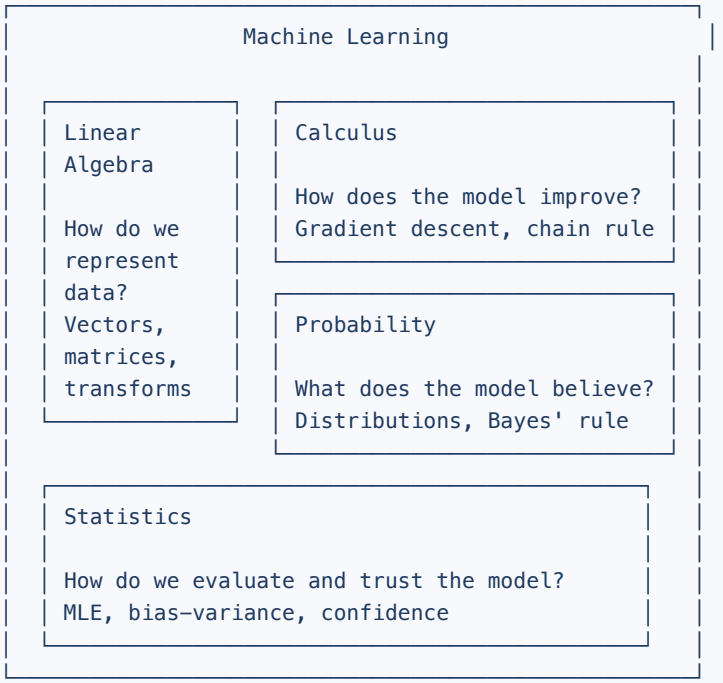
Here's what happens when you skip the math:

Situation	Without math	With math
Your model isn't learning	"Try different hyperparameters randomly"	"Learning rate is too high — gradient is overshooting the minimum"
Predictions are wildly wrong	"Must be a data issue"	"Features aren't normalized — large-scale features dominate the dot product"
Model works in dev, fails in prod	"Redeploy and hope"	"The distribution shifted — the model's assumptions about input variance broke"
You read a paper	"I'll wait for someone to implement it"	"I can implement this myself in a weekend"

The math gives you *debugging superpowers*. It's the difference between driving a car and understanding how an engine works.

1.3 The Four Pillars of ML Math

Everything in machine learning rests on four mathematical foundations:



Linear Algebra is the *data language*. Every image, sentence, user click, and product recommendation is a vector or matrix. Neural network layers are matrix multiplications.

Calculus is the *learning mechanism*. Training a model means minimizing error. Minimizing anything requires calculus — specifically, finding where the gradient equals zero.

Probability is the *uncertainty language*. ML models don't output facts; they output probabilities. Classification, generation, recommendation — all built on probability.

Statistics is the *trust layer*. How do you know your model isn't just memorizing training data? How do you compare two models fairly? Statistics answers this.

1.4 How This Book Is Organized

We go in dependency order — each chapter builds on the last:

```
Chapter 1  → This chapter (orientation)
Chapter 2  → Vectors (the atom of ML data)
Chapter 3  → Matrices (data in bulk)
Chapter 4  → Decompositions (SVD, PCA — deep matrix analysis)
Chapter 5  → Derivatives (rate of change)
Chapter 6  → Gradients & Optimization (how models learn)
Chapter 7  → Probability foundations
Chapter 8  → Key distributions (Gaussian, Bernoulli, softmax)
Chapter 9  → Statistics for ML (MLE, bias-variance)
Chapter 10 → Everything combined in a neural network
```

Each chapter follows this structure:

1. **The concept in plain English** — no equations first
2. **The notation** — the symbols and what they mean
3. **Worked examples** — step by step, nothing skipped
4. **Engineer's angle** — how this maps to code
5. **Python** — runnable examples using NumPy

1.5 How to Read Math Notation

Math notation intimidates most programmers at first. Here's a decoder ring.

Summation: $\sum$

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

In code:

```
total = sum(x[i] for i in range(1, n+1))
# or simply:
total = sum(x)
```

Think of $\sum$ as a `for` loop that adds things up.

Product: $\prod$

$$\prod_{i=1}^{n} x_i = x_1 \times x_2 \times x_3 \times \cdots \times x_n$$

In code:

```
import math
product = math.prod(x)
```

Think of $\prod$ as a `for` loop that multiplies things.

Subscript and Superscript

- x_i means "the i-th element of x" — like `x[i]`
- $x^{(i)}$ means "the i-th *example* in your dataset" — like `dataset[i]`
- x^2 means "x squared" — context tells you which one

How to tell the difference: In ML papers, superscripts in *parentheses* like $x^{(i)}$ always index examples. Superscripts *without* parentheses like x^2 or x^T are mathematical operations (squaring, transposing). When you see $\mathbf{w}^{(3)}$, that's the weights of example 3. When you see $\mathbf{w}^2$, that's the weights element-wise squared.

```
# x^(i) — index into dataset
dataset = [[1,2,3], [4,5,6], [7,8,9]]
x_2 = dataset[2]          # x^(2): the 2nd training example → [7,8,9]

# x^2 — mathematical operation
import math
x = 5
x_squared = x ** 2        # x^2: five squared → 25
```

Common Greek Letters

Symbol	Name	Common ML use
α	alpha	Learning rate
β	beta	Momentum coefficient
θ	theta	Model parameters (weights)
μ	mu	Mean of a distribution
σ	sigma	Standard deviation
ϵ	epsilon	Small error term
λ	lambda	Regularization strength
∇	nabla	Gradient operator

"For All" and "There Exists"

- $\forall x$ means "for all x" — like iterating over all elements
- $\exists x$ means "there exists an x" — like a search/find operation

Absolute Value and Norms

- $|x|$ — absolute value of a scalar: $|-3| = 3$
- $|v|$ — "norm" or "length" of a vector (more on this in Chapter 2)

Functions

$$f : \mathbb{R}^n \to \mathbb{R}$$

Read: "function f takes an n-dimensional real-valued vector as input and returns a real number."
In code: `def f(x: np.ndarray) -> float`

Sets of Numbers

Symbol	Meaning	Example
$\mathbb{N}$	Natural numbers	0, 1, 2, 3, ...
$\mathbb{Z}$	Integers	..., -2, -1, 0, 1, 2, ...
$\mathbb{R}$	Real numbers	3.14, -0.001, 1000.0
$\mathbb{R}^n$	n-dimensional real vector	`np.array` of length n
$\mathbb{R}^{m \times n}$	Real matrix with m rows, n cols	`np.array` of shape (m, n)

1.6 A Note on Proofs

This book does **not** require you to prove theorems. We'll show *why* things work using intuition and examples, and we'll verify with code. When a proof matters for understanding, we'll walk through it conversationally.

We'll clearly label two kinds of claims:

- **"Here's why:"** — we walk through the reasoning so you genuinely understand it
- **"Trust this result:"** — the proof is beyond our scope, but we tell you what it means and when to use it

Example of each style (previewing Chapter 3):

> **Here's why:** The dot product $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$ equals zero when two vectors are perpendicular. Why? Because $\cos(90^\circ) = 0$, and cosine measures the angle between them. We'll derive this geometrically in Chapter 2.

> **Trust this result:** Every real symmetric matrix can be decomposed into $A = Q\Lambda Q^T$ where Q is orthogonal and Λ is diagonal. The proof requires advanced linear algebra. What matters for ML: this guarantees that covariance matrices (always symmetric) can always be decomposed this way — making PCA possible.

Honesty about complexity is more useful than false simplicity.

1.7 Setting Up Your Environment

All code examples use Python with NumPy. Here's how to follow along:

```
# Verify your environment
import numpy as np
import matplotlib.pyplot as plt

print("NumPy version:", np.__version__)

# A tiny taste of what's coming — don't worry about the math yet,
# just confirm the code runs. We explain all of this in Chapters 2–
3.

# A vector: a 1D array of numbers (Chapter 2)
v = np.array([3.0, 4.0])

# A matrix: a 2D array (Chapter 3)
A = np.array([[1, 0],
              [0, 2]])

# Matrix × vector — the core operation of every neural network
layer
result = A @ v
print("A @ v =", result)  # [3. 8.]

# What just happened?
# Row 1 of A: [1, 0] · [3, 4] = 1×3 + 0×4 = 3
# Row 2 of A: [0, 2] · [3, 4] = 0×3 + 2×4 = 8
# You multiplied v by a "transformation matrix" that stretched its
y-component
```

Expected output:

```
NumPy version: 1.26.x
A @ v = [3. 8.]
```

The `@` operator is Python's matrix multiplication operator (PEP 465, Python 3.5+). You'll see it in every ML codebase. For now, think of it as "apply transformation A to vector v." Chapter 3 explains exactly how the numbers are computed.

1.8 Chapter Summary

Topic	Key Takeaway
Prerequisites	High school algebra + coding; nothing else required
Four pillars	Linear algebra (data), calculus (learning), probability (uncertainty), statistics (trust)
Notation guide	$\sum$ = loop, $\prod$ = multiply-loop, θ = weights, ∇ = gradient
Superscripts	$x^{(i)}$ = i-th example; x^2 = x squared; parentheses = indexing
Proof policy	"Here's why" = we explain it; "Trust this result" = we tell you what to use it for
Chapter structure	Concept → notation → examples → code (every chapter)

Exercises

1.1 Look at this expression: $\sum_{i=1}^{5} i^2$. Compute it by hand, then verify with Python.

Solution: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$

```
total = sum(i**2 for i in range(1, 6))
print(total)  # 55
```

1.2 Translate this Python function into math notation:

```
def f(x):
    return sum(x[i] * x[i] for i in range(len(x)))
```

Solution: $f(\mathbf{x}) = \sum_{i=1}^{n} x_i^2$
(This is the squared norm — you'll see it in Chapter 2.)

1.3 In ML papers you'll often see: $\hat{y} = f_\theta(\mathbf{x})$. What does each symbol mean?

Solution:

- $\hat{y}$ (y-hat) = predicted output (the hat means "estimated")
- f = the model function

- θ = the model's learned parameters (weights)
- $\mathbf{x}$ = the input features (a vector)

Read as: "the model with parameters θ predicts $\hat{y}$ given input $\mathbf{x}$."

Next: Chapter 2 — Vectors. The fundamental unit of everything in ML.

Chapter 2: Linear Algebra I — Vectors

> *"A vector is just an array with geometry attached."*

2.1 The Concept: What Is a Vector?

In programming, an array is just a container of numbers:

```
temperatures = [72.1, 68.4, 75.0, 71.3]
```

A **vector** is the same thing, but with an important extra property: the *position* of each number has a geometric meaning. When you have a vector `[3.0, 4.0]`, you're not just storing two numbers — you're describing a **point in 2D space**, or an **arrow pointing from the origin to that point**.

This geometric interpretation is what makes vectors powerful for ML. Every data point in your training set is a vector. Every image, sentence embedding, and user feature profile is a vector. The math we do on vectors — dot products, norms, projections — directly translates to ML operations.

Vectors as Arrows

Think of the vector $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ as an arrow:

```
y
5 |        ●  ← tip at (3, 4)
4 |       /
3 |      /
2 |     /  ← the arrow
1 |    /
0 |___/_____________ x
  0  1  2  3  4  5
  ↑
origin (tail)
```

The vector starts at the origin (0, 0) and points to (3, 4). The length of this arrow is something we'll compute shortly — it's called the **norm**.

Notation

We write vectors in bold lowercase: $\mathbf{v}$, $\mathbf{x}$, $\mathbf{w}$.

A vector in n-dimensional space:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$$

- $v_1, v_2, \ldots, v_n$ are the **components** (or elements)
- $\mathbb{R}^n$ means "a list of n real numbers"
- In code: `v = np.array([v1, v2, ..., vn])`

Column vs row vectors:
By convention, vectors are **columns** (tall) unless stated otherwise. A row vector is written $\mathbf{v}^T$ (transposed). This distinction matters for matrix multiplication in Chapter 3.

```
import numpy as np

# Column vector (conceptually)
v = np.array([3.0, 4.0])   # shape: (2,)

# Row vector (transpose)
v_row = v.reshape(1, -1)   # shape: (1, 2)
v_col = v.reshape(-1, 1)   # shape: (2, 1)
```

2.2 Vector Operations

2.2.1 Addition

Adding two vectors: add component by component.

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} + \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} u_1 + v_1 \\ u_2 + v_2 \end{bmatrix}$$

Worked example:

$$\begin{bmatrix} 1 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 + 4 \\ 3 + 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$$

Geometric meaning: Vector addition is like following two arrows end-to-end. Go 1 right and 3 up, then go 4 right and 2 up — you end up at (5, 5).

```
u = np.array([1.0, 3.0])
v = np.array([4.0, 2.0])
result = u + v
print(result)  # [5. 5.]
```

Rule: Vectors must have the same dimension to be added. Adding a 3D vector to a 2D vector is a type error — both in math and NumPy.

2.2.2 Scalar Multiplication

A **scalar** is a single number (as opposed to a vector). Multiplying a vector by a scalar stretches or shrinks it:

$$c \cdot \mathbf{v} = c \begin{bmatrix} v_1 & v_2 \end{bmatrix} = \begin{bmatrix} c \cdot v_1 & c \cdot v_2 \end{bmatrix}$$

Worked example:

$$3 \cdot \begin{bmatrix} 2 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 3 \end{bmatrix}$$

If $c > 1$: stretches the vector (makes it longer).
If $0 < c < 1$: shrinks it (makes it shorter).
If $c < 0$: flips the direction AND scales it.

```
v = np.array([2.0, 1.0])
print(3 * v)    # [6. 3.]
print(-1 * v)   # [-2. -1.]  ← flipped direction
print(0.5 * v)  # [1. 0.5]   ← shrunk
```

2.2.3 Element-wise Multiplication (Hadamard Product)

Not to be confused with the dot product. The Hadamard product multiplies corresponding elements:

$$\mathbf{u} \odot \mathbf{v} = \begin{bmatrix} u_1 \cdot v_1 & u_2 \cdot v_2 \end{bmatrix}$$

```
u = np.array([2.0, 3.0])
v = np.array([4.0, 5.0])
print(u * v)   # [8. 15.]  ← element-wise (Hadamard)
```

This is used in neural network attention mechanisms and gating. The dot product (next section) is completely different.

2.3 The Dot Product

The dot product is the single most important vector operation in ML. It appears in:

- Every linear regression prediction
- Every neuron in a neural network
- Cosine similarity for embeddings
- The attention mechanism in Transformers

Definition

Given two vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, their dot product is:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{n} u_i v_i = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

The result is a **scalar** (a single number), not a vector.

Worked example — 3D vectors:

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

$$\mathbf{u} \cdot \mathbf{v} = (1)(4) + (2)(5) + (3)(6) = 4 + 10 + 18 = 32$$

```
u = np.array([1.0, 2.0, 3.0])
v = np.array([4.0, 5.0, 6.0])

# Three equivalent ways to compute the dot product:
print(np.dot(u, v))   # 32.0
print(u @ v)          # 32.0  ← preferred in modern ML code
print(sum(u * v))     # 32.0  ← explicit version
```

The Geometric Meaning

Here's the key insight the formula hides:

$$\mathbf{u} \cdot \mathbf{v} = |\mathbf{u}||\mathbf{v}| \cos\theta$$

where θ is the angle between the two vectors.

> **Trust this result:** The algebraic proof that $\sum u_i v_i = |\mathbf{u}||\mathbf{v}| \cos\theta$ requires the Law of Cosines from trigonometry. We won't prove it here, but you can verify it numerically: for any two vectors, compute both sides and confirm they match. The important thing is *what this formula tells you*: the dot product is zero when $\theta = 90^\circ$, positive when the angle is acute, and negative when obtuse.

This means:

- If $\mathbf{u}$ and $\mathbf{v}$ point in the **same direction**: $\cos(0^\circ) = 1$, so the dot product is maximized (positive)
- If they're **perpendicular**: $\cos(90^\circ) = 0$, so the dot product is zero
- If they point in **opposite directions**: $\cos(180^\circ) = -1$, so the dot product is negative

Why does this matter for ML?

In a neural network, a neuron computes:

$$\text{output} = \mathbf{w} \cdot \mathbf{x} = \sum_i w_i x_i$$

where $\mathbf{w}$ is the weight vector and $\mathbf{x}$ is the input feature vector. The dot product measures how much $\mathbf{x}$ "aligns with" $\mathbf{w}$. High alignment = high activation. This is literally what it means for a neuron to "fire" strongly.

Worked example — geometric interpretation:

Let $\mathbf{u} = \begin{bmatrix} 1 & 0 \end{bmatrix}$ (pointing right) and $\mathbf{v} = \begin{bmatrix} 0 & 1 \end{bmatrix}$ (pointing up).

$$\mathbf{u} \cdot \mathbf{v} = (1)(0) + (0)(1) = 0$$

Zero — they're perpendicular. In ML terms: these two features are completely uncorrelated in the direction they "point."

```
u = np.array([1.0, 0.0])
v = np.array([0.0, 1.0])
print(u @ v)  # 0.0 — perpendicular vectors

# Same direction:
a = np.array([1.0, 0.0])
b = np.array([3.0, 0.0])
print(a @ b)  # 3.0 — positive, aligned

# Opposite directions:
c = np.array([1.0, 0.0])
d = np.array([-1.0, 0.0])
print(c @ d)  # -1.0 — negative, opposed
```

2.4 Vector Norms (Length)

The **norm** of a vector is its length — the distance from the origin to its tip.

L2 Norm (Euclidean Norm)

The most common norm. For vector $\mathbf{v} = \begin{bmatrix} v_1 \; v_2 \vdots v_n \end{bmatrix}$:

$$\| \mathbf{v} \|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\sum_{i=1}^n v_i^2}$$

This is just the Pythagorean theorem generalized to n dimensions.

Worked example:

$$\mathbf{v} = \begin{bmatrix} 3 \; 4 \end{bmatrix}$$

$$|\mathbf{v}|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

This is the famous 3-4-5 right triangle. The vector has length 5.

```
v = np.array([3.0, 4.0])
print(np.linalg.norm(v))     # 5.0
print(np.linalg.norm(v, 2))  # 5.0  ← explicit L2
print(np.sqrt(v @ v))        # 5.0  ← computed from dot product
```

Note: $|\mathbf{v}|^2 = \mathbf{v} \cdot \mathbf{v}$ — the squared norm equals the dot product with itself. This identity appears constantly in ML loss functions.

L1 Norm (Manhattan Norm)

Sum of absolute values:

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n| = \sum_{i=1}^{n} |v_i|$$

Worked example:

$$\mathbf{v} = \begin{bmatrix} 3 & -4 \end{bmatrix}, \quad |\mathbf{v}|_1 = |3| + |-4| = 3 + 4 = 7$$

The L1 norm doesn't use the Pythagorean theorem — it's the distance you'd travel if you could only move along grid lines (like city blocks). This is why it's called the "Manhattan" norm.

```
v = np.array([3.0, -4.0])
print(np.linalg.norm(v, 1))   # 7.0
print(np.sum(np.abs(v)))      # 7.0
```

When each norm is used in ML:

Norm	Symbol	Use case
L2	$\|\mathbf{w}\|_2$	L2 regularization (Ridge); distance metrics; default norm
L1	$\|\mathbf{w}\|_1$	L1 regularization (Lasso); promotes sparsity (many weights $\to 0$)
L∞	$\|\mathbf{w}\|_\infty$	Max absolute element; used in adversarial robustness (e.g., FGSM attacks)

L∞ Norm

$$|\mathbf{v}|_\infty = \max_i |v_i|$$

```
v = np.array([3.0, -4.0, 1.0])
print(np.linalg.norm(v, np.inf))  # 4.0 — largest absolute value
```

2.5 Unit Vectors and Normalization

A **unit vector** has norm exactly 1. You create one by dividing a vector by its norm:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{|\mathbf{v}|}$$

(The hat notation ^ means "unit vector in direction of.")

Worked example:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad |\mathbf{v}| = 5$$

$$\hat{\mathbf{v}} = \frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

Verify: $|\hat{\mathbf{v}}| = \sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = \sqrt{1.0} = 1$ ✓

```
v = np.array([3.0, 4.0])
v_hat = v / np.linalg.norm(v)
print(v_hat)                    # [0.6 0.8]
print(np.linalg.norm(v_hat))    # 1.0
```

Why normalize? In ML, unnormalized features cause problems. If feature A is in dollars (0–100,000) and feature B is in years (0–100), the dollar feature dominates the dot product by sheer magnitude. Normalization puts all features on the same scale so the model can compare them fairly.

2.6 Cosine Similarity

Cosine similarity measures how similar two vectors are *in direction*, regardless of magnitude. It's the backbone of recommendation systems, search, and NLP embeddings.

$$\text{cosine similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}||\mathbf{v}|}$$

This is exactly $\cos\theta$ from the geometric dot product formula. Output range: $[-1, 1]$.

Value	Meaning
1.0	Identical direction (most similar)
0.0	Perpendicular (no similarity)
-1.0	Opposite direction (most dissimilar)

Worked example — word embeddings:

Imagine words are represented as 3D vectors (in reality, 768 or more dimensions):

$$\text{"king"} = [0.8\ 0.3\ 0.1], \quad \text{"queen"} = [0.7\ 0.4\ 0.2], \quad \text{"apple"} = [0.1\ 0.9\ 0.8]$$

Step 1 — compute dot products:

$$\text{"king"} \cdot \text{"queen"} = (0.8)(0.7) + (0.3)(0.4) + (0.1)(0.2) = 0.56 + 0.12 + 0.02$$

$$\text{"king"} \cdot \text{"apple"} = (0.8)(0.1) + (0.3)(0.9) + (0.1)(0.8) = 0.08 + 0.27 + 0.08$$

Step 2 — compute norms:

$$|\text{"king"}| = \sqrt{0.64 + 0.09 + 0.01} = \sqrt{0.74} \approx 0.860$$

$$|\text{"queen"}| = \sqrt{0.49 + 0.16 + 0.04} = \sqrt{0.69} \approx 0.831$$

$$|\text{"apple"}| = \sqrt{0.01 + 0.81 + 0.64} = \sqrt{1.46} \approx 1.208$$

Step 3 — compute cosine similarity:

$$\text{sim}(\text{"king"}, \text{"queen"}) = \frac{0.70}{0.860 \times 0.831} \approx \frac{0.70}{0.715} \approx 0.980$$

$$\text{sim}(\text{"king"}, \text{"apple"}) = \frac{0.43}{0.860 \times 1.208} \approx \frac{0.43}{1.039} \approx 0.414$$

"King" and "queen" have similarity 0.979 (nearly identical direction) while "king" and "apple" have 0.414 — far less similar. The model has learned that kings and queens are semantically close.

```
def cosine_similarity(u, v):
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

king  = np.array([0.8, 0.3, 0.1])
queen = np.array([0.7, 0.4, 0.2])
apple = np.array([0.1, 0.9, 0.8])

print(f"king  vs queen: {cosine_similarity(king, queen):.4f}")  #
~0.9796
print(f"king  vs apple: {cosine_similarity(king, apple):.4f}")  #
~0.4137
```

2.7 Linear Combinations and Span

A **linear combination** of vectors is the result of scaling and adding them:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

where $c_1, c_2, \ldots, c_k$ are scalars.

Example:

$$3\begin{bmatrix}1 & 0\end{bmatrix} + 2\begin{bmatrix}0 & 1\end{bmatrix} = \begin{bmatrix}3 & 0\end{bmatrix} + \begin{bmatrix}0 & 2\end{bmatrix} = \begin{bmatrix}3 & 2\end{bmatrix}$$

The **span** of a set of vectors is all possible linear combinations — the set of all points you can reach by choosing any scalars $c_1, c_2, \ldots$

When do vectors span all of $\mathbb{R}^2$? Only when they're *linearly independent* (§2.8). Here's the contrast:

- $\mathbf{v}_1 = \begin{bmatrix}1 \backslash 0\end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix}0 \backslash 1\end{bmatrix}$ → span all of $\mathbb{R}^2$. Any point $(a, b) = a\mathbf{v}_1 + b\mathbf{v}_2$. ✓
- $\mathbf{v}_1 = \begin{bmatrix}1 \backslash 2\end{bmatrix}$ and $\mathbf{v}_2 = \begin{bmatrix}2 \backslash 4\end{bmatrix}$ → span only a *line* through the origin. Because $\mathbf{v}_2 = 2\mathbf{v}_1$, any combination $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = (c_1 + 2c_2)\mathbf{v}_1$ — you never escape the line $y = 2x$. ✗

The zero vector $\mathbf{0} = \begin{bmatrix}0 \backslash 0\end{bmatrix}$ is always in the span of any set (set all scalars to 0). It plays an important role in ML: zero-initialized weights, zero gradients at convergence, and the null space of a matrix.

Why this matters for ML: Every prediction a linear model makes is a linear combination of the input features:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_nx_n = \mathbf{w} \cdot \mathbf{x}$$

This is literally the dot product — and the entire expressiveness of a linear model is limited to linear combinations of its input features.

2.8 Linear Independence

Two vectors are **linearly independent** if neither is a scalar multiple of the other — they point in genuinely different directions and neither can be built from the other.

Example — dependent:

$$\mathbf{v}_1 = \begin{bmatrix}1 & 2\end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix}2 & 4\end{bmatrix} = 2\mathbf{v}_1$$

$\mathbf{v}_2$ is just $\mathbf{v}_1$ scaled by 2. They're on the same line — linearly **dependent**.

Example — independent:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Neither can be built from the other. They're linearly **independent**.

Why this matters for ML:

- **Redundant features are linearly dependent.** If feature A is always twice feature B, one of them carries no new information.
- **Matrix rank** (Chapter 3) and **PCA** (Chapter 4) are fundamentally about linear independence.
- **Multicollinearity** in regression is the case where features are nearly linearly dependent — it destabilizes model training.

2.9 Engineer's Angle: Vectors in ML Code

Let's connect everything to real ML workflows.

Features as Vectors

Every row in your training data is a feature vector:

```
# A dataset of 4 users with 3 features each: [age, income,
years_experience]
X = np.array([
    [25.0, 50000.0, 2.0],   # user 0: x^(0)
    [34.0, 85000.0, 8.0],   # user 1: x^(1)
    [28.0, 62000.0, 4.0],   # user 2: x^(2)
    [45.0, 120000.0, 20.0], # user 3: x^(3)
])
# Shape: (4, 3) — 4 examples, 3 features

# Each row is one feature vector
user_0 = X[0]  # np.array([25., 50000., 2.])

# The feature "income" across all users is a column vector
incomes = X[:, 1]  # np.array([50000., 85000., 62000., 120000.])
```

The Problem with Different Scales

Look at the data above: age is ~25–45, but income is 50,000–120,000. If we compute dot products directly, income will dominate by a factor of

thousands.

```
# Without normalization
weights = np.array([1.0, 1.0, 1.0])  # equal weights (naive)
pred_user0 = weights @ X[0]
print(pred_user0)  # 50027.0 — income dominates completely

# With normalization (zero mean, unit std)
X_mean = X.mean(axis=0)        # per-feature mean
X_std  = X.std(axis=0, ddof=0) # per-feature std (ddof=0 =
population std)
# Note: ddof=0 is NumPy's default and matches sklearn's
StandardScaler.
# Use ddof=1 for sample std (e.g., in pandas .std()), but ddof=0 is
standard
# for feature normalization in ML.
X_norm = (X - X_mean) / X_std

print(X_norm)
# Now all features are on comparable scales
```

This is **feature normalization** — it converts each feature to a unit vector space. The math is straightforward: subtract the mean (shift to center), divide by std (scale to unit variance).

Distance Between Examples

How "similar" are two users? Use the L2 distance (Euclidean distance):

$$d(\mathbf{x}^{(0)}, \mathbf{x}^{(1)}) = |\mathbf{x}^{(0)} - \mathbf{x}^{(1)}|_2$$

```
# After normalization, distance is meaningful
user0_norm = X_norm[0]
user2_norm = X_norm[2]

diff = user0_norm - user2_norm
distance = np.linalg.norm(diff)
print(f"Distance user0-user2: {distance:.4f}")
# Smaller = more similar users
```

This is exactly what **k-NN (k-Nearest Neighbors)** does — finds training examples closest in feature space.

2.10 Full Code Example

```
import numpy as np

# ── Setup ──────────────────────────────────────────────────
np.set_printoptions(precision=4, suppress=True)

# ── Basic operations ──────────────────────────────────────
u = np.array([1.0, 2.0, 3.0])
v = np.array([4.0, 5.0, 6.0])

print("=== Basic Vector Operations ===")
print("u + v =", u + v)          # [5. 7. 9.]
print("3 * u =", 3 * u)          # [3. 6. 9.]
print("u · v =", u @ v)          # 32.0

# ── Norms ─────────────────────────────────────────────────
print("\n=== Norms ===")
print("||u||_2 =", np.linalg.norm(u))            # 3.7417
print("||u||_1 =", np.linalg.norm(u, 1))         # 6.0
print("||u||_inf =", np.linalg.norm(u, np.inf))  # 3.0

# ── Normalization ─────────────────────────────────────────
print("\n=== Unit Vector ===")
u_hat = u / np.linalg.norm(u)
print("û =", u_hat)                              # [0.2673 0.5345
0.8018]
print("||û|| =", np.linalg.norm(u_hat))          # 1.0

# ── Cosine Similarity ─────────────────────────────────────
print("\n=== Cosine Similarity ===")
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

king  = np.array([0.8, 0.3, 0.1])
queen = np.array([0.7, 0.4, 0.2])
apple = np.array([0.1, 0.9, 0.8])

print(f"king vs queen: {cosine_sim(king, queen):.4f}")  # ~0.9796
print(f"king vs apple: {cosine_sim(king, apple):.4f}")  # ~0.4137

# ── Linear Combination ────────────────────────────────────
print("\n=== Linear Combination ===")
e1 = np.array([1.0, 0.0])
e2 = np.array([0.0, 1.0])
result = 3 * e1 + 2 * e2
print("3*e1 + 2*e2 =", result)  # [3. 2.]

# ── Neural network neuron (dot product in action) ─────────
print("\n=== Single Neuron Computation ===")
# Input features: [age_normalized, income_normalized,
experience_normalized]
x = np.array([0.5, 1.2, -0.3])
# Learned weights
w = np.array([0.8, 0.1, 0.6])
# Bias term
```

```
b = 0.2

# Neuron output (before activation)
z = w @ x + b
print(f"z = w·x + b = {z:.4f}")  # 0.5 + 0.12 - 0.18 + 0.2 = 0.64

# Apply ReLU activation: max(0, z)
# (Activation functions are covered in depth in Chapter 10)
activation = max(0, z)
print(f"ReLU(z) = {activation:.4f}")  # 0.64 (positive, so
unchanged)
```

2.11 Chapter Summary

Concept	Formula	Code
Vector	$\mathbf{v} \in \mathbb{R}^n$	`np.array([...])`
Addition	$\mathbf{u} + \mathbf{v}$	`u + v`
Scalar multiply	$c\mathbf{v}$	`c * v`
Dot product	$\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i$	`u @ v`
L2 norm	$\|\mathbf{v}\|_2 = \sqrt{\sum v_i^2}$	`np.linalg.norm(v)`
L1 norm	$\| \mathbf{v} \|_1 = \sum$	v_i
Unit vector	$\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$	`v / np.linalg.norm(v)`
Hadamard product	$\mathbf{u} \odot \mathbf{v}$	`u * v` (element-wise)
Cosine similarity	$\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$	see §2.6
Zero vector	$\mathbf{0} = \begin{bmatrix} 0 \\ vdots\0 \end{bmatrix}$	`np.zeros(n)`

Key insight: The dot product measures alignment between vectors. Every linear model, every neural network layer, every embedding comparison uses this one operation.

Exercises

2.1 Compute $\mathbf{u} \cdot \mathbf{v}$ by hand, then in Python:

$$\mathbf{u} = \begin{bmatrix} 2 & -1 & 3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 1 & 4 & 2 \end{bmatrix}$$

Solution: $(2)(1) + (-1)(4) + (3)(2) = 2 - 4 + 6 = 4$

```
u = np.array([2, -1, 3])
v = np.array([1, 4, 2])
print(u @ v)  # 4
```

2.2 Find the L2 norm of $\mathbf{w} = \begin{bmatrix} 5 & 12 \end{bmatrix}$.

Solution: $|\mathbf{w}| = \sqrt{25 + 144} = \sqrt{169} = 13$

```
w = np.array([5, 12])
print(np.linalg.norm(w))  # 13.0
```

2.3 A user's feature vector is $\mathbf{x} = \begin{bmatrix} 0.3 & 0.7 & 0.1 \end{bmatrix}$ and a model's weights are $\mathbf{w} = \begin{bmatrix} 2.0 & -1.0 & 0.5 \end{bmatrix}$ with bias $b = 0.4$. Compute the neuron's output $z = \mathbf{w} \cdot \mathbf{x} + b$.

Solution: $(2.0)(0.3) + (-1.0)(0.7) + (0.5)(0.1) + 0.4 = 0.6 - 0.7 + 0.05 + 0.4 = 0.35$

2.4 Are these two vectors linearly independent? $\mathbf{a} = \begin{bmatrix} 3 & 6 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 1 & 2 \end{bmatrix}$

Solution: No. $\mathbf{a} = 3\mathbf{b}$. They're linearly **dependent** — both point in the same direction (along the line $y = 2x$). As features, one carries no new information.

2.5 (Challenge) Show algebraically why $|\mathbf{v}|^2 = \mathbf{v} \cdot \mathbf{v}$.

Solution:

$$\mathbf{v} \cdot \mathbf{v} = \sum_{i=1}^{n} v_i \cdot v_i = \sum_{i=1}^{n} v_i^2$$

$$|\mathbf{v}|^2 = \left(\sqrt{\sum_{i=1}^{n} v_i^2} \right)^2 = \sum_{i=1}^{n} v_i^2$$

They're equal. □

Next: Chapter 3 — Matrices. Vectors in groups, and the operations that transform them.

Chapter 3: Linear Algebra II — Matrices

> *"A matrix is a function in disguise — it transforms one vector into another."*

3.1 The Concept: What Is a Matrix?

In Chapter 2, a vector was a 1D array of numbers. A **matrix** is a 2D array — a grid of numbers organized into rows and columns.

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6]])
# Shape: (2, 3) — 2 rows, 3 columns
```

But here's the deeper insight: **a matrix is a transformation**. When you multiply a matrix by a vector, you're not just doing arithmetic — you're moving that vector to a new position in space. This is how neural network layers work, how images are transformed, and how data is projected from high dimensions to low ones.

Think of a matrix as a function: it takes a vector as input and produces a new vector as output.

$$A : \mathbb{R}^n \to \mathbb{R}^m$$

3.2 Notation and Anatomy

An $m \times n$ matrix has m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

- a_{ij} means row i, column j — like `A[i][j]` (1-indexed in math, 0-indexed in Python)

- Matrices are written in **bold uppercase**: A, B, W
- $A \in \mathbb{R}^{m \times n}$ means "A is a matrix with m rows and n columns of real numbers"

Concrete example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}$$

- $a_{12} = 2$ (row 1, col 2)
- $a_{21} = 4$ (row 2, col 1)
- In Python: `A[0, 1] = 2`, `A[1, 0] = 4`

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print(A.shape)    # (2, 3)
print(A[0, 1])    # 2    ← a_12 (0-indexed)
print(A[1, 0])    # 4    ← a_21 (0-indexed)
```

Special Matrix Shapes

Name	Shape	Description
Square	$n \times n$	Same rows and columns
Row vector	$1 \times n$	Single row
Column vector	$m \times 1$	Single column
Scalar	1×1	A single number as a matrix

3.3 Matrix Operations

3.3.1 Transpose

The **transpose** A^T flips a matrix across its diagonal — rows become columns, columns become rows:

$$\text{If } A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \text{ then } A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

If $A \in \mathbb{R}^{m \times n}$, then $A^T \in \mathbb{R}^{n \times m}$.

Formally: $(A^T)_{ij} = A_{ji}$

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.T)
# [[1 4]
#  [2 5]
#  [3 6]]
print(A.T.shape)  # (3, 2)
```

Why it matters in ML: The relationship between row and column vectors uses transpose. When computing $\mathbf{u} \cdot \mathbf{v}$ as a matrix product, it's written $\mathbf{u}^T\mathbf{v}$ (row vector times column vector = scalar).

3.3.2 Addition and Subtraction

Add element-by-element. Both matrices must have **identical** shape.

$$A + B = \begin{bmatrix} a_{11}+b_{11} & a_{12}+b_{12} & a_{21}+b_{21} & a_{22}+b_{22} \end{bmatrix}$$

Worked example:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 & 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 & 10 & 12 \end{bmatrix}$$

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print(A + B)
# [[ 6  8]
#  [10 12]]
```

3.3.3 Scalar Multiplication

Multiply every element by the scalar:

$$3 \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 9 & 12 \end{bmatrix}$$

```
A = np.array([[1, 2], [3, 4]])
print(3 * A)
# [[ 3  6]
#  [ 9 12]]
```

3.4 Matrix Multiplication

This is the most important operation in all of ML. **Every neural network forward pass is a sequence of matrix multiplications.**

The Rule

To multiply $A \in \mathbb{R}^{m \times k}$ by $B \in \mathbb{R}^{k \times n}$, the **inner dimensions must match**. The result is $C \in \mathbb{R}^{m \times n}$.

$$A_{m \times k} \times B_{k \times n} = C_{m \times n}$$

Memory trick:

```
(m × k) @ (k × n) = (m × n)
         ↑↑
      must match
```

Each element c_{ij} of C is the dot product of row i of A with column j of B:

$$c_{ij} = \sum_{l=1}^{k} a_{il} \cdot b_{lj}$$

Step-by-Step Worked Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Both are 2×2, so $C = AB$ is also 2×2:

$$c_{11} = \text{row 1 of } A \cdot \text{col 1 of } B = (1)(5) + (2)(7) = 5 + 14 = 19$$

$$c_{12} = \text{row 1 of } A \cdot \text{col 2 of } B = (1)(6) + (2)(8) = 6 + 16 = 22$$

$$c_{21} = \text{row 2 of } A \cdot \text{col 1 of } B = (3)(5) + (4)(7) = 15 + 28 = 43$$

$$c_{22} = \text{row 2 of } A \cdot \text{col 2 of } B = (3)(6) + (4)(8) = 18 + 32 = 50$$

$$C = AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

```python
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A @ B
print(C)
# [[19 22]
#  [43 50]]
```

Matrix × Vector (The Core ML Operation)

When a matrix multiplies a vector, the output is a new vector:

$$A\mathbf{x} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1x_1 + 2x_2 \\ 3x_1 + 4x_2 \\ 5x_1 + 6x_2 \end{bmatrix}$$

Each output element is a dot product of one row with $\mathbf{x}$.

Worked example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

$$A\mathbf{x} = \begin{bmatrix} (1)(5) + (2)(6) \\ (3)(5) + (4)(6) \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

```
A = np.array([[1, 2], [3, 4]])
x = np.array([5.0, 6.0])
print(A @ x)  # [17. 39.]
```

Neural network connection: A fully connected layer computes:

$$\mathbf{h} = W\mathbf{x} + \mathbf{b}$$

where:

- $W \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix
- $\mathbf{x} \in \mathbb{R}^{d_{in}}$ is the input vector
- $\mathbf{b} \in \mathbb{R}^{d_{out}}$ is the bias vector
- $\mathbf{h} \in \mathbb{R}^{d_{out}}$ is the output (hidden layer activations)

A 3-layer network is just: $\hat{y} = W_3(\text{activation}(W_2(\text{activation}(W_1\mathbf{x} + \mathbf{b}_1)) + \mathbf{b}_2)) + \mathbf{b}_3$

Critical Property: Matrix Multiplication is NOT Commutative

For scalars: $3 \times 5 = 5 \times 3$. For matrices: $AB \neq BA$ **in general.**

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}$$

These are different!

Real ML bug this causes: Suppose you build a two-layer network and accidentally swap the layer order:

```
# Correct: W2 applied after W1
h = W2 @ (W1 @ x)   # applies W1 first, then W2

# Wrong: layers swapped
h = W1 @ (W2 @ x)   # applies W2 first — different transformation
entirely
```

Both lines run without errors. Both produce output of the same shape. But you've built a completely different model. The non-commutativity of matrix multiplication means **layer order is semantically significant**, not just an implementation detail. This is one of the most common silent bugs in custom neural network implementations.

3.5 Special Matrices

Identity Matrix I

The identity matrix has 1s on the diagonal and 0s everywhere else:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Property: $AI = IA = A$ for any compatible matrix A. It's the matrix equivalent of multiplying by 1.

```
I = np.eye(3)  # 3×3 identity
A = np.array([[1, 2], [3, 4]])
I2 = np.eye(2)
print(A @ I2)  # [[1. 2.] [3. 4.]] — unchanged
```

Diagonal Matrix

Only the diagonal elements are non-zero:

$$D = \begin{bmatrix} d_1 & 0 & 0 \\ 0 & d_2 & 0 \\ 0 & 0 & d_3 \end{bmatrix}$$

Multiplying by a diagonal matrix scales each dimension independently.

```
D = np.diag([2.0, 3.0, 5.0])
x = np.array([1.0, 1.0, 1.0])
print(D @ x)  # [2. 3. 5.] — each dim scaled separately
```

Symmetric Matrix

$A = A^T$ — the matrix equals its own transpose.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 4 \\ 3 & 4 & 6 \end{bmatrix}$$

Why it matters: Covariance matrices (used in PCA, Gaussian distributions) are always symmetric. This is a critical structural property that simplifies decomposition (Chapter 4).

```
A = np.array([[1, 2, 3], [2, 5, 4], [3, 4, 6]])
print(np.allclose(A, A.T))  # True — it's symmetric
```

Orthogonal Matrix

A square matrix Q where $Q^T Q = I$ — the columns are all unit vectors and are mutually perpendicular.

Property: $Q^{-1} = Q^T$ (the inverse is free — just transpose!).

This property is used extensively in eigendecomposition and SVD (Chapter 4).

3.6 The Matrix Inverse

For a square matrix A, its inverse A^{-1} satisfies:

$$AA^{-1} = A^{-1}A = I$$

Think of it like the reciprocal: $3 \times \frac{1}{3} = 1$. The inverse "undoes" the transformation.

Worked example — 2×2 inverse:

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the inverse is:

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

The quantity $ad - bc$ is called the **determinant**.

> **Trust this result:** This formula is derived by solving $AA^{-1} = I$ as a system of four equations. For 3×3 and larger matrices, the formula becomes far more complex (Cramer's rule, cofactor expansion). In practice, you never compute inverses by hand for matrices larger than 2×2 — that's what `np.linalg.inv` is for. **Do not try to generalize the 2×2 formula to 3×3** — it doesn't extend directly.

$$A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$$

Step 1 — determinant: $\det(A) = (2)(3) - (1)(5) = 6 - 5 = 1$

Step 2 — inverse:

$$A^{-1} = \frac{1}{1} \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

Step 3 — verify: AA^{-1} should equal I:

$$\begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} = \begin{bmatrix} (6-5) & (-2+2) \\ (15-15) & (-5+6) \end{bmatrix} =$$

```
A = np.array([[2.0, 1.0], [5.0, 3.0]])
A_inv = np.linalg.inv(A)
print(A_inv)
# [[ 3. -1.]
#  [-5.  2.]]
print(A @ A_inv)
# [[1. 0.]
#  [0. 1.]]  ← identity (rounding errors possible; use np.allclose)
print(np.allclose(A @ A_inv, np.eye(2)))  # True
```

When Does the Inverse Not Exist?

When the determinant is zero: $\det(A) = 0$. This means the matrix "squashes" space — it collapses a dimension, so you can't undo it.

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \quad \det(B) = (1)(4) - (2)(2) = 4 - 4 = 0$$

B has no inverse. This matrix maps all of $\mathbb{R}^2$ onto a single line, losing all information in one direction.

```
B = np.array([[1.0, 2.0], [2.0, 4.0]])
print(np.linalg.det(B))  # ~0.0
# np.linalg.inv(B) would raise LinAlgError: Singular matrix
```

In ML, a **singular** (non-invertible) matrix is often a sign of **multicollinearity** — your features are linearly dependent.

3.7 The Determinant

The **determinant** of a square matrix is a single number that encodes whether the matrix is invertible and how it scales areas/volumes.

$$\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = ad - bc$$

Geometric interpretation: The determinant is the scale factor of the area transformation.

- $|\det(A)| = 2$: the matrix doubles areas
- $|\det(A)| = 0.5$: halves areas
- $\det(A) = 0$: squashes to zero area (singular matrix)
- $\det(A) < 0$: the transformation flips orientation (mirror)

```
A = np.array([[3.0, 0.0], [0.0, 2.0]])  # scale x by 3, y by 2
print(np.linalg.det(A))  # 6.0 — areas scaled by 6x
```

In ML: The determinant appears in the formula for multivariate Gaussian distributions (Chapter 8) and in understanding if a system of equations has a unique solution.

3.8 Matrix Rank

The **rank** of a matrix is the number of linearly independent rows (or columns) — equivalently, the "true" dimensionality of the information the matrix contains.

For $A \in \mathbb{R}^{m \times n}$:

- Maximum possible rank: $\min(m, n)$
- **Full rank:** rank = $\min(m, n)$ — no redundant information
- **Rank-deficient:** rank < $\min(m, n)$ — some rows/columns are redundant

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 7 & 9 \end{bmatrix}$$

Row 3 = Row 1 + Row 2: $[5, 7, 9] = [1, 2, 3] + [4, 5, 6]$. So rank = 2, not 3.

```
A = np.array([[1, 2, 3], [4, 5, 6], [5, 7, 9]], dtype=float)
print(np.linalg.matrix_rank(A))  # 2
```

In ML: Low-rank matrices appear everywhere:

- **Compressed representations:** images often have low-rank structure
- **Overfitting detection:** if your weight matrix becomes low rank during training, it's a sign of redundancy
- **LoRA** (Low-Rank Adaptation): the technique that makes fine-tuning LLMs cheap, by constraining weight updates to low-rank matrices

---

3.9 Systems of Linear Equations

A system of linear equations can always be written as $A\mathbf{x} = \mathbf{b}$:

System:

$$2x + y = 5$$

$$5x + 3y = 13$$

Matrix form: $$\underbrace{\begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 5 \\ 13 \end{bmatrix}}_{\mathbf{b}}$$

Solution: If A is invertible: $\mathbf{x} = A^{-1}\mathbf{b}$

$$\mathbf{x} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} \begin{bmatrix} 5 \\ 13 \end{bmatrix} = \begin{bmatrix} (15 - 13) \\ (-25 + 26) \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Verify: $x = 2, y = 1$: $2(2) + 1 = 5$ ✓ and $5(2) + 3(1) = 13$ ✓

```
A = np.array([[2.0, 1.0], [5.0, 3.0]])
b = np.array([5.0, 13.0])

# Method 1: inverse (not recommended for large systems —
numerically unstable)
x = np.linalg.inv(A) @ b
print(x)  # [2. 1.]

# Method 2: np.linalg.solve (preferred — uses numerically stable
algorithms)
x = np.linalg.solve(A, b)
print(x)  # [2. 1.]
```

> **Practical note:** For large or ill-conditioned systems, prefer `np.linalg.solve(A, b)` over `np.linalg.inv(A) @ b`. The solver uses LU decomposition — faster and more numerically stable. For small, well-conditioned systems (e.g., 3×3), the difference is negligible.

Connection to ML: Linear regression solves exactly this — find weights $\mathbf{w}$ such that $X\mathbf{w} \approx \mathbf{y}$. The closed-form solution (the Normal Equation) is:

$$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$$

> **Warning:** This formula requires computing a matrix inverse, which is $O(n^3)$ and numerically unstable when features are nearly linearly dependent (making $X^T X$ nearly singular). In practice, scikit-learn and other libraries use gradient descent or QR decomposition instead. We'll discuss when the Normal Equation is and isn't appropriate in Chapter 9.

3.10 Broadcasting in NumPy

One NumPy feature every ML engineer needs to understand is **broadcasting** — it's NumPy's way of automatically expanding shapes to make operations work.

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])  # shape (2, 3)
b = np.array([10, 20, 30]) # shape (3,)

# Adding a (3,) vector to a (2, 3) matrix:
# NumPy broadcasts b to [[10,20,30],[10,20,30]] automatically
result = A + b
print(result)
# [[11 22 33]
#  [14 25 36]]
```

The broadcasting rule — NumPy aligns shapes from the **right**:

1. If shapes have different number of dimensions, prepend 1s to the shorter shape
2. For each dimension, sizes must either be equal OR one of them must be 1
3. A dimension of size 1 gets "stretched" to match the other

```
A: (2, 3)
b:    (3,)   → treated as (1, 3) → stretched to (2, 3)  ✓ compatible

A: (2, 3)
c:    (2,)   → treated as (1, 2) → can't match (2, 3)   ✗ ERROR
```

```
A = np.array([[1, 2, 3], [4, 5, 6]])  # (2, 3)
b = np.array([10, 20, 30])             # (3,) → broadcasts to (2,
3)
c = np.array([10, 20])                 # (2,) → shapes (2,3) and
(2,) → ERROR

print(A + b)   # works: [[11,22,33],[14,25,36]]
# print(A + c) # ValueError: operands could not be broadcast
# Fix: reshape c to a column vector
c_col = c.reshape(-1, 1)  # shape (2, 1) → broadcasts to (2, 3)
print(A + c_col)  # [[11,12,13],[24,25,26]]
```

This is used constantly for adding bias vectors in neural networks:

```
# Adding bias b to all examples in a batch
W = np.random.randn(4, 3)   # weight matrix (4 output neurons, 3
inputs)
x_batch = np.random.randn(100, 3)  # 100 training examples, 3
features each
b = np.zeros(4)              # bias vector

# Without broadcasting, you'd need a loop.
# With broadcasting:
h = x_batch @ W.T + b       # shape: (100, 4)
# Each of the 100 examples gets the bias added automatically
```

3.11 Full Code Example

```
import numpy as np

np.set_printoptions(precision=4, suppress=True)

# ── Matrix basics ──────────────────────────────────────────
A = np.array([[1.0, 2.0], [3.0, 4.0]])
B = np.array([[5.0, 6.0], [7.0, 8.0]])

print("=== Matrix Multiplication ===")
print("A @ B =\n", A @ B)
# [[19. 22.]
#  [43. 50.]]

print("\n=== Transpose ===")
print("A.T =\n", A.T)
# [[1. 3.]
#  [2. 4.]]

print("\n=== Determinant ===")
print("det(A) =", np.linalg.det(A))  # -2.0

print("\n=== Inverse ===")
A_inv = np.linalg.inv(A)
print("A_inv =\n", A_inv)
# [[-2.   1. ]
#  [ 1.5 -0.5]]
print("A @ A_inv =\n", A @ A_inv)  # identity

# ── Rank ───────────────────────────────────────────────────
print("\n=== Rank ===")
C_fullrank = np.array([[1, 0], [0, 1]])
C_degenerate = np.array([[1, 2], [2, 4]])
print("rank(I₂) =", np.linalg.matrix_rank(C_fullrank))       # 2
print("rank(degenerate) =", np.linalg.matrix_rank(C_degenerate))  #
1

# ── Solving linear system ──────────────────────────────────
print("\n=== Linear System Ax = b ===")
A_sys = np.array([[2.0, 1.0], [5.0, 3.0]])
b = np.array([5.0, 13.0])
x = np.linalg.solve(A_sys, b)
print("Solution x =", x)          # [2. 1.]
print("Verify Ax =", A_sys @ x)   # [5. 13.]

# ── Neural network layer ───────────────────────────────────
print("\n=== Neural Network Layer ===")
W = np.array([[0.5, -0.3,  0.8],   # weight matrix: 2 output
neurons, 3 inputs
              [0.1,  0.9, -0.2]])
b_nn = np.array([0.1, -0.1])        # bias

x_input = np.array([1.0, 0.5, -1.0])  # input feature vector
h = W @ x_input + b_nn
print("h = W @ x + b =", h)
```

```
# h[0] = 0.5*1 + (-0.3)*0.5 + 0.8*(-1) + 0.1 = 0.5 - 0.15 - 0.8 +
0.1 = -0.35
# h[1] = 0.1*1 + 0.9*0.5 + (-0.2)*(-1) + (-0.1) = 0.1 + 0.45 + 0.2
- 0.1 = 0.65
```

Let's verify those neuron outputs manually:

```python
# Manual verification
h0 = 0.5*1.0 + (-0.3)*0.5 + 0.8*(-1.0) + 0.1
h1 = 0.1*1.0 + 0.9*0.5 + (-0.2)*(-1.0) + (-0.1)
print(f"h[0] = {h0:.4f}")  # -0.3500
print(f"h[1] = {h1:.4f}")  # 0.6500
```

3.12 Chapter Summary

Concept	Definition	Code
Matrix	$A \in \mathbb{R}^{m \times n}$	`np.array([[...], [...]])`
Transpose	$(A^T)_{ij} = A_{ji}$	`A.T`
Matrix multiply	$c_{ij} = \sum_l a_{il} b_{lj}$	`A @ B`
Identity	$AI = A$	`np.eye(n)`
Inverse	$AA^{-1} = I$	`np.linalg.inv(A)`
Determinant	Scalar; 0 = singular	`np.linalg.det(A)`
Rank	# independent rows/cols	`np.linalg.matrix_rank(A)`
Solve $A\mathbf{x} = \mathbf{b}$	$\mathbf{x} = A^{-1}\mathbf{b}$	`np.linalg.solve(A, b)`

Key insights:

1. Matrix multiplication = applying transformations in sequence
2. $AB \neq BA$ — order matters
3. A singular matrix ($\det = 0$) loses information irreversibly
4. Every neural network layer = one matrix multiplication + bias

---

Exercises

3.1 Compute AB by hand:

$$A = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 4 \\ 2 & 1 \end{bmatrix}$$

Solution:

$$c_{11} = 2(1) + 0(2) = 2, \quad c_{12} = 2(4) + 0(1) = 8$$

$$c_{21} = 1(1) + 3(2) = 7, \quad c_{22} = 1(4) + 3(1) = 7$$

$$AB = \begin{bmatrix} 2 & 8 \\ 7 & 7 \end{bmatrix}$$

```
A = np.array([[2, 0], [1, 3]])
B = np.array([[1, 4], [2, 1]])
print(A @ B)  # [[ 2  8] [ 7  7]]
```

3.2 Is $AB = BA$ for the matrices in 3.1? Compute BA and compare.

Solution:

$$b_{11} = 1(2) + 4(1) = 6, \quad b_{12} = 1(0) + 4(3) = 12$$

$$b_{21} = 2(2) + 1(1) = 5, \quad b_{22} = 2(0) + 1(3) = 3$$

$$BA = \begin{bmatrix} 6 & 12 \\ 5 & 3 \end{bmatrix} \neq AB = \begin{bmatrix} 2 & 8 \\ 7 & 7 \end{bmatrix}$$

Matrix multiplication is **not** commutative.

3.3 A neural network layer has weight matrix $W = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & -2 \end{bmatrix}$ and bias $\mathbf{b} = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}$. Compute the output for input $\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$.

Solution:

$$W\mathbf{x} = \begin{bmatrix} 1(1) + (-1)(2) + 2(3) \\ 0(1) + 3(2) + (-2)(3) \end{bmatrix} = \begin{bmatrix} 1 - 2 + 6 \\ 0 + 6 - 6 \end{bmatrix}$$

$$W\mathbf{x} + \mathbf{b} = \begin{bmatrix} 5.5 & -0.5 \end{bmatrix}$$

3.4 Find the inverse of $A = \begin{bmatrix} 3 & 1 \\ 2 & 1 \end{bmatrix}$ by hand using the 2×2 formula.

Solution: $\det(A) = 3(1) - 1(2) = 1$

$$A^{-1} = \frac{1}{1} \begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix}$$

Verify: $\begin{bmatrix} 3 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} 3 - 2 & -3 + 3 \\ 2 - 2 & -2 + 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ ✓

3.5 (Challenge) Why is `np.linalg.solve(A, b)` preferred over `np.linalg.inv(A) @ b`?

Answer: `solve` uses LU decomposition — a numerically stable algorithm that avoids the amplification of floating-point errors that can occur when computing the explicit inverse. For large matrices, it's also significantly faster since computing the full inverse is $O(n^3)$ operations while solving is also $O(n^3)$ but with a smaller constant and without the intermediate storage.

Next: Chapter 4 — Decompositions. We'll break matrices apart to reveal their hidden structure — and discover PCA.

Chapter 4: Linear Algebra III — Decompositions

> *"Every matrix hides a story. Decomposition tells you that story."*

4.1 The Concept: Why Decompose a Matrix?

A matrix looks like a grid of numbers. Decomposition reveals the *structure* underneath those numbers: the directions of maximum variance, the rank, the meaningful components.

Three decompositions matter most for ML:

Decomposition	What it reveals	ML use case
Eigendecomposition	"Natural" axes of the transformation	PCA, graph algorithms
SVD	Full structure for any matrix	PCA, recommendation systems, text analysis
PCA	Directions of maximum variance in data	Dimensionality reduction, visualization

We'll build them in order, each one resting on the previous.

4.2 Eigenvectors and Eigenvalues

The Core Idea

Most vectors, when multiplied by a matrix, both rotate *and* scale. But some special vectors only **scale** — they don't rotate. These are called **eigenvectors**.

Formally, for a square matrix A:

$$A\mathbf{v} = \lambda\mathbf{v}$$

- $\mathbf{v}$ is an **eigenvector** (a non-zero vector that doesn't rotate when A is applied)
- λ (lambda) is the corresponding **eigenvalue** (the scalar it gets stretched by)

A Visual Intuition

Imagine a rubber sheet stretched by matrix A. Most points on the sheet move to new locations — they rotate and scale. But there are special directions along which points only slide forward or backward (scale), never rotating. Those directions are the eigenvectors.

```
Before A:                    After A:
   ↑                            ↑↑   ← eigenvalue = 2 (stretched)
   v₁                           v₁

   → v₂                  →→→→→ v₂    ← eigenvalue = 3 (stretched more)
```

Finding Eigenvalues

The condition $A\mathbf{v} = \lambda\mathbf{v}$ can be rewritten as:

$$A\mathbf{v} - \lambda\mathbf{v} = \mathbf{0}$$

$$(A - \lambda I)\mathbf{v} = \mathbf{0}$$

For this to have a non-trivial solution (non-zero $\mathbf{v}$), the matrix $(A - \lambda I)$ must be singular:

$$\det(A - \lambda I) = 0$$

This is the **characteristic equation**. Solving it gives the eigenvalues.

Worked example — finding eigenvalues of a 2×2 matrix:

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

$$A - \lambda I = \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix}$$

$$\det(A - \lambda I) = (4 - \lambda)(3 - \lambda) - (1)(2) = 0$$

$$= 12 - 4\lambda - 3\lambda + \lambda^2 - 2 = 0$$

$$= \lambda^2 - 7\lambda + 10 = 0$$

$$= (\lambda - 5)(\lambda - 2) = 0$$

So $\lambda_1 = 5$ and $\lambda_2 = 2$.

Finding Eigenvectors

For each eigenvalue, solve $(A - \lambda I)\mathbf{v} = \mathbf{0}$.

For $\lambda_1 = 5$:

$$A - 5I = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}$$

Both rows say the same thing: $-v_1 + v_2 = 0$, so $v_1 = v_2$.

Eigenvector: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (or any scalar multiple)

Verify: $A\mathbf{v}_1 = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ ✓

For $\lambda_2 = 2$:

$$A - 2I = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}$$

Both rows say: $2v_1 + v_2 = 0$, so $v_2 = -2v_1$.

Eigenvector: $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

Verify: $A\mathbf{v}_2 = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ -4 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ -2 \end{bmatrix}$ ✓

```python
import numpy as np

A = np.array([[4.0, 1.0], [2.0, 3.0]])
eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:", eigenvalues)
# [5. 2.]  ← matches our hand calculation

print("Eigenvectors (columns):\n", eigenvectors)
# Each COLUMN is one eigenvector (NumPy convention)
# Column 0 corresponds to eigenvalue 5
# Column 1 corresponds to eigenvalue 2

# Verify: A @ v = λ * v
for i in range(2):
    v = eigenvectors[:, i]
    lam = eigenvalues[i]
    print(f"λ={lam}: A@v = {A@v}, λ*v = {lam*v}")
    print(f"  Match: {np.allclose(A @ v, lam * v)}")
```

What Eigenvalues Tell You

- **Large eigenvalue:** that eigenvector direction is strongly amplified by A
- **Small eigenvalue:** that direction is shrunk
- **Zero eigenvalue:** that direction is completely collapsed — the matrix is singular
- **Negative eigenvalue:** that direction is flipped and scaled

In ML, large eigenvalues = directions of large variance in your data. This is the foundation of PCA.

4.3 Eigendecomposition

If a matrix $A \in \mathbb{R}^{n \times n}$ has n linearly independent eigenvectors, we can write:

> **Edge case:** Not all matrices are diagonalizable. If a matrix has repeated eigenvalues, it may not have n independent eigenvectors (e.g., $\begin{bmatrix}1 & 1\\0 & 1\end{bmatrix}$ has eigenvalue 1 with multiplicity 2 but only one independent eigenvector). In ML practice, covariance matrices (which are symmetric positive semi-definite) are always diagonalizable, so this edge case rarely bites you.

$$A = Q\Lambda Q^{-1}$$

where:

- Q = matrix whose **columns are eigenvectors** $[\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n]$
- Λ = **diagonal matrix** with eigenvalues $\text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$
- Q^{-1} = inverse of Q

For **symmetric** matrices (covariance matrices are always symmetric), $Q^{-1} = Q^T$ (the eigenvectors form an orthogonal basis), so:

$$A = Q\Lambda Q^T$$

This is called the **spectral decomposition**. It decomposes the matrix into its natural "components."

Worked example:

Using $A = \begin{bmatrix}4 & 1\\2 & 3\end{bmatrix}$, $\lambda_1 = 5$, $\lambda_2 = 2$, $\mathbf{v}_1 = \begin{bmatrix}1\\1\end{bmatrix}$, $\mathbf{v}_2 = \begin{bmatrix}1\\-2\end{bmatrix}$:

$$Q = \begin{bmatrix}1 & 1\\1 & -2\end{bmatrix}, \quad \Lambda = \begin{bmatrix}5 & 0\\0 & 2\end{bmatrix}$$

```
# Reconstruct A from eigendecomposition
eigenvalues, Q = np.linalg.eig(A)
Lambda = np.diag(eigenvalues)
A_reconstructed = Q @ Lambda @ np.linalg.inv(Q)
print("Reconstructed A:\n", A_reconstructed.round(10))
# [[4. 1.]
#  [2. 3.]]  ← original A recovered
```

4.4 Singular Value Decomposition (SVD)

Eigendecomposition only works for square matrices. **SVD works for any matrix**, which is why it's the more general and practically useful tool.

The Decomposition

For any matrix $A \in \mathbb{R}^{m \times n}$:

$$A = U \Sigma V^T$$

where:

- $U \in \mathbb{R}^{m \times m}$ — left singular vectors (orthogonal matrix: $U^T U = I$)
- $\Sigma \in \mathbb{R}^{m \times n}$ — singular values on the diagonal (non-negative, sorted descending)
- $V \in \mathbb{R}^{n \times n}$ — right singular vectors (orthogonal matrix: $V^T V = I$)

The **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ are the key. They measure how much "action" the matrix has in each direction.

Geometric Interpretation

Any matrix A can be understood as three operations in sequence:

1. V^T: Rotate the input space
2. Σ: Scale each axis (by the singular values)
3. U: Rotate the output space

```
Input space  →(V^T)→  Rotated  →(Σ)→  Scaled  →(U)→  Output space
```

Worked Example

$$A = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}$$

```
A = np.array([[3.0, 2.0],
              [2.0, 3.0],
              [2.0, -2.0]])

U, sigma, Vt = np.linalg.svd(A)
print("U shape:", U.shape)       # (3, 3)
print("sigma:", sigma)           # singular values (descending)
print("Vt shape:", Vt.shape)     # (2, 2)

# Reconstruct A
Sigma = np.zeros_like(A)
Sigma[:len(sigma), :len(sigma)] = np.diag(sigma)
A_reconstructed = U @ Sigma @ Vt
print("Reconstructed:\n", A_reconstructed.round(10))
```

Low-Rank Approximation: The Key Application

Here's the most important property of SVD: **you can approximate a matrix by keeping only the top-k singular values** and discarding the rest. This gives you the best possible rank-k approximation.

$$A \approx A_k = U_k \Sigma_k V_k^T$$

where we keep only the first k columns of U, the first k singular values of Σ, and the first k rows of V^T.

Why this works: Large singular values carry most of the "energy" (information) of the matrix. Small singular values contribute noise or redundancy. By keeping only the large ones, you get a compressed representation.

Worked example — image compression:

```
# Simulate a small "image" (matrix of pixel values)
image = np.array([
    [10, 20, 20, 10],
    [20, 40, 40, 20],
    [20, 40, 40, 20],
    [10, 20, 20, 10],
], dtype=float)

U, sigma, Vt = np.linalg.svd(image)
print("Singular values:", sigma.round(2))
# [100.  0.   0.   0.]  ← rank 1! All info in first singular value

# Rank-1 approximation (k=1)
k = 1
image_approx = sigma[0] * np.outer(U[:, 0], Vt[0, :])
print("Original:\n", image)
print("Rank-1 approx:\n", image_approx.round(2))
# Identical! This image has perfect rank-1 structure.

# Compute compression ratio
original_values = image.size                    # 16 numbers
compressed_values = k * (U.shape[0] + 1 + Vt.shape[1])  # k * (m +
1 + n)
print(f"Compression: {original_values} → {compressed_values}
values")
```

Real-world impact: The Netflix Prize-winning algorithms (2009) used **matrix factorization** — a technique closely related to SVD — to decompose the user-movie rating matrix into low-rank latent factors. The factors represented latent "taste profiles." True SVD requires all entries to be present; matrix

factorization extends the idea to sparse, incomplete matrices with missing ratings.

4.5 Principal Component Analysis (PCA)

PCA is the most commonly used dimensionality reduction technique in ML. It finds the directions of **maximum variance** in your data and projects onto them.

The Problem PCA Solves

Suppose your data has 100 features (dimensions). Visualizing or training on 100D data is hard. PCA finds the 2 or 3 directions that capture the most information, letting you project to 2D or 3D for visualization or downstream modeling.

Key insight: The directions of maximum variance are the eigenvectors of the **covariance matrix** of your data.

The Covariance Matrix

For a dataset $X \in \mathbb{R}^{n \times d}$ (n examples, d features), after centering (subtracting the mean):

$$\Sigma = \frac{1}{n-1} X^T X \in \mathbb{R}^{d \times d}$$

Each entry Σ_{ij} measures how much feature i and feature j vary together.

- Σ_{ii} = variance of feature i
- $\Sigma_{ij} = \Sigma_{ji}$ = covariance of features i and j (always symmetric!)

PCA Step by Step

Step 1: Center the data (subtract mean of each feature)

$$\tilde{X} = X - \bar{X}$$

Step 2: Compute the covariance matrix (using the *centered* $\tilde{X}$ from Step 1)

$$\Sigma = \frac{1}{n-1} \tilde{X}^T \tilde{X}$$

Note: this formula only works on the already-centered data $\tilde{X}$. Using the raw X here would give the wrong result (it would measure distance from the origin, not from the mean).

Step 3: Compute eigendecomposition of Σ

$$\Sigma = Q \Lambda Q^T$$

The eigenvectors (columns of Q) are the **principal components** — the new axes.

Step 4: Sort by eigenvalue (largest first)

The eigenvector with the largest eigenvalue is the direction of maximum variance (1st principal component). Second largest is perpendicular to it with second-most variance, etc.

Step 5: Project data onto top-k components

$$X_{\text{reduced}} = \tilde{X} \cdot Q_k$$

where Q_k contains the top-k eigenvectors.

Worked Example — PCA from Scratch

```
import numpy as np

# ── Toy dataset: 5 points in 2D ──────────────────────────
X = np.array([
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
    [3.1, 3.0],
    [2.3, 2.7],
    [2.0, 1.6],
    [1.0, 1.1],
    [1.5, 1.6],
    [1.1, 0.9],
])
n, d = X.shape  # 10 examples, 2 features

# ── Step 1: Center ──────────────────────────────────────
X_mean = X.mean(axis=0)
X_centered = X - X_mean
print("Feature means:", X_mean.round(4))

# ── Step 2: Covariance matrix ──────────────────────────
Cov = (X_centered.T @ X_centered) / (n - 1)
print("\nCovariance matrix:\n", Cov.round(4))
# Should be 2×2 and symmetric

# ── Step 3: Eigendecomposition ──────────────────────────
eigenvalues, eigenvectors = np.linalg.eigh(Cov)
# eigh (not eig!) for symmetric matrices:
# - Guarantees real eigenvalues (symmetric matrices always have
real eigenvalues)
# - More numerically stable and faster than eig for symmetric
inputs
# - Returns eigenvalues sorted ascending (we flip to descending in
Step 4)

# ── Step 4: Sort descending ──────────────────────────
idx = np.argsort(eigenvalues)[::-1]  # sort indices descending
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print("\nEigenvalues (variance explained):", eigenvalues.round(4))
print("Principal components (columns):\n", eigenvectors.round(4))

# ── Step 5: Project to 1D (keep top-1 component) ─────────
k = 1
PC = eigenvectors[:, :k]           # top-k eigenvectors
X_projected = X_centered @ PC      # shape: (10, 1)
print("\nProjected data (1D):\n", X_projected.round(4))

# ── Variance explained ──────────────────────────────────
variance_explained = eigenvalues / eigenvalues.sum()
```

```
print("\nVariance explained per component:", (variance_explained *
100).round(1), "%")
# The first PC should explain ~96% of the variance for this dataset
```

PCA with sklearn (Production Use)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=1)
X_reduced = pca.fit_transform(X)

print("Explained variance ratio:", pca.explained_variance_ratio_)
# Should match our manual calculation
```

Why Variance = Information

Variance measures how "spread out" data is along a direction. Low variance means the data points are nearly the same along that axis — there's little information. High variance means points vary a lot — there's rich information.

PCA finds the directions where your data is most "interesting."

Reading a Scree Plot

A **scree plot** shows eigenvalues (variance explained) per component. You look for the "elbow" — where adding more components gives diminishing returns.

```
Variance
100% |■
 80% |■
 60% |  ■
 40% |     ■
 20% |        ■  ■  ■  ■
  0% |_____________________
      PC1 PC2 PC3 PC4 PC5 ...
           ↑
       elbow here → use 3 PCs
```

4.6 SVD vs PCA: What's the Connection?

PCA can be computed via SVD of the centered data matrix directly — without explicitly computing the covariance matrix:

If $\tilde{X} = U\Sigma V^T$, then:

- The principal components are the columns of V (right singular vectors)
- The eigenvalues of the covariance matrix are $\sigma_i^2/(n-1)$

```
# PCA via SVD (numerically more stable for high-dimensional data)
X_centered = X - X.mean(axis=0)
U, sigma, Vt = np.linalg.svd(X_centered, full_matrices=False)

# IMPORTANT: np.linalg.svd returns V-TRANSPOSE (Vt), not V itself.
# The formula is A = U @ diag(sigma) @ Vt
# Principal components are the COLUMNS of V = ROWS of Vt
# So we take Vt.T to get the matrix whose columns are PCs
PC_via_svd = Vt.T   # shape: (n_features, n_features); columns =
principal components
print("Principal components via SVD:\n", PC_via_svd.round(4))

# Variance explained
variance_via_svd = (sigma**2) / (n - 1)
print("Eigenvalues via SVD:", variance_via_svd.round(4))
```

Why sklearn uses SVD internally: For high-dimensional data (thousands of features), computing an explicit $d \times d$ covariance matrix is expensive. SVD on the data matrix is more efficient and numerically stable.

4.7 Engineer's Angle: When to Use Each Technique

Technique	Use when	Don't use when
PCA	Visualizing data, removing correlated features, speeding up downstream models	Data is not linearly structured; interpretability required (PCA components are mixtures)
SVD	Matrix factorization, recommendation systems, text analysis (LSA)	You need interpretable components
Eigendecomposition	Symmetric matrices only; graph algorithms (PageRank); understanding covariance	Non-square matrices

LoRA: Low-Rank Adaptation in LLMs

One of the most important recent applications of SVD principles is **LoRA** (Low-Rank Adaptation), which enables efficient fine-tuning of large language models.

The idea: instead of updating the full weight matrix W (billions of parameters), decompose the update into a low-rank matrix:

$$W_{\text{updated}} = W_{\text{pretrained}} + \Delta W, \quad \text{where} \quad \Delta W = BA$$

with $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times d}$ and $r \ll d$.

Instead of updating $d \times d$ parameters, you update $2 \times d \times r$ — a massive reduction for small r (e.g., $r = 8$ vs $d = 4096$).

This works because: weight updates tend to be low-rank — the "meaningful" directions of change concentrate in a subspace, just like SVD shows that matrices often have low effective rank.

4.8 Full Code Example

```
import numpy as np

np.set_printoptions(precision=4, suppress=True)

# ── Eigendecomposition ──────────────────────────────────────
print("=== Eigendecomposition ===")
A = np.array([[4.0, 1.0], [2.0, 3.0]])
vals, vecs = np.linalg.eig(A)
print("Eigenvalues:", vals)       # [5. 2.]
print("Eigenvectors:\n", vecs)

# Verify Av = λv for each eigenpair
for i in range(len(vals)):
    v = vecs[:, i]
    print(f"λ={vals[i]:.0f}: Av={A@v}, λv={vals[i]*v}, match=
{np.allclose(A@v, vals[i]*v)}")

# ── SVD ─────────────────────────────────────────────────────
print("\n=== SVD ===")
M = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])
U, sigma, Vt = np.linalg.svd(M)
print("Singular values:", sigma)

# Reconstruct
Sigma = np.zeros_like(M)
Sigma[:len(sigma), :len(sigma)] = np.diag(sigma)
M_reconstructed = U @ Sigma @ Vt
print("Reconstruction error:", np.linalg.norm(M - M_reconstructed))
# ~0

# Low-rank approximation (k=1)
k = 1
M_approx = sigma[0] * np.outer(U[:, 0], Vt[0, :])
print("Rank-1 approximation:\n", M_approx)
print("Approximation error:", np.linalg.norm(M -
M_approx).round(4))

# ── PCA from scratch ────────────────────────────────────────
print("\n=== PCA ===")
X = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9],
              [1.9, 2.2], [3.1, 3.0], [2.3, 2.7],
              [2.0, 1.6], [1.0, 1.1], [1.5, 1.6], [1.1, 0.9]])

n = len(X)
X_c = X - X.mean(axis=0)
Cov = X_c.T @ X_c / (n - 1)
vals, vecs = np.linalg.eigh(Cov)

# Sort descending
idx = np.argsort(vals)[::-1]
vals, vecs = vals[idx], vecs[:, idx]

print("Eigenvalues:", vals)
```

```
var_explained = vals / vals.sum() * 100
print("Variance explained:", var_explained.round(1), "%")
print("PC1:", vecs[:, 0].round(4))
print("PC2:", vecs[:, 1].round(4))

# Project to 1D
X_1d = X_c @ vecs[:, :1]
print("Projected shape:", X_1d.shape)  # (10, 1)
```

4.9 Chapter Summary

Concept	Formula	Key Property
Eigenvector/value	$A\mathbf{v} = \lambda\mathbf{v}$	Direction unchanged by A; scaled by λ
Eigendecomposition	$A = Q\Lambda Q^{-1}$	Square matrices with n independent eigenvectors
Symmetric decomp	$A = Q\Lambda Q^T$	$Q^{-1} = Q^T$ for symmetric matrices
SVD	$A = U\Sigma V^T$	Any matrix; singular values sorted descending
PCA	Project onto top-k eigenvectors	Directions of max variance
Low-rank approx	$A \approx U_k \Sigma_k V_k^T$	Best rank-k approximation (Eckart-Young theorem)

Key insights:

- Eigenvectors are the "natural axes" of a matrix transformation
- SVD is the universal version: any matrix, any shape
- PCA = SVD applied to centered data
- Large singular/eigenvalues = important directions; small = noise/redundancy
- LoRA in LLMs exploits the low-rank structure of weight updates

---

Exercises

4.1 Find the eigenvalues of $A = \begin{bmatrix} 2 & 0 \\ 0 & 5 \end{bmatrix}$ without computation. Explain.

Solution: The matrix is diagonal, so the eigenvalues are just the diagonal entries: $\lambda_1 = 2$, $\lambda_2 = 5$. The eigenvectors are $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ — the coordinate axes. Diagonal matrices don't rotate anything, they only scale.

4.2 A covariance matrix has eigenvalues $[25.3, 4.1, 0.8, 0.2]$. How many principal components should you keep to explain at least 95% of the variance?

Solution: Total variance $= 25.3 + 4.1 + 0.8 + 0.2 = 30.4$

- 1 PC: $25.3 / 30.4 = 83.2$ — not enough
- 2 PCs: $(25.3 + 4.1) / 30.4 = 96.7$ ✓

Keep **2 principal components**.

4.3 In the SVD $A = U\Sigma V^T$, what does it mean if the smallest singular value is 0?

Solution: A singular value of 0 means the matrix maps some non-zero input direction to zero — the matrix has a **null space**. The matrix is rank-deficient (not full rank) and therefore singular (no inverse exists). In ML, zero singular values in a feature matrix indicate perfectly linearly dependent features.

4.4 (Challenge) Why does PCA on the centered data $\tilde{X}$ via SVD give the same principal components as the eigendecomposition of the covariance matrix $\Sigma = \frac{1}{n-1}\tilde{X}^T\tilde{X}$?

Solution: If $\tilde{X} = U\Sigma V^T$, then:

$$\tilde{X}^T\tilde{X} = (U\Sigma V^T)^T(U\Sigma V^T) = V\Sigma^T U^T U\Sigma V^T = V\Sigma^T\Sigma V^T = V(\Sigma^2)V^T$$

This is exactly the eigendecomposition of $\tilde{X}^T\tilde{X}$: eigenvectors are the columns of V (right singular vectors of $\tilde{X}$) and eigenvalues are σ_i^2. Dividing by $(n-1)$ gives the covariance matrix eigenvalues $\sigma_i^2/(n-1)$. The eigenvectors are identical. □

Next: Chapter 5 — Calculus. We've described data; now we learn how models improve.

Chapter 5: Calculus I — Derivatives

> *"The derivative tells you which way is downhill. Gradient descent is just repeatedly following that direction."*

5.1 The Concept: What Is a Derivative?

Imagine you're driving on a hilly road. At any point, you can ask: "Am I currently going uphill or downhill, and how steep is it?" The answer is the **derivative**.

More precisely, the derivative of a function f at a point x is the **instantaneous rate of change** of f at x — the slope of the function at that exact point.

This is the foundation of how models learn. The loss function tells you how wrong your model is. The derivative tells you which way to adjust the parameters to make it less wrong.

From Average Rate of Change to Instantaneous

The **average rate of change** of f between x and $x + h$ is:

$$\frac{f(x+h) - f(x)}{h}$$

This is the slope of the line connecting two points. As we make h smaller and smaller, this approaches the **instantaneous rate of change** — the derivative:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

The symbol $\lim_{h \to 0}$ means "as h approaches zero" — we're making the gap infinitesimally small.

Visual intuition:

```
f(x)
   |          ●  ← f(x+h)
   |         /
   |        /  ← slope = [f(x+h)−f(x)]/h
   |       ●  ← f(x)
   |___________ x
             x  x+h

As h→0, the secant line becomes a tangent line — that's the
derivative.
```

5.2 Notation

There are several equivalent ways to write "derivative of f":

Notation	Reads as	Notes
$f'(x)$	"f prime of x"	Lagrange notation — common for single-variable
$\frac{df}{dx}$	"df by dx"	Leibniz notation — shows what's varying
$\frac{d}{dx}f(x)$	"d by dx of f(x)"	Leibniz, explicit
$\dot{f}$	"f dot"	Newton notation — common in physics (time derivatives)

In ML, you'll most often see $\frac{df}{dx}$ or $\frac{\partial f}{\partial x}$ (the partial derivative — Chapter 6).

5.3 Basic Derivative Rules

You don't need to compute limits every time. These rules handle 95% of ML calculus.

Rule 1: Power Rule

$$\frac{d}{dx}x^n = n \cdot x^{n-1}$$

Examples:

Function	Derivative	Shown
x^2	$2x$	$n=2$: $2 \cdot x^{2-1} = 2x$
x^3	$3x^2$	$n=3$: $3 \cdot x^{3-1} = 3x^2$
$x^1 = x$	1	$n=1$: $1 \cdot x^0 = 1$
$x^0 = 1$	0	$n=0$: $0 \cdot x^{-1} = 0$
$x^{-1} = 1/x$	$-x^{-2} = -1/x^2$	$n=-1$
$x^{1/2} = \sqrt{x}$	$\frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}}$	$n=1/2$

Worked example:

$$f(x) = x^4$$

$$f'(x) = 4x^3$$

At $x = 2$: $f'(2) = 4(2)^3 = 32$. The function is rising at a rate of 32 units per unit of x at this point.

Rule 2: Constant Rule

$$\frac{d}{dx}c = 0 \quad \text{(c is any constant)}$$

A constant never changes, so its rate of change is zero.

Rule 3: Constant Multiple Rule

$$\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$$

Example: $\frac{d}{dx}[5x^2] = 5 \cdot 2x = 10x$

Rule 4: Sum/Difference Rule

$$\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$$

Example:

$$\frac{d}{dx}[3x^3 - 2x^2 + 7x - 5] = 9x^2 - 4x + 7$$

(Differentiate term by term; the constant -5 disappears.)

Rule 5: Product Rule

$$\frac{d}{dx}[f(x) \cdot g(x)] = f'(x)g(x) + f(x)g'(x)$$

Memory trick: "First times derivative of second, plus second times derivative of first"

Worked example:

$$h(x) = x^2 \cdot \sin(x)$$

$$h'(x) = 2x \cdot \sin(x) + x^2 \cdot \cos(x)$$

ML use: The product rule appears in attention mechanisms. The scaled dot-product attention score $\text{score}(q, k) = q^T k / \sqrt{d}$ involves a product of two learned vectors; when computing gradients through both q and k, the product rule applies.

Rule 6: Chain Rule (CRITICAL for ML)

The chain rule computes the derivative of a **composed** function.

If $h(x) = f(g(x))$, then:

$$h'(x) = f'(g(x)) \cdot g'(x)$$

In words: "derivative of outer function (evaluated at inner function) times derivative of inner function"

Leibniz notation (clearer for ML):

$$\frac{dh}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Worked example:

$$h(x) = (3x + 1)^5$$

Here $f(u) = u^5$ and $g(x) = 3x + 1$:

- $f'(u) = 5u^4$
- $g'(x) = 3$

$$h'(x) = 5(3x + 1)^4 \cdot 3 = 15(3x + 1)^4$$

Another example with nesting:

$$h(x) = \sin(x^2)$$

- Outer: $f(u) = \sin(u)$, $f'(u) = \cos(u)$
- Inner: $g(x) = x^2$, $g'(x) = 2x$

$$h'(x) = \cos(x^2) \cdot 2x$$

Why the chain rule dominates ML: A neural network is a composition of functions: $\hat{y} = f_L(f_{L-1}(\cdots f_1(\mathbf{x})))$. Backpropagation — the algorithm that trains neural networks — is just the chain rule applied repeatedly from output to input.

5.4 Derivatives of Common Functions

Function	Derivative	Why it matters in ML
$\sin(x)$	$\cos(x)$	Fourier features, positional encodings
$\cos(x)$	$-\sin(x)$	Same
e^x	e^x	**Self-derivative!** Used in softmax, exponential loss
e^{ax}	ae^{ax}	Scaled exponential
$\ln(x)$, $x > 0$	$1/x$	Cross-entropy loss contains ln
$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$	Sigmoid activation, logistic regression

The Sigmoid Derivative — Worked

This appears in every logistic regression and sigmoid-activated network.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Let's compute $\sigma'(x)$ using the quotient rule and chain rule:

$$\sigma(x) = (1 + e^{-x})^{-1}$$

Using chain rule with $u = 1 + e^{-x}$:

$$\frac{d\sigma}{dx} = -1 \cdot (1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Now we simplify using a trick. Note that $\sigma(x) = \frac{1}{1+e^{-x}}$ so $1 - \sigma(x) = \frac{e^{-x}}{1+e^{-x}}$:

$$\frac{d\sigma}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \sigma(x)(1-\sigma(x))$$

The sigmoid derivative is expressible entirely in terms of $\sigma(x)$ itself! This is computationally efficient in backpropagation — you already computed $\sigma(x)$ in the forward pass, so the backward pass needs no new function calls. (Compare to e^x, which is the only function that literally equals its own derivative; sigmoid is different but similarly convenient.)

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_deriv(x):
    s = sigmoid(x)
    return s * (1 - s)

# Test at several points
for x in [-2, 0, 1, 2]:
    s = sigmoid(x)
    ds = sigmoid_deriv(x)
    print(f"x={x:3d}: σ(x)={s:.4f}, σ'(x)={ds:.4f}")
```

Output:

```
x= -2: σ(x)=0.1192, σ'(x)=0.1050
x=  0: σ(x)=0.5000, σ'(x)=0.2500  ← maximum gradient at x=0
x=  1: σ(x)=0.7311, σ'(x)=0.1966
x=  2: σ(x)=0.8808, σ'(x)=0.1050
```

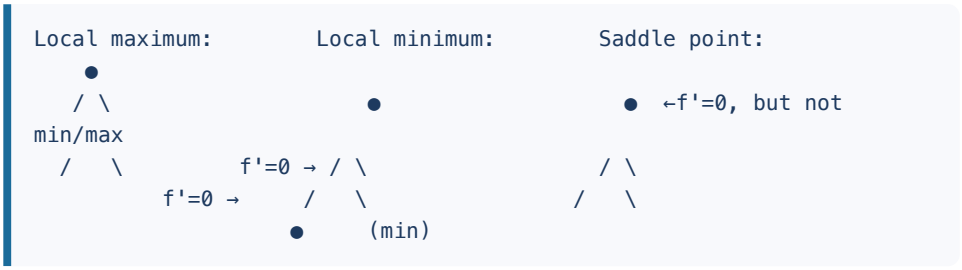
Note: the gradient is maximum (0.25) at $x = 0$ and decreases as $|x|$ grows. This is the root cause of the **vanishing gradient problem** — for very negative or positive pre-activations, the sigmoid gradient is nearly zero, and gradients stop flowing through the network. Chapter 10 covers why ReLU largely replaced sigmoid to address this issue.

5.5 Critical Points: Minima, Maxima, Saddle Points

Training an ML model means **minimizing** a loss function. Calculus tells us where minima live.

Critical points occur where $f'(x) = 0$ — the function is momentarily flat.

Types of Critical Points



For a differentiable function:

- $f'(x) = 0$ AND $f''(x) > 0$: **local minimum** (curve is concave up)
- $f'(x) = 0$ AND $f''(x) < 0$: **local maximum** (curve is concave down)
- $f'(x) = 0$ AND $f''(x) = 0$: **inconclusive** — the second derivative test fails; the point could be a local min, local max, or saddle point. Higher-order analysis is required.

The Second Derivative

The second derivative $f''(x)$ is the derivative of the derivative — it measures how the slope is changing (curvature):

- $f''(x) > 0$: slope is increasing → concave up → like a valley
- $f''(x) < 0$: slope is decreasing → concave down → like a hill

Worked Example

$$f(x) = x^3 - 3x + 2$$

Step 1: Find critical points — set $f'(x) = 0$:

$$f'(x) = 3x^2 - 3 = 0$$

$$3x^2 = 3 \implies x^2 = 1 \implies x = \pm 1$$

Step 2: Classify using second derivative:

$$f''(x) = 6x$$

- At $x = 1$: $f''(1) = 6 > 0 \rightarrow$ **local minimum**. $f(1) = 1 - 3 + 2 = 0$
- At $x = -1$: $f''(-1) = -6 < 0 \rightarrow$ **local maximum**. $f(-1) = -1 + 3 + 2 = 4$

```
import math

def f(x):    return x**3 - 3*x + 2
def fp(x):   return 3*x**2 - 3
def fpp(x):  return 6*x

# Critical points
for x_crit in [-1, 1]:
    print(f"x={x_crit}: f={f(x_crit)}, f'={fp(x_crit)}, f''=
{fpp(x_crit)}")
    typ = "minimum" if fpp(x_crit) > 0 else "maximum"
    print(f"  → {typ}")
```

Why Saddle Points Matter in ML

In high dimensions (millions of weights), true local minima are rare. Most critical points are **saddle points** — they're local minima in some directions but local maxima in others.

Research shows that for large neural networks, most saddle points have similar loss values to the global minimum — so gradient descent naturally finds good solutions even without reaching a true minimum.

5.6 The Chain Rule in Detail: Backpropagation Preview

Let's trace the chain rule through a tiny 2-layer network to preview how backpropagation works.

Network setup:

```
Input x → [Layer 1: z₁ = wx + b] → [Activation: a₁ = σ(z₁)] → [Layer
2: z₂ = v·a₁] → Loss: L = (y − z₂)²
```

We want $\frac{dL}{dw}$ — how the loss changes with respect to weight w in Layer 1. (We omit bias b from the computation below for clarity, but note that $\frac{\partial z_1}{\partial b} = 1$, so the gradient of L with respect to b is the same chain product without the final $\frac{dz_1}{dw} = x$ factor — just $dL/dz_2 \cdot dz_2/da_1 \cdot da_1/dz_1 \cdot 1$.)

Forward pass (compute intermediate values):

- $z_1 = wx$ (linear transformation, bias omitted for clarity)
- $a_1 = \sigma(z_1)$ (sigmoid activation)

- $z_2 = v \cdot a_1$ (second layer)
- $L = (y - z_2)^2$ (loss)

Backward pass (apply chain rule repeatedly):

$$\frac{dL}{dw} = \frac{dL}{dz_2} \cdot \frac{dz_2}{da_1} \cdot \frac{da_1}{dz_1} \cdot \frac{dz_1}{dw}$$

Compute each factor:

1. $\frac{dL}{dz_2} = -2(y - z_2)$
2. $\frac{dz_2}{da_1} = v$
3. $\frac{da_1}{dz_1} = \sigma(z_1)(1 - \sigma(z_1))$
4. $\frac{dz_1}{dw} = x$

$$\frac{dL}{dw} = -2(y - z_2) \cdot v \cdot \sigma(z_1)(1 - \sigma(z_1)) \cdot x$$

Worked numerical example:

Let $x = 1.0$, $w = 0.5$, $v = 2.0$, $y = 3.0$:

```
# Forward pass
x, w, v, y_true = 1.0, 0.5, 2.0, 3.0

z1 = w * x                      # 0.5
a1 = sigmoid(z1)                 # σ(0.5) ≈ 0.6225
z2 = v * a1                      # 2 × 0.6225 ≈ 1.2450
L  = (y_true - z2) ** 2          # (3 - 1.245)² ≈ 3.0806

print(f"z1={z1:.4f}, a1={a1:.4f}, z2={z2:.4f}, L={L:.4f}")

# Backward pass: chain rule
dL_dz2  = -2 * (y_true - z2)                    # ≈ 3.5100
dz2_da1 = v                                       # 2.0
da1_dz1 = sigmoid_deriv(z1)                       # σ(0.5)(1-
σ(0.5)) ≈ 0.2350
dz1_dw  = x                                       # 1.0

dL_dw = dL_dz2 * dz2_da1 * da1_dz1 * dz1_dw
print(f"dL/dw = {dL_dw:.4f}")  # ≈ -1.6498
```

This number $\frac{dL}{dw} \approx -1.65$ tells us: increase w by a tiny amount, and the loss *decreases* by ~1.65 (negative gradient = loss goes down with larger w). To **reduce** the loss, move w in the direction of the negative gradient — in this case, *increase* w. That's gradient descent.

5.7 Numerical Differentiation: Verifying Derivatives

When you're unsure about a derivative, you can approximate it numerically using the definition:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

(Using the symmetric "central difference" formula — more accurate than the one-sided version.)

```
def numerical_gradient(f, x, h=1e-5):
    """Approximate the derivative of f at x using central
differences."""
    return (f(x + h) - f(x - h)) / (2 * h)

# Test on f(x) = x^3 - 3x + 2
f = lambda x: x**3 - 3*x + 2
fp_exact = lambda x: 3*x**2 - 3

for x in [-2, 0, 1, 2]:
    numerical = numerical_gradient(f, x)
    exact     = fp_exact(x)
    print(f"x={x}: numerical={numerical:.6f}, exact={exact:.6f},
diff={abs(numerical-exact):.2e}")
```

Output:

```
x=-2: numerical=9.000000, exact=9.000000, diff=2.84e-11
x= 0: numerical=-3.000000, exact=-3.000000, diff=1.18e-11
x= 1: numerical=0.000000, exact=0.000000, diff=4.44e-16
x= 2: numerical=9.000000, exact=9.000000, diff=4.26e-11
```

This technique — called **gradient checking** — was commonly used to verify backpropagation implementations before automatic differentiation (autograd) became standard.

5.8 Full Code Example

```
import math

# ─── Basic derivatives (numerical verification) ──────────
def num_grad(f, x, h=1e-7):
    return (f(x + h) - f(x - h)) / (2 * h)

print("=== Power Rule Verification ===")
for n in [2, 3, 4]:
    f  = lambda x, n=n: x**n
    fp = lambda x, n=n: n * x**(n-1)
    x = 2.5
    print(f"f(x)=x^{n}: exact f'(x)={fp(x):.4f}, numerical=
{num_grad(f, x):.4f}")

print("\n=== Chain Rule: h(x) = (3x+1)^5 ===")
h    = lambda x: (3*x + 1)**5
hp   = lambda x: 15 * (3*x + 1)**4
x = 0.5
print(f"Exact: {hp(x):.4f}, Numerical: {num_grad(h, x):.4f}")

print("\n=== Sigmoid and its Derivative ===")
def sigmoid(x): return 1 / (1 + math.exp(-x))
def sig_d(x):
    s = sigmoid(x)
    return s * (1 - s)

sig_fn = sigmoid
for x in [-2, -1, 0, 1, 2]:
    s  = sigmoid(x)
    sd = sig_d(x)
    nd = num_grad(sig_fn, x)
    print(f"x={x:2d}: σ={s:.4f}, σ'={sd:.4f}, numerical={nd:.4f}")

print("\n=== Finding minimum of f(x) = x² + 4x + 5 ===")
# Analytical: f'(x) = 2x + 4 = 0 → x = -2
f  = lambda x: x**2 + 4*x + 5
fp = lambda x: 2*x + 4
x_min = -4 / 2  # = -2
print(f"Minimum at x={x_min}, f(x_min)={f(x_min)}")
print(f"f'(x_min) = {fp(x_min)} (should be 0)")

print("\n=== Mini Backprop Demo ===")
x_in, w, v, y_true = 1.0, 0.5, 2.0, 3.0

z1 = w * x_in
a1 = sigmoid(z1)
z2 = v * a1
L  = (y_true - z2)**2

dL_dz2  = -2 * (y_true - z2)
dz2_da1 = v
da1_dz1 = sig_d(z1)
dz1_dw  = x_in
dL_dw   = dL_dz2 * dz2_da1 * da1_dz1 * dz1_dw
```

```
print(f"Forward: z1={z1:.4f}, a1={a1:.4f}, z2={z2:.4f}, L={L:.4f}")
print(f"dL/dw = {dL_dw:.6f}")

# Verify numerically
L_fn = lambda w_val: (y_true - v * sigmoid(w_val * x_in))**2
grad_check = num_grad(L_fn, w)
print(f"Gradient check: {grad_check:.6f}")
print(f"Match: {abs(dL_dw - grad_check) < 1e-4}")
```

5.9 Chapter Summary

Concept	Formula	Use in ML
Derivative definition	$f'(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$	Foundation of all optimization
Power rule	$\frac{d}{dx}x^n = nx^{n-1}$	Polynomial loss functions
Chain rule	$\frac{dh}{dx} = \frac{df}{dg}\frac{dg}{dx}$	Backpropagation
e^x derivative	$\frac{d}{dx}e^x = e^x$	Softmax, exp loss
$\ln(x)$ derivative	$\frac{d}{dx}\ln(x) = 1/x$	Cross-entropy gradient
Sigmoid derivative	$\sigma'(x) = \sigma(x)(1-\sigma(x))$	Logistic regression, RNNs
Critical point	$f'(x) = 0$	Where minima/maxima live
Numerical gradient	$\frac{f(x+h)-f(x-h)}{2h}$	Gradient checking

Key insight: Backpropagation = chain rule applied backward through the network. Every "gradient" in ML is a derivative computed by the chain rule.

Exercises

5.1 Compute the derivative of $f(x) = 4x^3 - 2x^2 + 5x - 7$.

Solution:

$$f'(x) = 12x^2 - 4x + 5$$

(Power rule + sum rule + constant disappears)

5.2 Find all critical points of $f(x) = x^2 - 6x + 9$ and classify them.

Solution:

$$f'(x) = 2x - 6 = 0 \implies x = 3$$

$$f''(x) = 2 > 0 \implies \text{local minimum at } x = 3$$

$$f(3) = 9 - 18 + 9 = 0$$

(This is $(x-3)^2$, a perfect square — minimum value is 0 at $x = 3$.)

5.3 Use the chain rule to differentiate $h(x) = \ln(x^2 + 1)$.

Solution:

- Outer: $f(u) = \ln(u)$, $f'(u) = 1/u$
- Inner: $g(x) = x^2 + 1$, $g'(x) = 2x$

$$h'(x) = \frac{1}{x^2+1} \cdot 2x = \frac{2x}{x^2+1}$$

```
h  = lambda x: math.log(x**2 + 1)
hp = lambda x: 2*x / (x**2 + 1)
x = 3.0
print(f"Exact: {hp(x):.6f}")
print(f"Numerical: {num_grad(h, x):.6f}")
```

5.4 The ReLU activation function is $\text{ReLU}(x) = \max(0, x)$. What is its derivative?

Solution:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$

In practice, the gradient at $x = 0$ is defined to be 0 (or sometimes 1, implementation-dependent). ReLU's derivative is called the **Heaviside step function**. The simplicity of this derivative is a key reason ReLU replaced sigmoid as the default activation — no $\sigma(1-\sigma)$ computation needed, just 0 or 1.

```
def relu(x):    return max(0.0, x)
def relu_d(x):  return 1.0 if x > 0 else 0.0

for x in [-2, -0.1, 0, 0.1, 2]:
    print(f"x={x}: ReLU={relu(x)}, ReLU'={relu_d(x)}")
```

5.5 (Challenge) Show that $\frac{d}{dx}\ln(\sigma(x)) = 1 - \sigma(x)$.

Solution: Using chain rule: $\frac{d}{dx}\ln(\sigma(x)) = \frac{1}{\sigma(x)} \cdot \sigma'(x) = \frac{1}{\sigma(x)} \cdot \sigma(x)(1 - \sigma(x)) = 1 - \sigma(x)$ $\square$

This identity appears in the gradient of the binary cross-entropy loss (Chapter 9).

Next: Chapter 6 — Gradients and Optimization. We extend derivatives to multiple dimensions and show how gradient descent trains models.

Chapter 6: Calculus II — Gradients and Optimization

> *"Gradient descent is not magic. It is calculus applied one step at a time."*

6.1 From Derivatives to Gradients

In Chapter 5, we computed derivatives of functions with **one input** — $f(x)$. But ML models have millions of parameters. We need derivatives with respect to **many variables simultaneously**.

The **gradient** is the generalization of the derivative to multi-variable functions. Instead of a single number $f'(x)$, the gradient is a **vector** of partial derivatives.

If $f : \mathbb{R}^n \to \mathbb{R}$ (takes an n-dimensional vector, returns a scalar):

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1} \; \frac{\partial f}{\partial x_2} \; \vdots \; \frac{\partial f}{\partial x_n} \right]$$

Each component $\frac{\partial f}{\partial x_i}$ is a **partial derivative** — the rate of change of f when only x_i changes, holding all others constant.

6.2 Partial Derivatives

A **partial derivative** differentiates with respect to one variable, treating all others as constants.

Notation: $\frac{\partial f}{\partial x}$ (curly ∂ instead of straight d)

Worked example:

$$f(x, y) = x^2 + 3xy + y^3$$

Partial derivative with respect to x (treat y as a constant):

$$\frac{\partial f}{\partial x} = 2x + 3y$$

Partial derivative with respect to y (treat x as a constant):

$$\frac{\partial f}{\partial y} = 3x + 3y^2$$

At the point $(x, y) = (2, 1)$: $$\frac{\partial f}{\partial x}\bigg|_{(2,1)} = 2(2) + 3(1) = 7$$

$$\frac{\partial f}{\partial y}\bigg|_{(2,1)} = 3(2) + 3(1)^2 = 9$$

So the gradient at $(2, 1)$ is $\nabla f(2, 1) = \begin{bmatrix} 7 & 9 \end{bmatrix}$.

```
import math

def f(x, y):    return x**2 + 3*x*y + y**3
def df_dx(x,y): return 2*x + 3*y
def df_dy(x,y): return 3*x + 3*y**2

x, y = 2.0, 1.0
print(f"f({x},{y}) = {f(x,y)}")       # 4 + 6 + 1 = 11
print(f"∂f/∂x = {df_dx(x,y)}")        # 7
print(f"∂f/∂y = {df_dy(x,y)}")        # 9
print(f"gradient = [{df_dx(x,y)}, {df_dy(x,y)}]")  # [7, 9]
```

6.3 The Gradient as a Direction

The gradient $\nabla f(\mathbf{x})$ is a vector that points in the direction of **steepest ascent** — the direction in which f increases most rapidly.

The **negative gradient** $-\nabla f(\mathbf{x})$ points toward **steepest descent** — the direction of fastest decrease.

Geometric picture (2D landscape):

```
f(x,y) as elevation:

        ▲ gradient points uphill
        |
   _____|_____
  /    hill   \
 /   ●         \
 |   ↗  ← gradient here
 |               \
  \     valley    /
   \_____________/
         ↓ negative gradient points downhill
```

Key properties of the gradient:

1. **Direction:** Points toward steepest ascent
2. **Magnitude:** $|\nabla f|$ tells you how steep the terrain is
3. **Zero gradient:** $\nabla f = \mathbf{0}$ at a critical point (local min, max, or saddle)

Worked example — gradient at two points:

$$f(x, y) = x^2 + y^2 \quad \text{(bowl-shaped, minimum at origin)}$$

$$\nabla f = \begin{bmatrix} 2x & 2y \end{bmatrix}$$

At $(3, 4)$: $\nabla f = \begin{bmatrix} 6 & 8 \end{bmatrix}$ — points outward from origin (uphill)

At $(0, 0)$: $\nabla f = \begin{bmatrix} 0 & 0 \end{bmatrix}$ — zero gradient at the minimum ✓

6.4 Gradient Descent

Gradient descent is the algorithm that trains virtually every ML model. The idea is elegant:

1. Start at some point (random initialization of parameters)
2. Compute the gradient at that point (how to go uphill)
3. Take a small step in the **opposite** direction (downhill)
4. Repeat until you reach a minimum

The Update Rule

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla_\theta L(\theta)$$

- θ — model parameters (weights), a vector
- α — **learning rate** (step size, a small positive number like 0.01)
- $L(\theta)$ — loss function (what we're minimizing)
- $\nabla_\theta L$ — gradient of the loss with respect to parameters

The minus sign is critical: we go **against** the gradient to go downhill.

Worked Example — Minimizing a 1D Function

$$L(w) = w^2 - 4w + 5$$

Exact minimum: $L'(w) = 2w - 4 = 0 \implies w^* = 2$, $L(2) = 4 - 8 + 5 = 1$

Gradient descent simulation ($\alpha = 0.3$, starting at $w = 0$):

Step	w	$L(w)$	$\nabla L = 2w - 4$	Update: $w - 0.3\nabla L$
0	0.000	5.000	-4.000	0 - 0.3(-4) = 1.200
1	1.200	1.640	-1.600	1.2 - 0.3(-1.6) = 1.680
2	1.680	1.102	-0.640	1.68 - 0.3(-0.64) = 1.872
3	1.872	1.016	-0.256	1.872 - 0.3(-0.256) = 1.949
4	1.949	1.003	-0.102	→ converging to 2.0

```
def loss(w):  return w**2 - 4*w + 5
def grad(w):  return 2*w - 4

w = 0.0          # starting point
alpha = 0.3      # learning rate
n_steps = 20

print("Step |   w    |  L(w)  | gradient")
print("-----|--------|--------|----------")
for step in range(n_steps):
    g = grad(w)
    print(f"  {step:2d} | {w:6.4f} | {loss(w):6.4f} | {g:8.4f}")
    w = w - alpha * g
    if abs(g) < 1e-6:
        print(f"Converged at step {step}")
        break

print(f"\nFinal: w = {w:.6f}, L(w) = {loss(w):.6f}")
print(f"True minimum: w = 2.0, L = 1.0")
```

The Learning Rate Effect

The learning rate α is the most important hyperparameter in gradient descent:

Too small: Takes many steps; training is slow.

Too large: Overshoot the minimum; loss oscillates or diverges.

Just right: Converges efficiently.

```
def gradient_descent(w_init, alpha, n_steps=50):
    w = w_init
    losses = []
    for _ in range(n_steps):
        losses.append(loss(w))
        g = grad(w)
        w = w - alpha * g
        if abs(g) < 1e-8:
            break
    return w, losses

# Compare learning rates
for alpha in [0.01, 0.3, 0.9, 1.1]:
    w_final, hist = gradient_descent(0.0, alpha)
    status = "converged" if abs(w_final - 2.0) < 0.01 else
"DIVERGED"
    print(f"α={alpha}: final w={w_final:.4f}, {status}")
```

Output:

```
α=0.01: final w=1.6484, converged slowly
α=0.30: final w=2.0000, converged
α=0.90: final w=2.0000, converged (oscillated)
α=1.10: final w=HUGE, DIVERGED
```

6.5 Variants of Gradient Descent

In practice, computing the gradient over the entire dataset at each step is expensive. Three variants are used:

Batch Gradient Descent

Use **all** training examples to compute the gradient:

$$\nabla_\theta L = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta L^{(i)}$$

- **Pro:** Stable, accurate gradient estimate
- **Con:** Slow for large datasets; must load all data into memory
- **Use:** Small datasets or when exact gradient is critical

Stochastic Gradient Descent (SGD)

Use **one random example** at each step:

$$\nabla_\theta L \approx \nabla_\theta L^{(i)} \quad \text{for random } i$$

- **Pro:** Fast updates; can escape saddle points due to noise
- **Con:** Noisy — loss bounces around; needs learning rate decay
- **Use:** Online learning; very large datasets

Mini-Batch Gradient Descent

Use a **small random batch** (typically 32–512 examples):

$$\nabla_\theta L \approx \frac{1}{B} \sum_{i \in \text{batch}} \nabla_\theta L^{(i)}$$

- **Pro:** Best of both worlds; efficient on GPUs (vectorized)
- **Con:** Adds batch size as a hyperparameter
- **Use: Standard in modern deep learning**

```
import random

def mini_batch_gradient_descent(X, y, w_init, alpha=0.01,
batch_size=32, epochs=100):
    """
    Simple linear regression: prediction = w * x
    Loss = mean squared error

    Note: Uses plain Python lists for clarity. In production, use
numpy arrays
    for vectorized operations (50-100x faster on large datasets).
    """
    w = w_init
    n = len(X)

    for epoch in range(epochs):
        # Shuffle data
        indices = list(range(n))
        random.shuffle(indices)

        for start in range(0, n, batch_size):
            batch_idx = indices[start:start + batch_size]
            X_batch = [X[i] for i in batch_idx]
            y_batch = [y[i] for i in batch_idx]
            B = len(batch_idx)

            # Compute gradient: d(MSE)/dw = 2/B * sum((wx - y) * x)
            grad_w = 2/B * sum((w * X_batch[j] - y_batch[j]) *
X_batch[j]
                               for j in range(B))
            w = w - alpha * grad_w

    return w
```

6.6 The Loss Landscape

Understanding the loss landscape — the "terrain" of $L(\theta)$ over all possible parameters — is key to understanding why training works or fails.

Convex vs Non-Convex

Convex: One global minimum, no local minima. Gradient descent always finds it.

```
Convex (bowl):           Non-convex (hills):
     L                        L
     |    *global min         |   * local  *global
     |   / \                  |  / \ min  / \
     |  /   \                 | /   \    /   \
     |_/     \_               |/     \__/     \_
              θ                                 θ
```

Linear regression has a convex loss (squared error on linear model) — guaranteed to find global minimum.

Neural networks have non-convex loss landscapes. In practice, the many saddle points and local minima have similar loss values, and modern optimizers handle this well.

The Role of Initialization

Where you start matters in non-convex landscapes:

```
# Two runs of gradient descent on a non-convex function
# f(w) = w^4 - 4w^2 + w  (two local minima near w≈-1.4 and w≈1.3)
def f_nonconvex(w):  return w**4 - 4*w**2 + w
def gf_nonconvex(w): return 4*w**3 - 8*w + 1

for w_init in [-2.0, 0.5, 2.0]:
    w = w_init
    for _ in range(1000):
        g = gf_nonconvex(w)
        w -= 0.01 * g
        if abs(g) < 1e-6:
            break
    print(f"Start: {w_init:5.1f} → final w: {w:.4f}, L:
{f_nonconvex(w):.4f}")

# Expected output:
# Start:  -2.0 → final w: -1.4730, L: -5.4442   ← local minimum
# Start:   0.5 → final w:  1.3470, L: -2.6186   ← different local
minimum
# Start:   2.0 → final w:  1.3470, L: -2.6186   ← same local
minimum
# Different starting points → different solutions on non-convex
landscapes
```

6.7 Beyond Vanilla Gradient Descent: Momentum and Adam

Momentum

Vanilla gradient descent "forgets" previous steps — it only looks at the current gradient. **Momentum** adds memory: it accumulates a velocity vector, smoothing out oscillations.

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla L(\theta)$$

$$\theta_t = \theta_{t-1} - \alpha \mathbf{v}_t$$

- $\mathbf{v}$ is the velocity (exponential moving average of gradients)
- β is the momentum coefficient (typically 0.9)

Think of it like a ball rolling down a hill — it builds up speed in consistent directions and doesn't change course sharply on noisy gradients.

Adam (Adaptive Moment Estimation)

Adam is the most widely used optimizer in deep learning. It combines:

- **Momentum:** exponential moving average of gradients ($\mathbf{m}$, first moment)
- **RMSProp:** exponential moving average of squared gradients ($\mathbf{v}$, second moment)

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\nabla L$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)(\nabla L)^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \quad \text{(bias correction)}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

Default hyperparameters: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Why Adam works: It normalizes gradient updates by the historical magnitude of each parameter's gradient. Parameters with historically large gradients get smaller updates; parameters with small gradients get relatively larger updates. This adapts the step size per parameter automatically.

```
def adam_step(grad, m, v, t, alpha=0.001, beta1=0.9, beta2=0.999,
eps=1e-8):
    """Single Adam update step."""
    m_new = beta1 * m + (1 - beta1) * grad
    v_new = beta2 * v + (1 - beta2) * grad**2
    m_hat = m_new / (1 - beta1**t)   # bias correction
    v_hat = v_new / (1 - beta2**t)   # bias correction
    update = alpha * m_hat / (v_hat**0.5 + eps)
    return update, m_new, v_new
```

6.8 The Jacobian and Hessian (Brief Introduction)

For completeness — these appear in optimization literature.

Jacobian

When $f : \mathbb{R}^n \to \mathbb{R}^m$ (vector input, vector output), the derivative is a matrix called the **Jacobian**:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

In ML: the Jacobian of the network output with respect to inputs tells you how sensitive each output is to each input feature — useful for saliency maps and explainability.

Hessian

The **Hessian** H is the matrix of second-order partial derivatives of a scalar function:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

The Hessian tells you about the curvature of the loss landscape. Positive definite Hessian = local minimum. Indefinite Hessian = saddle point.

Why not commonly used in deep learning: The Hessian of a neural network has dimension $(n_\theta \times n_\theta)$ where n_θ is the number of parameters. For a network with 100M parameters, the Hessian would have 10^{16} entries — impossible to compute or store. Gradient descent only needs first-order information.

6.9 Gradient Descent for Linear Regression — Full Example

Let's see everything together on a concrete ML problem: fitting a line to data.

Model: $\hat{y} = wx + b$

Loss (MSE):

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{n} \sum_{i=1}^{n} (wx^{(i)} + b - y^{(i)})^2$$

Gradients:

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}(wx^{(i)} + b - y^{(i)}) \cdot x^{(i)}$$

$$\frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(wx^{(i)} + b - y^{(i)})$$

```
import random
import math

# ── Generate synthetic data: y = 2x + 1 + noise ──────────
random.seed(42)
n = 50
X_data = [i/10.0 for i in range(n)]
y_data = [2.0*x + 1.0 + random.gauss(0, 0.3) for x in X_data]
# True w=2, b=1

# ── Gradient descent ──────────────────────────────────────
w, b = 0.0, 0.0    # initialize at zero
alpha = 0.05       # learning rate
epochs = 200

def mse_loss(w, b):
    return sum((w*x + b - y)**2 for x,y in zip(X_data,y_data)) / n

def gradients(w, b):
    residuals = [w*x + b - y for x,y in zip(X_data,y_data)]
    dw = 2/n * sum(r*x for r,x in zip(residuals, X_data))
    db = 2/n * sum(residuals)
    return dw, db

print(f"Initial: w={w}, b={b}, Loss={mse_loss(w,b):.4f}")

for epoch in range(epochs):
    dw, db = gradients(w, b)
    w -= alpha * dw
    b -= alpha * db

print(f"Final:   w={w:.4f}, b={b:.4f}, Loss={mse_loss(w,b):.4f}")
print(f"True:    w=2.0000, b=1.0000")
```

Expected output:

```
Initial: w=0, b=0, Loss=10.XXXX
Final:   w=1.9XXX, b=1.0XXX, Loss=0.0XXX
True:    w=2.0000, b=1.0000
```

6.10 Chapter Summary

Concept	Formula	Use in ML
Partial derivative	$\frac{\partial f}{\partial x_i}$ (hold others fixed)	Per-parameter gradient
Gradient	$\nabla f = [\partial f/\partial x_1, \ldots, \partial f/\partial x_n]^T$	Direction of steepest ascent
Gradient descent	$\theta \leftarrow \theta - \alpha \nabla L(\theta)$	Training all models
SGD	One example per step	Large datasets
Mini-batch	Batch of B examples per step	Standard in deep learning
Momentum	Velocity accumulation	Smoother convergence
Adam	Adaptive per-parameter LR	Default optimizer for most nets
Jacobian	Matrix of first derivatives for vector functions	Backprop, saliency
Hessian	Matrix of second derivatives	Curvature; impractical for large nets

Key insight: Gradient descent is the engine of all ML. The gradient tells us direction. The learning rate controls the step size. Everything else (Adam, momentum, schedulers) is engineering to make this work better in practice.

Exercises

6.1 Compute the gradient of $f(x, y) = x^2 y + y^3$ at $(x, y) = (1, 2)$.

Solution:

$$\frac{\partial f}{\partial x} = 2xy, \quad \frac{\partial f}{\partial y} = x^2 + 3y^2$$

At $(1, 2)$: $\frac{\partial f}{\partial x} = 2(1)(2) = 4$, $\frac{\partial f}{\partial y} = 1 + 12 = 13$

$$\nabla f(1, 2) = \begin{bmatrix} 4 & 13 \end{bmatrix}$$

6.2 Run 3 steps of gradient descent on $L(w) = (w - 5)^2$ starting at $w = 0$ with $\alpha = 0.4$.

Solution: $L'(w) = 2(w - 5)$

Step	w	$L'(w)$	$w_{\text{new}} = w - 0.4 \cdot L'(w)$
0	0.0	-10.0	0 - 0.4(-10) = 4.0
1	4.0	-2.0	4 - 0.4(-2) = 4.8
2	4.8	-0.4	4.8 - 0.4(-0.4) = 4.96

Converging toward 5.0. ✓

6.3 What is the gradient of $L(w, b) = (wx + b - y)^2$ at $w = 1, b = 0, x = 2, y = 3$?

Solution: Residual $= wx + b - y = 1(2) + 0 - 3 = -1$

$$\frac{\partial L}{\partial w} = 2(wx + b - y) \cdot x = 2(-1)(2) = -4$$

$$\frac{\partial L}{\partial b} = 2(wx + b - y) \cdot 1 = 2(-1) = -2$$

Gradient: $\begin{bmatrix} -4 & -2 \end{bmatrix}$. Gradient descent would increase both w and b (subtract negative gradient), moving toward the true $w = 1.5, b = 0$ line passing through $(2, 3)$.

6.4 Why does Adam divide the gradient update by $\sqrt{\hat{\mathbf{v}}_t} + \epsilon$?

Solution: $\hat{\mathbf{v}}_t$ is an estimate of the squared gradient magnitude. Dividing by $\sqrt{\hat{\mathbf{v}}_t}$ normalizes the update so parameters with historically large gradients (high $\hat{v}$) receive smaller updates, and parameters with small gradients receive relatively larger updates. This acts as an adaptive, per-parameter learning rate. The ϵ prevents division by zero when $\hat{v}$ is near zero.

6.5 (Challenge) Show that for $L(w) = \frac{1}{n}\sum_{i=1}^{n}(wx_i - y_i)^2$, the gradient is:

$$\frac{dL}{dw} = \frac{2}{n}\sum_{i=1}^{n}(wx_i - y_i)x_i$$

Solution: Apply the chain rule to each term:

$$\frac{d}{dw}(wx_i - y_i)^2 = 2(wx_i - y_i) \cdot \frac{d}{dw}(wx_i - y_i) = 2(wx_i - y_i) \cdot x_i$$

Summing over all n examples and dividing by n:

$$\frac{dL}{dw} = \frac{1}{n}\sum_{i=1}^{n} 2(wx_i - y_i)x_i = \frac{2}{n}\sum_{i=1}^{n}(wx_i - y_i)x_i \quad \square$$

Next: Chapter 7 — Probability Foundations. We've covered how data is represented and how models improve. Now: what the model actually predicts.

Chapter 7: Probability I — Foundations

> *"All models are wrong, but some are useful. Probability theory is how we quantify exactly how wrong."*

7.1 Sample Spaces, Events, and Outcomes

Plain English First

When something random happens — flipping a coin, classifying an image, measuring sensor noise — there is a set of all the things that *could* happen. Probability theory starts by forcing you to write that set down explicitly. Once you have the full menu of possibilities, you can assign numbers (probabilities) to subsets of it.

Three vocabulary words matter:

- **Outcome**: a single, specific result of one experiment (e.g., "heads", "image is a cat", "temperature is 98.6°F").
- **Sample space** Ω: the complete set of all possible outcomes. Nothing outside Ω can happen.
- **Event**: a collection (subset) of outcomes you care about (e.g., "I got a number greater than 4 on a die").

Think of the sample space as an enum in code — the complete, exhaustive list of variants. An event is a predicate that matches some of those variants.

Formal Notation

Let Ω be the **sample space** — a non-empty set. Any **outcome** $\omega \in \Omega$ is a single element. An **event** A is a subset $A \subseteq \Omega$.

Symbol	Meaning	Code analogy
Ω	Sample space (all outcomes)	`enum` or `set` of all variants
ω	A single outcome	One enum value
$A \subseteq \Omega$	An event	`{x for x in outcomes if predicate(x)}`
$A^c = \Omega \setminus A$	Complement event ("not A")	`outcomes - A` in Python
$A \cup B$	Union event ("A or B")	`A | B`
$A \cap B$	Intersection event ("A and B")	`A & B`

Worked Example 7.1.1 — A Fair Die

The experiment: roll one fair six-sided die.

$$\Omega = 1, 2, 3, 4, 5, 6$$

Define events:

- A = "roll an even number" $= 2, 4, 6$
- B = "roll a number greater than 4" $= 5, 6$
- $A \cap B$ = "even AND greater than 4" $= 6$
- $A \cup B$ = "even OR greater than 4" $= 2, 4, 5, 6$
- A^c = "not even" $= 1, 3, 5$

Verification: $|A| + |A^c| = 3 + 3 = 6 = |\Omega|$. Correct.

Worked Example 7.1.2 — Email Classification

The experiment: a classifier observes one email and assigns a label.

$$\Omega = \text{spam}, \text{ham}$$

- Event S = "email is spam" $= \text{spam}$
- Event S^c = "email is not spam" $= \text{ham}$

For a document classifier with three classes:

$$\Omega = \text{positive}, \text{negative}, \text{neutral}$$

The event "model is wrong on a negative example" is $A = \text{positive}, \text{neutral}$ — everything except the correct label.

Engineer's Angle

In ML, the sample space is often implicit. When you define a classification task with K classes, you are implicitly defining:

$$\Omega = 0, 1, 2, \ldots, K-1$$

A model's output (a probability vector of length K) assigns a number to each singleton event ω. The requirement that "they all sum to 1" is exactly the third Kolmogorov axiom — coming in the next section.

```
# Sample space and events as Python sets
omega = {1, 2, 3, 4, 5, 6}
A = {2, 4, 6}          # even numbers
B = {5, 6}             # greater than 4

intersection = A & B   # {6}
union = A | B          # {2, 4, 5, 6}
complement_A = omega - A  # {1, 3, 5}

print(f"A ∩ B = {sorted(intersection)}")   # [6]
print(f"A ∪ B = {sorted(union)}")          # [2, 4, 5, 6]
print(f"A^c   = {sorted(complement_A)}")   # [1, 3, 5]
print(f"|A| + |A^c| == |Ω|: {len(A) + len(complement_A) ==
len(omega)}")  # True
```

7.2 Probability Axioms (Kolmogorov)

Plain English First

We want to assign a "likelihood number" to every event. For that assignment to be internally consistent — so we never end up with paradoxes like "the probability of something happening is 120%" — we need three ground rules. Andrey Kolmogorov wrote these down in 1933 and they haven't changed since. Everything else in probability follows from these three rules.

Formal Notation

A **probability measure** P is a function that assigns a real number to every event. It must satisfy three axioms:

Axiom 1 (Non-negativity):

$$P(A) \geq 0 \quad \text{for every event } A$$

Axiom 2 (Normalization):

$$P(\Omega) = 1$$

Axiom 3 (Countable Additivity):
If A and B are **mutually exclusive** (disjoint: $A \cap B = \emptyset$), then:

$$P(A \cup B) = P(A) + P(B)$$

More generally, for any countable collection of pairwise disjoint events $A_1, A_2, \ldots$:

$$P!\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Trust this result: The entire edifice of probability — Bayes' theorem, expected value, everything — is derived from these three axioms.

Key Consequences Derived from the Axioms

Here's why the complement rule holds: $P(A^c) = 1 - P(A)$

Since A and A^c are disjoint and their union is Ω:

$$P(A \cup A^c) = P(A) + P(A^c) \quad \text{(by Axiom 3)}$$

$$P(\Omega) = P(A) + P(A^c) \quad \text{(since } A \cup A^c = \Omega\text{)}$$

$$1 = P(A) + P(A^c) \quad \text{(by Axiom 2)}$$

$$\therefore \quad P(A^c) = 1 - P(A) \quad \square$$

Here's why the inclusion-exclusion rule holds: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Write $A \cup B$ as three disjoint pieces:

$$A \cup B = (A \setminus B) \cup (A \cap B) \cup (B \setminus A)$$

By Axiom 3: $P(A \cup B) = P(A \setminus B) + P(A \cap B) + P(B \setminus A)$

Also: $P(A) = P(A \setminus B) + P(A \cap B)$ and $P(B) = P(B \setminus A) + P(A \cap B)$

Adding: $P(A) + P(B) = P(A \setminus B) + 2P(A \cap B) + P(B \setminus A)$

Subtracting $P(A \cap B)$: $P(A) + P(B) - P(A \cap B) = P(A \cup B)$ $\square$

Worked Example 7.2.1 — Die Probabilities

For a fair six-sided die, each outcome is equally likely:

$$P(k) = \frac{1}{6} \quad \text{for } k = 1, 2, 3, 4, 5, 6$$

Verify Axiom 2:

$$P(\Omega) = P(1) + P(2) + \cdots + P(6) = \frac{1}{6} \times 6 = 1 \checkmark$$

Compute $P(A)$ where $A = 2, 4, 6$:

Since 2, 4, 6 are disjoint, by Axiom 3:

$$P(A) = P(2) + P(4) + P(6) = \frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6} = \frac{1}{2}$$

Complement: $P(A^c) = 1 - \frac{1}{2} = \frac{1}{2}$. Indeed $A^c = 1, 3, 5$ also has three elements.

Inclusion-exclusion with $B = 5, 6$:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) = \frac{3}{6} + \frac{2}{6} - \frac{1}{6} = \frac{4}{6} = \frac{2}{3}$$

Cross-check: $A \cup B = 2, 4, 5, 6$ has 4 elements, so $P(A \cup B) = \frac{4}{6} = \frac{2}{3}$. ✓

Engineer's Angle: Axioms in Softmax

A softmax layer in a neural network takes a vector of raw scores (logits) $\mathbf{z} \in \mathbb{R}^K$ and outputs:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

This construction is deliberately engineered to satisfy all three Kolmogorov axioms:

- **Axiom 1:** $e^{z_k} > 0$ always, so each output is positive.
- **Axiom 2:** The sum equals $\frac{\sum_k e^{z_k}}{\sum_j e^{z_j}} = 1$.
- **Axiom 3:** Disjoint classes, non-overlapping.

```python
import math

def softmax(logits):
    # Subtract max for numerical stability
    max_z = max(logits)
    exps = [math.exp(z - max_z) for z in logits]
    total = sum(exps)
    return [e / total for e in exps]

logits = [2.1, 0.5, -0.3]
probs = softmax(logits)
print(f"Probabilities: {[round(p,4) for p in probs]}")
print(f"Sum: {sum(probs):.6f}")    # Axiom 2: must be 1.0
print(f"All >= 0: {all(p >= 0 for p in probs)}")  # Axiom 1
```

7.3 Conditional Probability

Plain English First

Conditional probability answers: "Given that I already know B happened, how likely is A?" Learning new information shrinks the universe of possibilities. If you know the first die came up 5, you no longer care about the other 35 outcomes — your world has collapsed to just the 6 outcomes where the first die shows 5.

Formal Notation

The **conditional probability** of A given B (assuming $P(B) > 0$) is:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Read as: "the fraction of B's probability mass that also belongs to A."

Rearranging (this is the **multiplication rule**, used constantly):

$$P(A \cap B) = P(A \mid B) \cdot P(B)$$

Worked Example 7.3.1 — Two Dice

Roll two fair dice. Let:

- B = "first die shows 5" $\Rightarrow P(B) = \frac{6}{36} = \frac{1}{6}$

- A = "sum equals 8"

Find $P(A \mid B)$.

Step 1 — Find $A \cap B$: pairs where first die = 5 AND sum = 8.

$$\text{Need second die} = 8 - 5 = 3 \Rightarrow (5, 3) \text{ only}$$

$$P(A \cap B) = \frac{1}{36}$$

Step 2 — Apply the formula:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{1/36}{6/36} = \frac{1/36}{1/6} = \frac{1}{36} \times \frac{6}{1} = \frac{6}{36} = \frac{1}{6}$$

Sanity check: Given that first die = 5, the second die can be 1–6 equally. Sum = 8 requires second = 3. That's 1 outcome out of 6. So $P(A \mid B) = \frac{1}{6}$. ✓

Worked Example 7.3.2 — Email Spam

Suppose 30% of emails are spam ($P(S) = 0.3$), and 80% of spam emails contain the word "win" ($P(\text{"win"} \mid S) = 0.8$). What fraction of *all* emails are spam emails containing "win"?

This is a multiplication rule question:

$$P(S \cap \text{"win"}) = P(\text{"win"} \mid S) \cdot P(S) = 0.8 \times 0.3 = 0.24$$

24% of all emails are spam emails that contain "win". Note this is NOT the same as $P(S \mid \text{"win"})$ — that requires Bayes' theorem (Section 7.5).

Engineer's Angle: Generative vs. Discriminative Models

Every supervised ML model is essentially trying to learn one of two things:

- **Discriminative:** learn $P(y \mid \mathbf{x})$ directly — given features $\mathbf{x}$, predict the label y. (Logistic regression, neural networks)
- **Generative:** learn $P(\mathbf{x}, y) = P(\mathbf{x} \mid y) \cdot P(y)$ — model how features are generated given a class. (Naive Bayes, GMMs)

The conditional probability $P(y \mid \mathbf{x})$ is the *fundamental object* of classification.

```python
from fractions import Fraction

# P(S) = spam prior, P("win"|S) = likelihood
p_spam = Fraction(3, 10)      # 30%
p_win_given_spam = Fraction(8, 10)  # 80%

# Multiplication rule
p_spam_and_win = p_win_given_spam * p_spam
print(f"P(spam ∩ 'win') = {p_win_given_spam} × {p_spam} =
{p_spam_and_win}")
# = 24/100 = 6/25

# Verify conditional probability definition
p_B = Fraction(6, 36)   # P(first die = 5)
p_AB = Fraction(1, 36)  # P(sum=8 AND first die=5)
p_A_given_B = p_AB / p_B
print(f"P(sum=8 | first=5) = {p_A_given_B}")  # 1/6
```

7.4 Independence of Events

Plain English First

Two events are independent if knowing one happened gives you zero information about whether the other happened. Flipping a coin twice: the second flip doesn't care what the first flip did. This is distinct from events being *mutually exclusive* (which means knowing one happened tells you the other definitely didn't happen — the opposite of independent).

Independence is one of the most important — and most abused — assumptions in ML.

Formal Notation

Events A and B are **independent** if and only if:

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalently (when $P(B) > 0$):

$$P(A \mid B) = P(A)$$

Here's why these are equivalent: if $P(A \cap B) = P(A) \cdot P(B)$, then:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} = \frac{P(A) \cdot P(B)}{P(B)} = P(A)$$

Knowing B occurred doesn't change A's probability. □

Events $A_1, A_2, \ldots, A_n$ are **mutually independent** if for every subset $A_{i_1}, \ldots, A_{i_k}$:

$$P(A_{i_1} \cap \cdots \cap A_{i_k}) = P(A_{i_1}) \cdots P(A_{i_k})$$

Pairwise independence does NOT imply mutual independence — but mutual independence implies pairwise.

Worked Example 7.4.1 — Two Coin Flips

Flip a fair coin twice. Sample space:

$$\Omega = HH, HT, TH, TT, \quad P(\text{each}) = \frac{1}{4}$$

Let A = "first flip is H" $= HH, HT$, $P(A) = \frac{2}{4} = \frac{1}{2}$

Let B = "second flip is H" $= HH, TH$, $P(B) = \frac{2}{4} = \frac{1}{2}$

Check independence:

$$P(A \cap B) = P(HH) = \frac{1}{4}$$

$$P(A) \cdot P(B) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

Since $P(A \cap B) = P(A) \cdot P(B)$, the events are **independent**. ✓

Intuitively: the second flip doesn't "remember" the first.

Worked Example 7.4.2 — Dependent Events

Roll one die. Let A = "result is even" $= 2, 4, 6$ and B = "result is greater than 4" $= 5, 6$.

$$P(A) = \frac{3}{6} = \frac{1}{2}, \quad P(B) = \frac{2}{6} = \frac{1}{3}$$

$$P(A \cap B) = P(6) = \frac{1}{6}$$

$$P(A) \cdot P(B) = \frac{1}{2} \times \frac{1}{3} = \frac{1}{6}$$

These happen to be independent! (The outcome being even gives no information about whether it exceeds 4, on a fair die.)

Now let C = "result is greater than 3" $= 4, 5, 6$, $P(C) = \frac{3}{6} = \frac{1}{2}$.

$$P(A \cap C) = P(4, 6) = \frac{2}{6} = \frac{1}{3}$$

$$P(A) \cdot P(C) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$$

Since $\frac{1}{3} \neq \frac{1}{4}$, events A and C are **dependent**. Knowing the die shows more than 3 increases the chance it's even (from $\frac{1}{2}$ to $\frac{2}{3}$).

Engineer's Angle: The Naive Bayes Assumption

Naive Bayes classifiers classify documents by computing:

$$P(y \mid x_1, x_2, \ldots, x_n) \propto P(y) \cdot \prod_{i=1}^{n} P(x_i \mid y)$$

The word "Naive" refers to the assumption that features $x_1, \ldots, x_n$ are **mutually independent** given the class y. In spam filtering, this means "the probability of seeing 'free' and 'winner' together equals the product of their individual probabilities, given the email is spam."

This is usually **false** in the real world — "free winner" is more common than chance would predict. Yet Naive Bayes often works surprisingly well in practice despite this violated assumption. This is an important engineering lesson: a wrong assumption that leads to tractable computation can outperform a correct model that's too expensive to use.

```python
from fractions import Fraction

# Test for independence: P(A ∩ B) == P(A) * P(B)?
omega = {1, 2, 3, 4, 5, 6}
A = {2, 4, 6}
B = {5, 6}

p = lambda event: Fraction(len(event), len(omega))

p_A = p(A)
p_B = p(B)
p_AB = p(A & B)

print(f"P(A) = {p_A}")
print(f"P(B) = {p_B}")
print(f"P(A ∩ B) = {p_AB}")
print(f"P(A)*P(B) = {p_A * p_B}")
print(f"Independent: {p_AB == p_A * p_B}")  # True
```

7.5 Bayes' Theorem

Plain English First

Bayes' theorem is the mathematical rule for *updating your beliefs* when you get new evidence. Before seeing evidence: you have a **prior** belief ($P(\text{hypothesis})$). After seeing evidence: you compute a **posterior** belief ($P(\text{hypothesis} \mid \text{evidence})$). The theorem tells you exactly how to do this update.

It is arguably the most important formula in machine learning.

Derivation — "Here's why"

Start from the **symmetry of joint probability**. The joint event "A and B" can be factored two ways using the multiplication rule:

$$P(A \cap B) = P(A \mid B) \cdot P(B) \quad \text{...(i)}$$

$$P(A \cap B) = P(B \mid A) \cdot P(A) \quad \text{...(ii)}$$

Since both equal $P(A \cap B)$, set them equal:

$$P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

Divide both sides by $P(B)$ (assuming $P(B) > 0$):

$$\boxed{P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}}$$

This is **Bayes' theorem**. □

The names for each piece:

- $P(A)$: **prior** — your belief before seeing evidence
- $P(B \mid A)$: **likelihood** — probability of the evidence given the hypothesis
- $P(B)$: **marginal likelihood** or **evidence** — how probable the evidence is overall
- $P(A \mid B)$: **posterior** — your updated belief after seeing evidence

Worked Example 7.5.1 — Medical Test (the Classic)

A disease affects 1 in 1,000 people. A test for it has:

- **Sensitivity:** $P(+ \mid \text{disease}) = 0.99$ (detects 99% of real cases)
- **False positive rate:** $P(+ \mid \text{no disease}) = 0.05$ (5% false alarms)

You test positive. What is $P(\text{disease} \mid +)$?

Step 1 — State the priors:

$$P(D) = \frac{1}{1000} = 0.001, \qquad P(D^c) = \frac{999}{1000} = 0.999$$

Step 2 — State the likelihoods:

$$P(+ \mid D) = 0.99, \qquad P(+ \mid D^c) = 0.05$$

Step 3 — Compute $P(+)$ using the total probability theorem (derived in Section 7.6):

$$\begin{aligned} P(+) &= P(+ \mid D) \cdot P(D) + P(+ \mid D^c) \cdot P(D^c) \\ &= 0.99 \times 0.001 + 0.05 \times 0.999 \\ &= 0.00099 + 0.04995 = 0.05094 \end{aligned}$$

Step 4 — Apply Bayes:

$$P(D \mid +) = \frac{P(+ \mid D) \cdot P(D)}{P(+)} = \frac{0.99 \times 0.001}{0.05094} = \frac{0.00099}{0.05094}$$

Arithmetic (exact fractions):

$$P(D) = \frac{1}{1000}, \quad P(+ \mid D) = \frac{99}{100}$$

$$\text{Numerator} = \frac{99}{100} \times \frac{1}{1000} = \frac{99}{100000}$$

$$P(D^c) = \frac{999}{1000}, \quad P(+ \mid D^c) = \frac{5}{100}$$

$$P(+ \mid D^c) \times P(D^c) = \frac{5}{100} \times \frac{999}{1000} = \frac{4995}{100000}$$

$$P(+) = \frac{99}{100000} + \frac{4995}{100000} = \frac{5094}{100000} = \frac{2547}{50000}$$

$$P(D \mid +) = \frac{99/100000}{2547/50000} = \frac{99}{100000} \times \frac{50000}{2547} = \frac{99 \times 50000}{100000 \times 2547} = \frac{4950000}{25470000}$$

P(D \mid +) = \frac{11}{566} \approx 0.0194 = \mathbf{1.94\%}

Interpretation: Even with a 99% sensitive test, a positive result means you have less than a 2% chance of actually having the disease. Why? Because the disease is so rare (0.1 prevalence) that the 5% false positive rate generates far more false alarms than real cases.

This result shocks most people and is the core of why medical tests need to be interpreted in context.

Worked Example 7.5.2 — Naive Bayes Spam Classification

An email arrives with words "free" and "meeting". Prior probabilities and likelihoods:

$$P(S) = 0.4, \quad P(H) = 0.6$$

$$P(\text{"free"} \mid S) = 0.6, \quad P(\text{"free"} \mid H) = 0.1$$

$$P(\text{"meeting"} \mid S) = 0.05, \quad P(\text{"meeting"} \mid H) = 0.5$$

Under the Naive Bayes (independence) assumption:

$$P(S \mid \text{email}) \propto P(S) \cdot P(\text{"free"} \mid S) \cdot P(\text{"meeting"} \mid S)$$

$$= 0.4 \times 0.6 \times 0.05 = 0.012$$

$$P(H \mid \text{email}) \propto P(H) \cdot P(\text{"free"} \mid H) \cdot P(\text{"meeting"} \mid H)$$

$$= 0.6 \times 0.1 \times 0.5 = 0.030$$

Normalize (these are proportional, not the actual posteriors yet):

$$P(S \mid \text{email}) = \frac{0.012}{0.012 + 0.030} = \frac{0.012}{0.042} = \frac{12}{42} = \frac{2}{7} \approx 0.286$$

$$P(H \mid \text{email}) = \frac{0.030}{0.042} = \frac{30}{42} = \frac{5}{7} \approx 0.714$$

Decision: Classify as ham (probability 71.4%). The word "meeting" was a strong ham signal that outweighed "free".

Engineer's Angle

```
from fractions import Fraction

# --- Medical Test ---
p_D  = Fraction(1, 1000)
p_Dc = 1 - p_D
p_pos_D  = Fraction(99, 100)    # sensitivity
p_pos_Dc = Fraction(5, 100)     # false positive rate

# Total probability
p_pos = p_pos_D * p_D + p_pos_Dc * p_Dc
# Bayes
p_D_given_pos = (p_pos_D * p_D) / p_pos

print(f"P(+)             = {p_pos} ≈ {float(p_pos):.5f}")
print(f"P(disease | +)   = {p_D_given_pos} ≈
{float(p_D_given_pos):.5f}")

# --- Naive Bayes spam ---
p_S = Fraction(4, 10)
p_H = Fraction(6, 10)

p_free_S    = Fraction(6, 10)
p_free_H    = Fraction(1, 10)
p_meet_S    = Fraction(5, 100)
p_meet_H    = Fraction(5, 10)

score_S = p_S * p_free_S * p_meet_S
score_H = p_H * p_free_H * p_meet_H
total   = score_S + score_H

print(f"\nP(spam | email)  = {score_S}/{total} = {score_S/total} ≈
{float(score_S/total):.4f}")
print(f"P(ham  | email)  = {score_H}/{total} = {score_H/total} ≈
{float(score_H/total):.4f}")
```

7.6 Total Probability Theorem

Plain English First

Sometimes you want $P(B)$ but you don't know it directly. However, you do know how likely B is within each "category" of some partition. The total probability theorem says: take each category's probability of B, weight it by how common that category is, and add them up.

This is exactly how you compute the denominator in Bayes' theorem.

Formal Notation

If events $A_1, A_2, \ldots, A_n$ form a **partition** of Ω (mutually exclusive, collectively exhaustive: $A_i \cap A_j = \emptyset$ for $i \neq j$ and $\bigcup_i A_i = \Omega$), then for any event B:

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i) \cdot P(A_i)$$

Here's why this works: since the A_i partition Ω, we can write:

$$B = B \cap \Omega = B \cap \left(\bigcup_{i=1}^{n} A_i \right) = \bigcup_{i=1}^{n} (B \cap A_i)$$

The sets $B \cap A_i$ are disjoint (since A_i are disjoint), so by Axiom 3:

$$P(B) = \sum_{i=1}^{n} P(B \cap A_i) = \sum_{i=1}^{n} P(B \mid A_i) \cdot P(A_i) \quad \square$$

Worked Example 7.6.1 — Spam Word Frequency

Three types of emails: spam (30%), newsletters (50%), personal (20%). Probability the word "win" appears:

- $P(\text{"win"} \mid \text{spam}) = 0.80$
- $P(\text{"win"} \mid \text{newsletter}) = 0.15$
- $P(\text{"win"} \mid \text{personal}) = 0.05$

Total probability of seeing "win" in a randomly chosen email:

$$P(\text{"win"}) = 0.80 \times 0.30 + 0.15 \times 0.50 + 0.05 \times 0.20$$

Arithmetic:

$$= 0.24 + 0.075 + 0.010 = 0.325$$

Cross-check: the weighted average should be between the lowest (0.05) and highest (0.80) likelihood. 0.325 is in that range. ✓

Worked Example 7.6.2 — Mixture Model (ML Connection)

A Gaussian Mixture Model (GMM) assumes data is generated from K Gaussian components with mixing weights π_k. The marginal density of an observation x is:

$$p(x) = \sum_{k=1}^{K} \pi_k \cdot p(x \mid \text{component } k)$$

This is the total probability theorem in continuous form. The "partition" is over the latent variable (which component generated the point).

Engineer's Angle

```python
from fractions import Fraction

# Partition: spam, newsletter, personal
types      = ['spam', 'newsletter', 'personal']
priors     = [Fraction(3,10), Fraction(5,10), Fraction(2,10)]
likelihoods = [Fraction(8,10), Fraction(15,100), Fraction(5,100)]

# Verify partition
assert sum(priors) == 1, "Priors must sum to 1"

# Total probability
p_win = sum(lk * pr for lk, pr in zip(likelihoods, priors))
print(f"P('win') = {p_win} = {float(p_win):.4f}")  # 13/40 = 0.325

# Cross-check
terms = [(lk, pr, lk*pr) for lk, pr in zip(likelihoods, priors)]
for t, (lk, pr, product) in zip(types, terms):
    print(f"  P('win'|{t}) × P({t}) = {lk} × {pr} = {product}")
```

7.7 Random Variables

Plain English First

A **random variable** is a function that assigns a number to every outcome in the sample space. Instead of working with abstract outcomes like "the email is spam", we work with numbers — which we can do math on. A random variable turns qualitative events into quantities.

There are two flavors:

- **Discrete:** takes a countable set of values (e.g., number of errors in a batch, class label $0/1/2$)
- **Continuous:** takes any value in an interval (e.g., a model's confidence score, a pixel intensity)

Formal Notation

A **random variable** X is a function $X : \Omega \to \mathbb{R}$.

Discrete random variable: X takes values in $x_1, x_2, \ldots$. Characterized by its **probability mass function (PMF)**:

$$p_X(x) = P(X = x)$$

Requirements: $p_X(x) \geq 0$ and $\sum_x p_X(x) = 1$ (Kolmogorov's axioms).

Continuous random variable: X takes values in an interval. Characterized by its **probability density function (PDF)** $f_X(x)$ where:

$$P(a \leq X \leq b) = \int_a^b f_X(x), dx$$

Trust this result: The integral $\int_a^b f_X(x), dx$ computes the area under the curve of f_X between a and b — think of it as the sum of infinitely many thin slivers, each of width dx and height $f_X(x)$. For Python programmers, it's like `sum(f(x) * dx for x in range(a, b, dx))` as `dx` approaches zero.

Requirements for PDF: $f_X(x) \geq 0$ and $\int_{-\infty}^{\infty} f_X(x), dx = 1$.

Note: for a continuous RV, $P(X = x) = 0$ for any single point — probability lives in *intervals*, not points.

Worked Example 7.7.1 — Discrete: Die as a Random Variable

Define X = value shown by a fair die. The PMF is:

$$p_X(k) = \frac{1}{6} \quad \text{for } k \in 1, 2, 3, 4, 5, 6$$

Verify: $\sum_{k=1}^{6} \frac{1}{6} = \frac{6}{6} = 1$. ✓

We can also define derived RVs: Let $Y = \mathbf{1}[X \text{ is even}]$ (indicator of even outcome):

$$P(Y = 1) = P(X \in 2, 4, 6) = \frac{3}{6} = \frac{1}{2}, \qquad P(Y = 0) = \frac{1}{2}$$

Worked Example 7.7.2 — Discrete: Bernoulli and Binomial PMFs

A **Bernoulli** random variable $X \sim \text{Bern}(p)$ models a single trial:

$$P(X = 1) = p, \qquad P(X = 0) = 1 - p$$

In ML: $Y_i = 1$ (correct prediction) or $Y_i = 0$ (wrong prediction) with $p =$ model accuracy.

A **Binomial** $X \sim \text{Bin}(n, p)$ counts successes in n independent Bernoulli trials:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Example: $n = 4$ test examples, $p = 0.7$ accuracy. What is $P(X = 3)$ (exactly 3 correct)?

$$P(X = 3) = \binom{4}{3} (0.7)^3 (0.3)^1 = 4 \times 0.343 \times 0.3 = 4 \times 0.1029 = 0.4116$$

Verify: $4 \times 0.343 = 1.372$; $1.372 \times 0.3 = 0.4116$. ✓

Worked Example 7.7.3 — Continuous: Uniform Distribution

$X \sim \text{Uniform}(0, 1)$ has PDF:

$$f_X(x) = \begin{cases} 1 & 0 \le x \le 1 \\ 0 & \text{otherwise} \end{cases}$$

Verify: $\int_0^1 1 \, dx = 1$. ✓ (Area of a rectangle with width 1 and height 1.)

$$P(0.2 \le X \le 0.7) = \int_{0.2}^{0.7} 1 \, dx = 0.7 - 0.2 = 0.5$$

This represents: "a randomly initialized neural network weight (drawn uniformly) has a 50% chance of landing in the interval $[0.2, 0.7]$."

Engineer's Angle

```python
import math
from fractions import Fraction

# --- Discrete PMF: die ---
def pmf_die(k):
    return Fraction(1, 6) if 1 <= k <= 6 else Fraction(0)

values = range(1, 7)
total = sum(pmf_die(k) for k in values)
print(f"Die PMF sums to: {total}")  # 1

# --- Binomial PMF ---
def comb(n, k):
    return math.comb(n, k)

def binomial_pmf(k, n, p):
    return comb(n, k) * (p**k) * ((1-p)**(n-k))

n, p = 4, 0.7
prob_3 = binomial_pmf(3, n, p)
print(f"P(X=3) with n=4, p=0.7: {prob_3:.4f}")  # 0.4116

# Verify all probabilities sum to 1
total_bin = sum(binomial_pmf(k, n, p) for k in range(n+1))
print(f"Binomial PMF sums to: {total_bin:.6f}")  # 1.000000

# --- Continuous Uniform: approximate with fine grid ---
def uniform_pdf(x, a=0.0, b=1.0):
    return 1.0 / (b - a) if a <= x <= b else 0.0

# Riemann sum for P(0.2 <= X <= 0.7)
dx = 0.0001
xs = [0.2 + i * dx for i in range(int((0.7 - 0.2) / dx))]
approx_prob = sum(uniform_pdf(x) * dx for x in xs)
print(f"P(0.2 <= X <= 0.7) ≈ {approx_prob:.4f}")  # ≈ 0.5000
```

7.8 Expected Value and Variance

Plain English First

The **expected value** is the long-run average of a random variable — if you repeated the experiment millions of times, what would you expect the

average to be? It's the "center of gravity" of the probability distribution.

The **variance** measures how spread out the values are around that center. High variance means the random variable jumps around a lot; low variance means it's predictable.

In ML, expected value is the **expected loss** (the quantity we minimize during training), and variance is related to **model uncertainty** and the noise in stochastic gradient descent.

Formal Notation

Expected value for a discrete RV:

$$\mathbb{E}[X] = \sum_{x} x \cdot P(X = x)$$

Expected value for a continuous RV:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f_X(x), dx$$

Variance:

$$\text{Var}(X) = \mathbb{E}!\left[(X - \mathbb{E}[X])^2\right]$$

Computational shortcut (derived below):

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Standard deviation: $\sigma_X = \sqrt{\text{Var}(X)}$

Key properties (Trust these results — each follows from linearity of expectation):

- $\mathbb{E}[aX + b] = a, \mathbb{E}[X] + b$
- $\text{Var}(aX + b) = a^2\text{Var}(X)$
- If X and Y are independent: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$

Here's why the shortcut formula holds:

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2 - 2\mu X + \mu^2]$$

By linearity of expectation ($\mu = \mathbb{E}[X]$ is a constant):

$$= \mathbb{E}[X^2] - 2\mu, \mathbb{E}[X] + \mu^2 = \mathbb{E}[X^2] - 2\mu^2 + \mu^2 = \mathbb{E}[X^2] - \mu^2 \quad \square$$

Worked Example 7.8.1 — Fair Die: E[X] and Var(X)

Let X = outcome of a fair die.

Step 1 — Expected value:

$$\mathbb{E}[X] = \sum_{k=1}^{6} k \cdot \frac{1}{6} = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = \frac{7}{2} = 3.5$$

Step 2 — Compute $\mathbb{E}[X^2]$:

$$\mathbb{E}[X^2] = \frac{1}{6}(1^2+2^2+3^2+4^2+5^2+6^2) = \frac{1}{6}(1+4+9+16+25+36) = \frac{91}{6}$$

Step 3 — Variance via shortcut:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \frac{91}{6} - \left(\frac{7}{2}\right)^2 = \frac{91}{6} - \frac{49}{4}$$

Convert to common denominator (12):

$$= \frac{182}{12} - \frac{147}{12} = \frac{35}{12} \approx 2.917$$

Step 4 — Verify via definitional formula $\text{Var}(X) = \sum_k P(X=k)(k-\mu)^2$:

k	$k - 3.5$	$(k-3.5)^2$	$\frac{1}{6}(k-3.5)^2$
1	-2.5	6.25	25/24
2	-1.5	2.25	9/24
3	-0.5	0.25	1/24
4	0.5	0.25	1/24
5	1.5	2.25	9/24
6	2.5	6.25	25/24
Sum			**70/24 = 35/12**

Both methods give $\text{Var}(X) = \frac{35}{12}$. ✓

$\sigma_X = \sqrt{35/12} \approx \sqrt{2.917} \approx 1.708$

Worked Example 7.8.2 — Expected Loss

A model makes 4 predictions with losses $\ell_1 = 0.5$, $\ell_2 = 0.2$, $\ell_3 = 0.8$, $\ell_4 = 0.3$, all equally likely. Define L = loss on a random sample:

$$\mathbb{E}[L] = \frac{1}{4}(0.5 + 0.2 + 0.8 + 0.3) = \frac{1.8}{4} = 0.45$$

This is the **empirical risk** — the average loss over a finite sample. Training a model minimizes $\mathbb{E}[L]$ (or an approximation of it over the training set).

Worked Example 7.8.3 — Continuous: Uniform[0,1]

For $X \sim \text{Uniform}(0, 1)$:

$$\mathbb{E}[X] = \int_0^1 x \cdot 1 \, dx = \left[\frac{x^2}{2}\right]_0^1 = \frac{1}{2}$$

$$\mathbb{E}[X^2] = \int_0^1 x^2 \cdot 1 \, dx = \left[\frac{x^3}{3}\right]_0^1 = \frac{1}{3}$$

$$\text{Var}(X) = \frac{1}{3} - \left(\frac{1}{2}\right)^2 = \frac{1}{3} - \frac{1}{4} = \frac{4}{12} - \frac{3}{12} = \frac{1}{12}$$

Trust this result: The definite integral $\int_a^b x^n \, dx = \left[\frac{x^{n+1}}{n+1}\right]_a^b$ — plug in b and subtract plugging in a.

Engineer's Angle

Expected value = **average loss** (cross-entropy, MSE). Variance matters for:

1. **Bias-variance tradeoff:** $\text{MSE} = \text{Bias}^2 + \text{Variance}$ — models can fail by being wrong on average OR by being inconsistently wrong.
2. **SGD noise:** Mini-batch gradient estimates have high variance; reducing batch size increases variance of gradient estimates.

```python
from fractions import Fraction

# --- Die expected value and variance ---
p = Fraction(1, 6)
values = [Fraction(k) for k in range(1, 7)]

E_X  = sum(v * p for v in values)
E_X2 = sum(v**2 * p for v in values)
Var  = E_X2 - E_X**2

print(f"E[X]    = {E_X}  = {float(E_X)}")            # 7/2 = 3.5
print(f"E[X²]   = {E_X2} ≈ {float(E_X2):.4f}")      # 91/6 ≈
15.1667
print(f"Var(X)  = {Var}  ≈ {float(Var):.4f}")        # 35/12 ≈
2.9167

# Verify via definitional formula
mu = E_X
Var_def = sum(p * (v - mu)**2 for v in values)
print(f"Var(X) definitional = {Var_def} = {float(Var_def):.4f}")
print(f"Both methods match: {Var == Var_def}")

# --- Expected loss ---
losses = [0.5, 0.2, 0.8, 0.3]
E_loss = sum(losses) / len(losses)
print(f"\nE[Loss] = {E_loss}")  # 0.45

# --- Approximate E[X] and Var[X] for Uniform(0,1) ---
dx = 0.00001
xs = [i * dx for i in range(int(1/dx) + 1)]
E_approx  = sum(x * 1.0 * dx for x in xs)
E2_approx = sum(x**2 * 1.0 * dx for x in xs)
Var_approx = E2_approx - E_approx**2
print(f"\nUniform(0,1): E[X] ≈ {E_approx:.4f} (exact: 0.5)")
print(f"Uniform(0,1): Var(X) ≈ {Var_approx:.4f} (exact:
{1/12:.4f})")
```

7.9 Joint, Marginal, and Conditional Distributions

Plain English First

So far we've worked with one random variable at a time. In ML we almost always have many variables at once: features $x_1, x_2, \ldots, x_n$ and a label y. To reason about multiple variables together we need **joint distributions** — the probabilities of all combinations of values.

From the joint you can "zoom out" to a single variable (**marginal distribution**) by averaging over everything else, or "zoom in" to one variable

given a fixed value of another (**conditional distribution**).

Formal Notation

For two discrete RVs X and Y, the **joint PMF** is:

$$p_{X,Y}(x, y) = P(X = x, Y = y)$$

The **marginal PMF** of X is obtained by summing over all values of Y:

$$p_X(x) = \sum_y p_{X,Y}(x, y)$$

The **conditional PMF** of X given $Y = y$ is:

$$p_{X|Y}(x \mid y) = \frac{p_{X,Y}(x, y)}{p_Y(y)} \quad (\text{when } p_Y(y) > 0)$$

For continuous RVs, replace sums with integrals:

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y), dy$$

$$f_{X|Y}(x \mid y) = \frac{f_{X,Y}(x, y)}{f_Y(y)}$$

Independence in terms of distributions: X and Y are independent iff:

$$p_{X,Y}(x, y) = p_X(x) \cdot p_Y(y) \quad \text{for all } x, y$$

Worked Example 7.9.1 — Joint Distribution Table

Let X = model correct? (0=no, 1=yes) and Y = data augmented? (0=no, 1=yes).

Observed joint distribution over many experiments:

\begin{array}{c|cc|c} & Y=0 & Y=1 & \text{Marginal } P(X=x) \ \hline X=0 & 2/10 & 1/10 & 3/10 \ X=1 & 3/10 & 4/10 & 7/10 \ \hline \text{Marginal } P(Y=y) & 5/10 & 5/10 & 1 \ \end{array}

Step 1 — Verify: $\frac{2}{10} + \frac{1}{10} + \frac{3}{10} + \frac{4}{10} = \frac{10}{10} = 1$. ✓

Step 2 — Marginal of X:

$$P(X = 0) = P(X = 0, Y = 0) + P(X = 0, Y = 1) = \frac{2}{10} + \frac{1}{10} = \frac{3}{10}$$

$$P(X = 1) = \frac{3}{10} + \frac{4}{10} = \frac{7}{10}$$

Step 3 — Marginal of Y:

$$P(Y=0) = \frac{2}{10} + \frac{3}{10} = \frac{5}{10} = \frac{1}{2}$$

$$P(Y=1) = \frac{1}{10} + \frac{4}{10} = \frac{5}{10} = \frac{1}{2}$$

Step 4 — Conditional $P(X=1 \mid Y=0)$:

$$P(X=1 \mid Y=0) = \frac{P(X=1, Y=0)}{P(Y=0)} = \frac{3/10}{5/10} = \frac{3}{5} = 0.6$$

Interpretation: when no augmentation is used, the model is correct 60% of the time.

Step 5 — Check independence:

If X and Y were independent: $P(X=1, Y=0) = P(X=1) \cdot P(Y=0) = \frac{7}{10} \times \frac{5}{10} = \frac{35}{100} = \frac{7}{20}$.

But $P(X=1, Y=0) = \frac{3}{10} = \frac{6}{20} \neq \frac{7}{20}$.

So X and Y are **not independent** — augmentation is associated with higher accuracy.

Worked Example 7.9.2 — Chain Rule of Probability

The chain rule generalizes conditional probability to many variables. For three RVs:

$$P(X, Y, Z) = P(X \mid Y, Z) \cdot P(Y \mid Z) \cdot P(Z)$$

In an autoregressive language model generating tokens $w_1, w_2, w_3, \ldots$:

$$P(w_1, w_2, w_3) = P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2)$$

This factorization is exact (no independence assumptions needed). Language models are trained to predict each $P(w_t \mid w_1, \ldots, w_{t-1})$.

Engineer's Angle: P(x, y) Factorizations

Every supervised ML model implicitly or explicitly models the joint distribution $P(\mathbf{x}, y)$, which can be factored in two ways:

$$P(\mathbf{x}, y) = P(y \mid \mathbf{x}) \cdot P(\mathbf{x}) \quad \text{(discriminative decomposition)}$$

$$P(\mathbf{x}, y) = P(\mathbf{x} \mid y) \cdot P(y) \quad \text{(generative decomposition)}$$

Discriminative models (logistic regression, SVMs, neural networks) skip modeling $P(\mathbf{x})$ and learn $P(y \mid \mathbf{x})$ directly. Generative models (Naive Bayes, VAEs) learn $P(\mathbf{x} \mid y)$ and $P(y)$, then use Bayes to get $P(y \mid \mathbf{x})$.

```
from fractions import Fraction

# Joint distribution table
joint = {
    (0, 0): Fraction(2, 10),
    (0, 1): Fraction(1, 10),
    (1, 0): Fraction(3, 10),
    (1, 1): Fraction(4, 10),
}

# Marginals
def marginal_X(x):
    return sum(v for (xi, yi), v in joint.items() if xi == x)

def marginal_Y(y):
    return sum(v for (xi, yi), v in joint.items() if yi == y)

# Conditional
def conditional_X_given_Y(x, y):
    return joint[(x, y)] / marginal_Y(y)

print("Marginals:")
for x in [0, 1]: print(f"  P(X={x}) = {marginal_X(x)}")
for y in [0, 1]: print(f"  P(Y={y}) = {marginal_Y(y)}")

print("\nConditionals given Y=0:")
for x in [0, 1]:
    print(f"  P(X={x}|Y=0) = {conditional_X_given_Y(x, 0)}")

# Check independence
print("\nIndependence check:")
for x in [0, 1]:
    for y in [0, 1]:
        product = marginal_X(x) * marginal_Y(y)
        actual  = joint[(x, y)]
        print(f"  P(X={x},Y={y}): joint={actual}, product=
{product}, match={actual==product}")
```

7.10 Summary Table

Concept	Formula	ML Application
Sample space Ω	Complete set of outcomes	Output classes of a classifier
Event $A \subseteq \Omega$	Subset of outcomes	"Prediction is correct"
Complement	$P(A^c) = 1 - P(A)$	P(wrong) = 1 - accuracy
Inclusion-exclusion	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$	Union of error types
Axiom 2 (normalization)	$P(\Omega) = 1$	Softmax outputs sum to 1
Conditional probability	$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$	$P(\text{label} \mid \text{features})$
Multiplication rule	$P(A \cap B) = P(A \mid B) \cdot P(B)$	Chain rule in language models
Independence	$P(A \cap B) = P(A) \cdot P(B)$	Naive Bayes feature assumption
Bayes' theorem	$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$	Posterior over parameters, spam filter
Total probability	$P(B) = \sum_i P(B \mid A_i) \cdot P(A_i)$	Marginal likelihood in mixture models
PMF	$p_X(x) = P(X = x)$	Discrete class probabilities
PDF	$P(a \le X \le b) = \int_a^b f_X(x)\,dx$	Continuous output distributions
Expected value	$\mathbb{E}[X] = \sum_x x\, p_X(x)$	Expected (average) loss
Variance shortcut	$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$	Model uncertainty, SGD noise
Joint PMF	$P(X = x, Y = y)$	Joint dist. of features and labels
Marginal PMF	$p_X(x) = \sum_y p_{X,Y}(x, y)$	Class-marginal feature distribution

Conditional dist.	$p_{X\|Y}(x \mid y) = p_{X,Y}(x,y)/p_Y(y)$	Generative model likelihood

7.11 Exercises

Work through these before looking at the solutions. Each solution is fully worked out.

Exercise 7.1 [Easy] — Sample Space and Events

Two fair coins are flipped. Define the sample space Ω explicitly. Let A = "at least one head" and B = "exactly one head".

(a) List A, B, $A \cap B$, $A \cup B$, A^c.
(b) Compute $P(A)$, $P(B)$, $P(A \cap B)$, $P(A \cup B)$.
(c) Verify the inclusion-exclusion formula for $P(A \cup B)$.

Solution:

(a) $\Omega = HH, HT, TH, TT$, each with probability $\frac{1}{4}$.

$$A = HH, HT, TH \quad \text{(at least one H)}$$

$$B = HT, TH \quad \text{(exactly one H)}$$

$$A \cap B = HT, TH \quad \text{(at least one H AND exactly one H = exactly one H)}$$

$$A \cup B = HH, HT, TH = A \quad \text{(since } B \subseteq A\text{)}$$

$$A^c = TT$$

(b)

$$P(A) = \frac{3}{4}, \quad P(B) = \frac{2}{4} = \frac{1}{2}$$

$$P(A \cap B) = \frac{2}{4} = \frac{1}{2}$$

$$P(A \cup B) = P(A) = \frac{3}{4} \quad \text{(since } B \subseteq A\text{)}$$

(c) Inclusion-exclusion:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) = \frac{3}{4} + \frac{2}{4} - \frac{2}{4} = \frac{3}{4} \checkmark$$

Exercise 7.2 [Easy] — Conditional Probability

A bag has 3 red and 2 blue marbles. You draw one marble without looking, then draw a second (without replacement).

(a) What is $P(\text{2nd is red} \mid \text{1st is red})$?
(b) What is $P(\text{both red})$?
(c) What is $P(\text{2nd is red})$ using the total probability theorem?

Solution:

(a) After drawing a red marble first, the bag has 2 red and 2 blue (4 total):

$$P(\text{2nd red} \mid \text{1st red}) = \frac{2}{4} = \frac{1}{2}$$

(b) Multiplication rule:

$$P(\text{both red}) = P(\text{2nd red} \mid \text{1st red}) \cdot P(\text{1st red}) = \frac{2}{4} \times \frac{3}{5} = \frac{6}{20} = \frac{3}{10}$$

Verify: $\binom{3}{2} / \binom{5}{2} = 3/10$. ✓

(c) Partition on the first draw:

$$P(\text{2nd red}) = P(\text{2nd red} \mid \text{1st red}) \cdot P(\text{1st red}) + P(\text{2nd red} \mid \text{1st blue}) \cdot P(\text{1}$$

After drawing a blue first: 3 red, 1 blue (4 total), so $P(\text{2nd red} \mid \text{1st blue}) = \frac{3}{4}$.

$$P(\text{2nd red}) = \frac{2}{4} \times \frac{3}{5} + \frac{3}{4} \times \frac{2}{5} = \frac{6}{20} + \frac{6}{20} = \frac{12}{20} = \frac{3}{5}$$

Sanity check: $P(\text{2nd red}) = \frac{3}{5}$ — by symmetry, the second marble is equally likely to be any of the 5 original marbles, so the probability it's red is $\frac{3}{5}$. ✓

Exercise 7.3 [Medium] — Bayes' Theorem

A factory has three machines: Machine A produces 50% of widgets (2% defective), Machine B produces 30% (5% defective), Machine C produces 20%

(10% defective).

A widget is chosen at random and found to be defective. What is the probability it came from Machine C?

Solution:

Let D = defective, and let A, B, C denote the machine of origin.

Given:

$$P(A) = 0.5, \quad P(B) = 0.3, \quad P(C) = 0.2$$

$$P(D \mid A) = 0.02, \quad P(D \mid B) = 0.05, \quad P(D \mid C) = 0.10$$

Step 1 — Total probability (probability of a random widget being defective):

$$P(D) = P(D \mid A) \cdot P(A) + P(D \mid B) \cdot P(B) + P(D \mid C) \cdot P(C)$$

$$= 0.02 \times 0.5 + 0.05 \times 0.3 + 0.10 \times 0.2$$

$$= 0.010 + 0.015 + 0.020 = 0.045$$

Step 2 — Bayes' theorem:

$$P(C \mid D) = \frac{P(D \mid C) \cdot P(C)}{P(D)} = \frac{0.10 \times 0.20}{0.045} = \frac{0.020}{0.045} = \frac{20}{45} = \frac{4}{9} \approx 0.444$$

Verification (all three machines):

$$P(A \mid D) = \frac{0.010}{0.045} = \frac{2}{9} \approx 0.222$$

$$P(B \mid D) = \frac{0.015}{0.045} = \frac{3}{9} = \frac{1}{3} \approx 0.333$$

$$P(C \mid D) = \frac{0.020}{0.045} = \frac{4}{9} \approx 0.444$$

Sum: $\frac{2}{9} + \frac{3}{9} + \frac{4}{9} = \frac{9}{9} = 1$. ✓

Machine C produces only 20% of widgets but accounts for 44% of defects — its high defect rate makes it the most likely culprit.

Exercise 7.4 [Medium] — Expected Value and Variance

A fair four-sided die (values 1, 2, 3, 4) is rolled. Let X = outcome.

(a) Compute $\mathbb{E}[X]$.
(b) Compute $\text{Var}(X)$ using the shortcut formula. Verify with the definitional

formula.

(c) If $Y = 3X - 2$, compute $\mathbb{E}[Y]$ and $\text{Var}(Y)$ using the linear transformation rules.

Solution:

Each outcome has probability $\frac{1}{4}$.

(a)

$$\mathbb{E}[X] = \frac{1}{4}(1 + 2 + 3 + 4) = \frac{10}{4} = \frac{5}{2} = 2.5$$

(b) **Shortcut:**

$$\mathbb{E}[X^2] = \frac{1}{4}(1^2 + 2^2 + 3^2 + 4^2) = \frac{1}{4}(1 + 4 + 9 + 16) = \frac{30}{4} = \frac{15}{2}$$

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \frac{15}{2} - \left(\frac{5}{2}\right)^2 = \frac{15}{2} - \frac{25}{4} = \frac{30}{4} - \frac{25}{4} = \frac{5}{4} = 1.$$

Definitional check (with $\mu = \frac{5}{2}$):

k	$k - 2.5$	$(k - 2.5)^2$	$\frac{1}{4}(k - 2.5)^2$
1	-1.5	2.25	9/16
2	-0.5	0.25	1/16
3	0.5	0.25	1/16
4	1.5	2.25	9/16
Sum			**20/16 = 5/4**

✓ Both give $\text{Var}(X) = \frac{5}{4}$.

(c) Linear transformations with $Y = 3X - 2$:

$$\mathbb{E}[Y] = 3, \mathbb{E}[X] - 2 = 3 \times \frac{5}{2} - 2 = \frac{15}{2} - 2 = \frac{11}{2} = 5.5$$

$$\text{Var}(Y) = 3^2 \cdot \text{Var}(X) = 9 \times \frac{5}{4} = \frac{45}{4} = 11.25$$

Exercise 7.5 [Hard] — Joint Distribution and Naive Bayes

Three words appear in emails: "invoice", "meeting", "sale". The joint distribution of word presence and email type (work vs. promo) is given by

the following joint counts over 1,000 emails:

	Work emails	Promo emails
Contains "invoice" only	200	10
Contains "meeting" only	300	20
Contains "sale" only	50	300
None of the above	50	70
Total	**600**	**400**

(a) Compute the marginal probabilities $P(\text{work})$ and $P(\text{promo})$.
(b) Compute the likelihoods $P(\text{"sale"} \mid \text{work})$ and $P(\text{"sale"} \mid \text{promo})$.
(c) A new email contains the word "sale". Use Bayes' theorem to classify it.
(d) Now assume you use a Naive Bayes classifier that also observed "meeting" in the email. Compute $P(\text{work} \mid \text{"meeting"}, \text{"sale"})$ under the Naive Bayes assumption.

Solution:

(a) From totals:

$$P(\text{work}) = \frac{600}{1000} = \frac{3}{5}, \qquad P(\text{promo}) = \frac{400}{1000} = \frac{2}{5}$$

(b) Within work emails: "sale" appears in 50 of 600.

$$P(\text{"sale"} \mid \text{work}) = \frac{50}{600} = \frac{1}{12} \approx 0.0833$$

Within promo emails: "sale" appears in 300 of 400.

$$P(\text{"sale"} \mid \text{promo}) = \frac{300}{400} = \frac{3}{4} = 0.75$$

(c) Total probability:

$$P(\text{"sale"}) = P(\text{"sale"} \mid \text{work}) \cdot P(\text{work}) + P(\text{"sale"} \mid \text{promo}) \cdot P(\text{promo})$$

$$= \frac{1}{12} \times \frac{3}{5} + \frac{3}{4} \times \frac{2}{5} = \frac{3}{60} + \frac{6}{20} = \frac{1}{20} + \frac{6}{20} = \frac{7}{20}$$

Bayes:

$$P(\text{work} \mid \text{"sale"}) = \frac{P(\text{"sale"} \mid \text{work}) \cdot P(\text{work})}{P(\text{"sale"})} = \frac{1/12 \times 3/5}{7/20} = \frac{1/20}{7/20} = \frac{1}{7}$$

$$P(\text{promo} \mid \text{"sale"}) = 1 - \frac{1}{7} = \frac{6}{7} \approx 0.857$$

Classify as promo.

(d) Compute $P(\text{"meeting"} \mid \text{work})$ and $P(\text{"meeting"} \mid \text{promo})$:

$$P(\text{"meeting"} \mid \text{work}) = \frac{300}{600} = \frac{1}{2}$$

$$P(\text{"meeting"} \mid \text{promo}) = \frac{20}{400} = \frac{1}{20}$$

Naive Bayes scores (proportional to posterior, assuming independence of words given class):

$$\text{score}(\text{work}) = P(\text{work}) \cdot P(\text{"meeting"} \mid \text{work}) \cdot P(\text{"sale"} \mid \text{work})$$

$$= \frac{3}{5} \times \frac{1}{2} \times \frac{1}{12} = \frac{3}{120} = \frac{1}{40}$$

$$\text{score}(\text{promo}) = P(\text{promo}) \cdot P(\text{"meeting"} \mid \text{promo}) \cdot P(\text{"sale"} \mid \text{promo})$$

$$= \frac{2}{5} \times \frac{1}{20} \times \frac{3}{4} = \frac{6}{400} = \frac{3}{200}$$

Normalize:

$$\text{total} = \frac{1}{40} + \frac{3}{200} = \frac{5}{200} + \frac{3}{200} = \frac{8}{200} = \frac{1}{25}$$

$$P(\text{work} \mid \text{"meeting"}, \text{"sale"}) = \frac{1/40}{1/25} = \frac{25}{40} = \frac{5}{8} = 0.625$$

$$P(\text{promo} \mid \text{"meeting"}, \text{"sale"}) = \frac{3/200}{1/25} = \frac{3 \times 25}{200} = \frac{75}{200} = \frac{3}{8} = 0.375$$

Classify as work (probability 62.5%). Adding "meeting" flipped the classification from promo to work — the word "meeting" is a very strong work signal ($P(\text{"meeting"} \mid \text{work}) = 50$ vs. $P(\text{"meeting"} \mid \text{promo}) = 5$).

Verify: $\frac{5}{8} + \frac{3}{8} = 1$. ✓

Next: Chapter 8 — Probability II: Key Distributions, where we build on these foundations to study the distributions that appear everywhere in ML: Gaussian, Bernoulli, Categorical, Poisson, and more.

Chapter 8: Probability II — Key Distributions

> *"Knowing which distribution fits your problem is half the battle. The other half is knowing why."*

Chapter 7 gave you the machinery: sample spaces, probability axioms, Bayes' theorem, random variables, expected value, variance, and the PMF/PDF framework. This chapter puts that machinery to work. We tour the handful of distributions that appear constantly in machine learning — and for each one, we answer three questions: What does it model? What are its formulas? Where does it show up in code?

8.1 Bernoulli Distribution

Plain English First

The Bernoulli distribution is the simplest possible random variable: one trial, two outcomes — success or failure, 1 or 0, yes or no. A single fair coin flip follows a Bernoulli distribution. So does a single binary classifier prediction: either the model says "positive" (1) or "negative" (0).

The distribution has exactly one parameter, p, which is the probability of the success outcome. Everything else follows from that single number.

Formal Notation

Let X be a Bernoulli random variable with parameter $p \in [0, 1]$. We write $X \sim \text{Bernoulli}(p)$.

PMF:

$$P(X = k) = \begin{cases} p & \text{if } k = 1 \\ 1 - p & \text{if } k = 0 \end{cases}$$

A compact single-formula version: $P(X = k) = p^k(1-p)^{1-k}$ for $k \in 0, 1$.

Expected value:

$$\mathbb{E}[X] = 1 \cdot p + 0 \cdot (1-p) = p$$

Variance:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = p - p^2 = p(1-p)$$

Here's why $\mathbb{E}[X^2] = p$: since X only takes values 0 or 1, we have $X^2 = X$, so $\mathbb{E}[X^2] = \mathbb{E}[X] = p$.

Quantity	Value
Support	$0, 1$
Parameter	$p \in [0, 1]$ (success probability)
PMF	$P(X = 1) = p$, ; $P(X = 0) = 1 - p$
$\mathbb{E}[X]$	p
$\text{Var}(X)$	$p(1-p)$

Trust this result: Variance $p(1-p)$ is maximized at $p = 0.5$ (most uncertain) and equals zero at $p = 0$ or $p = 1$ (completely certain). This makes intuitive sense.

Worked Example 8.1.1 — A Biased Coin

A biased coin lands heads with probability $p = 0.7$. Define $X = 1$ for heads, $X = 0$ for tails.

Step 1 — PMF:

$$P(X = 1) = 0.7, \qquad P(X = 0) = 0.3$$

Step 2 — Verify normalization:

$$0.7 + 0.3 = 1.0 \checkmark$$

Step 3 — Expected value:

$$\mathbb{E}[X] = 0.7$$

Step 4 — Variance:

$$\text{Var}(X) = 0.7 \times (1 - 0.7) = 0.7 \times 0.3 = 0.21$$

Worked Example 8.1.2 — Binary Classifier Output

A spam classifier outputs probability $\hat{y} = 0.82$ for a given email. We model the predicted label $\hat{L} \sim \text{Bernoulli}(0.82)$.

$$P(\hat{L} = 1) = 0.82, \qquad P(\hat{L} = 0) = 0.18$$

If the true label is $y = 1$, the **binary cross-entropy loss** for this prediction is:

$$\begin{aligned}\mathcal{L} &= -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \\ &= -[1 \cdot \log(0.82) + 0 \cdot \log(0.18)] \\ &= -\log(0.82) \\ &\approx 0.1985\end{aligned}$$

For a wrong prediction $\hat{y} = 0.3$ with true label $y = 1$:

$$\mathcal{L} = -\log(0.3) \approx 1.2040$$

The loss is higher when the model is confidently wrong — exactly the behavior we want. We will derive where this formula comes from in Section 8.10.

Engineer's Angle

The Bernoulli distribution underlies **logistic regression**. The sigmoid function maps a real-valued score s to a probability $\hat{y} = \sigma(s) = \frac{1}{1+e^{-s}}$, and then the predicted label is $\hat{L} \sim \text{Bernoulli}(\hat{y})$. Training minimizes the expected binary cross-entropy loss over the dataset.

```python
import math

def bernoulli_pmf(k, p):
    """PMF of Bernoulli(p) at outcome k in {0, 1}."""
    if k == 1:
        return p
    elif k == 0:
        return 1 - p
    else:
        raise ValueError("k must be 0 or 1")

def bernoulli_mean(p):
    return p

def bernoulli_var(p):
    return p * (1 - p)

def sigmoid(s):
    return 1.0 / (1.0 + math.exp(-s))

def binary_cross_entropy(y_true, y_pred):
    """Loss for one sample under Bernoulli model."""
    return -(y_true * math.log(y_pred) + (1 - y_true) * math.log(1
- y_pred))

# Biased coin example
p = 0.7
print(f"Bernoulli(p=0.7): E={bernoulli_mean(p)}, Var=
{bernoulli_var(p)}")
# Bernoulli(p=0.7): E=0.7, Var=0.21

# Logistic regression workflow
score = 1.5                       # raw model score (logit)
y_hat = sigmoid(score)            # predicted probability
loss  = binary_cross_entropy(1, y_hat)  # true label = 1
print(f"score={score}, y_hat={y_hat:.4f}, loss={loss:.4f}")
# score=1.5, y_hat=0.8176, loss=0.2014
```

8.2 Binomial Distribution

Plain English First

If the Bernoulli distribution models one coin flip, the Binomial distribution models n independent flips and asks: how many heads? More concretely: if you run 10 independent trials each with success probability p, the Binomial distribution tells you the probability of getting exactly k successes.

The word "independent" is critical — each trial cannot depend on the previous ones. This assumption is what lets us multiply probabilities.

Formal Notation

Let X = number of successes in n independent Bernoulli(p) trials. We write $X \sim \text{Binomial}(n, p)$.

PMF:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k = 0, 1, 2, \ldots, n$$

where the **binomial coefficient** $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ counts the number of distinct ways to arrange k successes among n trials.

Here's why the formula has that shape: p^k is the probability of k successes in a specific order; $(1-p)^{n-k}$ is the probability of $n-k$ failures; $\binom{n}{k}$ counts how many orderings produce exactly k successes. By independence, each ordering has the same probability, so we multiply and sum.

Expected value:

$$\mathbb{E}[X] = np$$

Variance:

$$\text{Var}(X) = np(1-p)$$

Trust this result: Both formulas follow from viewing X as the sum of n independent Bernoulli(p) variables and applying the linearity of expectation and the variance-of-sum rule for independent variables.

Quantity	Value
Support	$0, 1, \ldots, n$
Parameters	$n \in \mathbb{N}$ (trials), $p \in [0, 1]$ (success prob.)
PMF	$\binom{n}{k} p^k (1-p)^{n-k}$
$\mathbb{E}[X]$	np
$\text{Var}(X)$	$np(1-p)$

Worked Example 8.2.1 — Quality Control

A factory produces widgets. Each widget has a 40% defect rate independently ($p = 0.4$). A batch of $n = 3$ widgets is inspected. Let X = number of defective widgets.

$X \sim \text{Binomial}(3,, 0.4)$

Step 1 — Full PMF:

$$P(X = 0) = \binom{3}{0}(0.4)^0(0.6)^3 = 1 \times 1 \times 0.216 = 0.216$$

$$P(X = 1) = \binom{3}{1}(0.4)^1(0.6)^2 = 3 \times 0.4 \times 0.36 = 0.432$$

$$P(X = 2) = \binom{3}{2}(0.4)^2(0.6)^1 = 3 \times 0.16 \times 0.6 = 0.288$$

$$P(X = 3) = \binom{3}{3}(0.4)^3(0.6)^0 = 1 \times 0.064 \times 1 = 0.064$$

Step 2 — Verify normalization:

$$0.216 + 0.432 + 0.288 + 0.064 = 1.000 \checkmark$$

Step 3 — Expected value and variance:

$$\mathbb{E}[X] = 3 \times 0.4 = 1.2$$

$$\text{Var}(X) = 3 \times 0.4 \times 0.6 = 0.72$$

Step 4 — Verify $\mathbb{E}[X]$ directly:

$$\mathbb{E}[X] = 0(0.216) + 1(0.432) + 2(0.288) + 3(0.064)$$

$$= 0 + 0.432 + 0.576 + 0.192 = 1.200 \checkmark$$

Worked Example 8.2.2 — Model Accuracy on a Batch

A classifier has true accuracy 70% ($p = 0.7$). On a batch of $n = 10$ samples, let X = number correct.

$$P(X = 7) = \binom{10}{7}(0.7)^7(0.3)^3$$

Step 1 — Binomial coefficient:

$$\binom{10}{7} = \frac{10!}{7!, 3!} = \frac{10 \times 9 \times 8}{3 \times 2 \times 1} = \frac{720}{6} = 120$$

Step 2 — Powers:

$$(0.7)^7 = 0.0823543, \qquad (0.3)^3 = 0.027$$

Step 3 — Multiply:

$$P(X = 7) = 120 \times 0.0823543 \times 0.027 = 120 \times 0.002224 \approx 0.2668$$

There is about a 26.7% chance the model gets exactly 7 out of 10 correct.

Expected correct: $\mathbb{E}[X] = 10 \times 0.7 = 7$

Engineer's Angle

Binomial distributions model **batch accuracy** and **A/B test counts**. When you split traffic 50/50 between model A and model B and count how many users in the B group convert, that count follows a Binomial distribution. Statistical significance tests for A/B results rely on the Binomial (or the Normal approximation to it via the Central Limit Theorem — see Section 8.5).

```
import math

def binom_coeff(n, k):
    return math.factorial(n) // (math.factorial(k) *
math.factorial(n - k))

def binom_pmf(n, k, p):
    return binom_coeff(n, k) * (p ** k) * ((1 - p) ** (n - k))

def binom_mean(n, p):
    return n * p

def binom_var(n, p):
    return n * p * (1 - p)

# Quality control example
n, p = 3, 0.4
pmf = [binom_pmf(n, k, p) for k in range(n + 1)]
print("Binomial(3, 0.4) PMF:")
for k, prob in enumerate(pmf):
    print(f"  P(X={k}) = {prob:.6f}")
print(f"  Sum = {sum(pmf):.10f}")       # 1.0000000000
print(f"  E[X] = {binom_mean(n, p)}")   # 1.2
print(f"  Var(X) = {binom_var(n, p)}")  # 0.72
```

8.3 Categorical and Multinomial Distributions

Plain English First

The Bernoulli distribution handles two outcomes. The **Categorical** distribution generalizes it to K outcomes — one draw that lands on exactly one of K categories. A single die roll is Categorical. A single image classifier prediction (cat, dog, or bird?) is Categorical.

The **Multinomial** distribution then generalizes Binomial: run n independent Categorical trials and count how many fall into each category. If you roll a die 60 times, the vector of counts (how many 1s, 2s, 3s, ...) follows a Multinomial distribution.

Formal Notation

Categorical distribution with K classes: parameters are a probability vector $\mathbf{p} = (p_1, p_2, \ldots, p_K)$ with $p_k \geq 0$ and $\sum_{k=1}^{K} p_k = 1$.

$$P(X = k) = p_k, \qquad k \in 1, 2, \ldots, K$$

The **one-hot representation** encodes outcome k as a vector $\mathbf{e}_k$ where the k-th entry is 1 and all others are 0.

Multinomial distribution with n trials: the count vector $(C_1, C_2, \ldots, C_K)$ where C_k = number of times category k appears, $\sum_k C_k = n$.

$$P(C_1 = c_1, \ldots, C_K = c_K) = \frac{n!}{c_1!, c_2!, \cdots, c_K!}, p_1^{c_1} p_2^{c_2} \cdots p_K^{c_K}$$

Expected value of each count:

$$\mathbb{E}[C_k] = n, p_k$$

Quantity	Categorical	Multinomial
Support	One category $k \in 1, \ldots, K$	Count vector $(c_1, \ldots, c_K)$, $\sum c_k = n$
Parameters	$\mathbf{p} = (p_1, \ldots, p_K)$	n, $\mathbf{p}$
$\mathbb{E}[\cdot]$	N/A (each p_k)	$\mathbb{E}[C_k] = np_k$

Worked Example 8.3.1 — Three-Class Image Classifier

A classifier assigns images to three classes with probabilities:

$$p_{\text{cat}} = 0.5, \quad p_{\text{dog}} = 0.3, \quad p_{\text{bird}} = 0.2$$

Verify normalization: $0.5 + 0.3 + 0.2 = 1.0$✓

For a batch of $n = 100$ images, the expected counts are:

$$\mathbb{E}[C_{\text{cat}}] = 100 \times 0.5 = 50$$

$$\mathbb{E}[C_{\text{dog}}] = 100 \times 0.3 = 30$$

$$\mathbb{E}[C_{\text{bird}}] = 100 \times 0.2 = 20$$

Multinomial probability of an exact count — say, 50 cats, 30 dogs, 20 birds from 100 draws:

$$P(C_{\text{cat}} = 50, C_{\text{dog}} = 30, C_{\text{bird}} = 20) = \frac{100!}{50!, 30!, 20!}(0.5)^{50}(0.3)^{30}(0.2)^{20}$$

This number is astronomically small (there are an enormous number of equally likely orderings). In practice we use the expected counts, not this exact probability.

Worked Example 8.3.2 — Rolling a Die Twice

A fair die has $K = 6$, each $p_k = \frac{1}{6}$. Roll $n = 2$ times. What is $P(C_1 = 1, C_2 = 1, C_3 = 0, \ldots, C_6 = 0)$? (Exactly one 1 and one 2.)

$$P = \frac{2!}{1!, 1!, 0! \cdots 0!}\left(\frac{1}{6}\right)^1\left(\frac{1}{6}\right)^1(1)^0 \cdots (1)^0 = 2 \times \frac{1}{36} = \frac{1}{18}$$

Verify: The probability of rolling a 1 then a 2 is $\frac{1}{6} \times \frac{1}{6} = \frac{1}{36}$, and the same for rolling a 2 then a 1. Total: $\frac{2}{36} = \frac{1}{18}$. ✓

Engineer's Angle

The Categorical distribution is the output model for **multi-class classification**. The softmax function (Section 8.7) converts raw model scores into a Categorical probability vector. The true label in multiclass classification is one-hot — a degenerate Categorical with all probability on one class.

```python
import math

def categorical_pmf(k, probs):
    """P(X = k) for Categorical(probs). k is 0-indexed."""
    return probs[k]

def multinomial_expected_counts(n, probs):
    """Expected count for each category in n trials."""
    return [n * p for p in probs]

# Three-class classifier
probs = [0.5, 0.3, 0.2]
classes = ['cat', 'dog', 'bird']

print("Categorical PMF:")
for i, cls in enumerate(classes):
    print(f"  P(X={cls}) = {categorical_pmf(i, probs)}")

print("\nExpected counts from 100 images:")
for cls, ec in zip(classes, multinomial_expected_counts(100,
probs)):
    print(f"  E[C_{cls}] = {ec}")
# Verify: sum of probs == 1
print(f"\nSum of probs: {sum(probs)}")   # 1.0
```

8.4 Uniform Distribution

Plain English First

The Uniform distribution is the "I have no information" distribution. Every outcome in a range is equally likely. There are two flavors: discrete (a finite set of equally likely outcomes, like a fair die) and continuous (any value in an interval is equally likely).

Formal Notation

Discrete Uniform on $a, a+1, \ldots, b$ with $n = b - a + 1$ values:

$$P(X = k) = \frac{1}{n}, \qquad k = a, a+1, \ldots, b$$

$$\mathbb{E}[X] = \frac{a+b}{2}, \qquad \text{Var}(X) = \frac{n^2 - 1}{12}$$

Continuous Uniform on interval $[a, b]$:

$$f(x) = \frac{1}{b-a}, \qquad a \le x \le b, \qquad f(x) = 0 \text{ otherwise}$$

$$\mathbb{E}[X] = \frac{a+b}{2}, \qquad \text{Var}(X) = \frac{(b-a)^2}{12}$$

Trust this result: The PDF value $\frac{1}{b-a}$ is chosen so that the total area under the curve equals 1: $\int_a^b \frac{1}{b-a}\,dx = \frac{b-a}{b-a} = 1$.

Quantity	Discrete $a, \ldots, b$	Continuous $[a, b]$
PMF / PDF	$1/n$	$1/(b-a)$
$\mathbb{E}[X]$	$(a+b)/2$	$(a+b)/2$
$\text{Var}(X)$	$(n^2-1)/12$	$(b-a)^2/12$

Worked Example 8.4.1 — Fair Six-Sided Die

$X \sim \text{DiscreteUniform}(1, 6)$, so $n = 6$.

$$\mathbb{E}[X] = \frac{1+6}{2} = \frac{7}{2} = 3.5$$

$$\text{Var}(X) = \frac{6^2-1}{12} = \frac{36-1}{12} = \frac{35}{12} \approx 2.917$$

Verify against Chapter 7 Example 7.8.1: We computed $\mathbb{E}[X] = 3.5$ and $\text{Var}(X) = \frac{35}{12}$ directly. ✓

Worked Example 8.4.2 — Continuous Uniform on $[-1, 1]$

$X \sim \text{Uniform}(-1, 1)$.

$$\mathbb{E}[X] = \frac{-1+1}{2} = 0$$

$$\text{Var}(X) = \frac{(1-(-1))^2}{12} = \frac{4}{12} = \frac{1}{3} \approx 0.333$$

Verify the PDF integrates to 1:

$$\int_{-1}^{1} \frac{1}{2}\,dx = \frac{1}{2} \times 2 = 1 ✓$$

Compute a probability: $P(-0.5 \le X \le 0.5)$:

$$P(-0.5 \le X \le 0.5) = \int_{-0.5}^{0.5} \frac{1}{2}\,dx = \frac{1}{2} \times (0.5 - (-0.5)) = \frac{1}{2} \times 1 = 0.5$$

Exactly half the interval has half the probability. ✓

Engineer's Angle

Uniform distributions appear in two key places:

1. **Weight initialization** (Xavier / Glorot initialization): initial weights are drawn from Uniform$(-c, c)$ where $c = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$. The Uniform assumption here is a maximum-entropy prior given a fixed range.
2. **Dropout masking**: each neuron is independently kept (1) or dropped (0) with equal probability — a discrete Uniform-like sampling per neuron. The sampling is Bernoulli, but the *decision* to use $p = 0.5$ often comes from a Uniform-random perspective.

```
import math

def discrete_uniform_mean(a, b):
    return (a + b) / 2

def discrete_uniform_var(a, b):
    n = b - a + 1
    return (n**2 - 1) / 12

def continuous_uniform_pdf(x, a, b):
    if a <= x <= b:
        return 1.0 / (b - a)
    return 0.0

def continuous_uniform_prob(x_lo, x_hi, a, b):
    """P(x_lo <= X <= x_hi) for X ~ Uniform(a, b)."""
    lo = max(x_lo, a)
    hi = min(x_hi, b)
    return max(0.0, (hi - lo) / (b - a))

# Fair die
print(f"DiscreteUniform(1,6): E={discrete_uniform_mean(1,6)}, Var=
{discrete_uniform_var(1,6):.6f}")
# DiscreteUniform(1,6): E=3.5, Var=2.916667

# Continuous uniform [-1, 1]
print(f"Uniform(-1,1): E={discrete_uniform_mean(-1,1)}")
print(f"Uniform(-1,1): Var={(2**2)/12:.6f}")
print(f"P(-0.5 <= X <= 0.5) = {continuous_uniform_prob(-0.5, 0.5,
-1, 1)}")
# P(-0.5 <= X <= 0.5) = 0.5

# Xavier init range for a layer: n_in=512, n_out=256
n_in, n_out = 512, 256
c = math.sqrt(6 / (n_in + n_out))
print(f"\nXavier init: Uniform(-{c:.4f}, {c:.4f})")
```

8.5 Gaussian (Normal) Distribution

Plain English First

The Gaussian (Normal) distribution is the most important distribution in all of statistics and machine learning. It describes the bell-shaped curve you've seen everywhere: test scores, measurement errors, heights, and — after appropriate scaling — the outputs of almost any averaging process. It is symmetric around its center, tails off rapidly in both directions, and is completely characterized by just two numbers: the mean (center) and variance (spread).

Why is it everywhere? Section 8.5.4 gives the answer (Central Limit Theorem). First, let's understand the shape.

Formal Notation

$X \sim \mathcal{N}(\mu, \sigma^2)$ (read: "X is Normal with mean μ and variance σ^2").

PDF:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \qquad x \in (-\infty, +\infty)$$

Expected value and variance:

$$\mathbb{E}[X] = \mu, \qquad \mathrm{Var}(X) = \sigma^2$$

Trust this result: $\int_{-\infty}^{+\infty} f(x), dx = 1$. This integral requires a clever polar-coordinate trick; the result is taken as given. What matters is understanding why the formula has this shape.

Here's why the bell curve looks the way it does: The term $(x-\mu)^2$ is zero at the center ($x = \mu$) and grows as you move away. Negating it and exponentiating gives a function that is 1 at the center and decays toward 0 symmetrically on both sides. The factor $\frac{1}{\sigma\sqrt{2\pi}}$ is a normalizing constant that scales the curve so the total area equals 1. The larger σ is, the wider the bell must be to maintain unit area, so the peak at $x = \mu$ is lower.

Key shape properties:

- The peak of the PDF is at $x = \mu$ with height $\frac{1}{\sigma\sqrt{2\pi}}$.
- The PDF has inflection points at $x = \mu \pm \sigma$ (where the curve transitions from concave down to concave up).
- The tails decay extremely fast because of the e^{-x^2} factor.

Quantity	Value
Support	$(-\infty, +\infty)$
Parameters	$\mu \in \mathbb{R}$ (mean), $\sigma^2 > 0$ (variance)
PDF	$\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$
$\mathbb{E}[X]$	μ
$\text{Var}(X)$	σ^2
Peak height	$\frac{1}{\sigma\sqrt{2\pi}}$

8.5.1 The Standard Normal and Z-Scores

The **standard normal** is the special case $\mu = 0$, $\sigma^2 = 1$, written $Z \sim \mathcal{N}(0, 1)$.

$$\phi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$$

The peak of the standard normal is at $z = 0$ with height $\frac{1}{\sqrt{2\pi}} \approx 0.3989$.

The **CDF** of the standard normal is:

$$\Phi(z) = P(Z \leq z) = \int_{-\infty}^{z} \phi(t), dt$$

This integral has no closed form in elementary functions, but it is related to the **error function** erf:

$$\Phi(z) = \frac{1}{2}\left[1 + \text{erf!}\left(\frac{z}{\sqrt{2}}\right)\right]$$

Converting any normal to standard: If $X \sim \mathcal{N}(\mu, \sigma^2)$, then:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1)$$

The value Z is called the **Z-score**: it measures how many standard deviations X is above or below the mean.

$$P(X \leq x) = P!\left(Z \leq \frac{x - \mu}{\sigma}\right) = \Phi!\left(\frac{x - \mu}{\sigma}\right)$$

8.5.2 The 68-95-99.7 Rule

For any Gaussian, almost all probability mass lies within three standard deviations of the mean:

$$P(\mu - \sigma \leq X \leq \mu + \sigma) = P(-1 \leq Z \leq 1) = \text{erf!}\left(\frac{1}{\sqrt{2}}\right) \approx 0.6827$$

$$P(\mu - 2\sigma \leq X \leq \mu + 2\sigma) = P(-2 \leq Z \leq 2) = \text{erf!}\left(\frac{2}{\sqrt{2}}\right) \approx 0.9545$$

$$P(\mu - 3\sigma \leq X \leq \mu + 3\sigma) = P(-3 \leq Z \leq 3) = \text{erf!}\left(\frac{3}{\sqrt{2}}\right) \approx 0.9973$$

Verification using erf (so this is not magic):

$$P(-1 \leq Z \leq 1) = \Phi(1) - \Phi(-1) = \frac{1}{2}!\left[1 + \text{erf!}\left(\frac{1}{\sqrt{2}}\right)\right] - \frac{1}{2}!\left[1 + \text{erf!}\left(\frac{-1}{\sqrt{2}}\right)\right.$$

Since $\text{erf}(-x) = -\text{erf}(x)$ (erf is an odd function):

$$= \frac{1}{2}!\left[\text{erf!}\left(\frac{1}{\sqrt{2}}\right) + \text{erf!}\left(\frac{1}{\sqrt{2}}\right)\right] = \text{erf!}\left(\frac{1}{\sqrt{2}}\right)$$

Numerically: $\text{erf}(1/\sqrt{2}) = \text{erf}(0.7071) \approx 0.6827$. ✓

8.5.3 Worked Examples

Worked Example 8.5.1 — Exam Scores

Exam scores follow $X \sim \mathcal{N}(\mu = 75, \sigma^2 = 64)$, so $\sigma = 8$.

(a) What fraction of students score below 83?

$$z = \frac{83 - 75}{8} = \frac{8}{8} = 1.0$$

$$P(X < 83) = \Phi(1.0) \approx 0.8413$$

About **84.1%** of students score below 83.

(b) What fraction score between 67 and 83?

$$z_{\text{lo}} = \frac{67 - 75}{8} = -1.0, \qquad z_{\text{hi}} = \frac{83 - 75}{8} = +1.0$$

$$P(67 < X < 83) = \Phi(1) - \Phi(-1) \approx 0.8413 - 0.1587 = 0.6827$$

About **68.3%** — exactly the 68-95-99.7 rule in action (one standard deviation band). ✓

(c) What fraction score above 91?

$$z = \frac{91 - 75}{8} = \frac{16}{8} = 2.0$$

$$P(X > 91) = 1 - \Phi(2.0) \approx 1 - 0.9772 = 0.0228$$

About **2.3%** — consistent with the 95.45% rule (leaving 4.55% outside two standard deviations, split equally on both tails gives about 2.28% on each side). ✓

Worked Example 8.5.2 — PDF Value at a Point

For $X \sim \mathcal{N}(170, 100)$ ($\mu = 170$, $\sigma = 10$), what is the PDF at $x = 185$?

$$z = \frac{185 - 170}{10} = 1.5$$

$$f(185) = \frac{1}{10\sqrt{2\pi}} e^{-1.5^2/2} = \frac{1}{10 \times 2.5066} e^{-1.125}$$

$$= \frac{1}{25.066} \times 0.3247 \approx \frac{0.3247}{25.066} \approx 0.01295$$

Remember: this is a *density*, not a probability. The probability of X falling in a tiny interval $[185, 185.01]$ would be approximately $0.01295 \times 0.01 = 0.0001295$.

8.5.4 Why Does the Gaussian Appear Everywhere? (Central Limit Theorem)

Here is one of the most important theorems in all of mathematics, stated intuitively:

> **Central Limit Theorem (informal):** The average of many independent, identically distributed random variables — regardless of the distribution of those individual variables — converges to a Gaussian as the number of variables grows.

In other words: add up 30 uniform random variables, or 30 exponential random variables, or 30 Bernoulli random variables, and the sum will look Gaussian. This is why:

- **Measurement noise** is Gaussian: physical sensors aggregate many tiny independent error sources.
- **Aggregated prediction errors** look Gaussian: a model's total error is the sum of many small independent mistakes.

- **Feature means in large datasets** are Gaussian: by the CLT, the sample mean of any feature has an approximately Gaussian distribution when the sample is large.
- **Neural network activations** before nonlinearities often look Gaussian for similar reasons.

No proof is required here. The takeaway for engineers: *when a quantity is the result of summing or averaging many independent contributions, expect it to look Gaussian.*

Engineer's Angle

The Gaussian is central to:

1. **Weight initialization**: Drawing weights from $\mathcal{N}(0, \sigma^2)$ (He init uses $\sigma^2 = 2/n_{\text{in}}$) keeps activation magnitudes stable through deep networks.
2. **VAE latent space**: The encoder outputs μ and σ^2, and the latent code $z \sim \mathcal{N}(\mu, \sigma^2)$ is sampled from this Gaussian — the KL divergence (Section 8.8) from this to the standard normal is the regularizer.
3. **Diffusion models**: The forward process corrupts data by adding Gaussian noise, step by step. The reverse (denoising) process learns to invert this.
4. **Gaussian noise layers**: Regularization by adding $\mathcal{N}(0, \sigma^2)$ noise to inputs.

```python
import math

def gaussian_pdf(x, mu, sigma):
    """PDF of N(mu, sigma^2) at x."""
    coeff = 1.0 / (sigma * math.sqrt(2 * math.pi))
    exponent = -0.5 * ((x - mu) / sigma) ** 2
    return coeff * math.exp(exponent)

def phi(z):
    """CDF of standard normal N(0,1) at z."""
    return 0.5 * (1.0 + math.erf(z / math.sqrt(2)))

def normal_cdf(x, mu, sigma):
    """CDF of N(mu, sigma^2) at x."""
    return phi((x - mu) / sigma)

def normal_prob_between(lo, hi, mu, sigma):
    """P(lo <= X <= hi) for X ~ N(mu, sigma^2)."""
    return normal_cdf(hi, mu, sigma) - normal_cdf(lo, mu, sigma)

# Standard normal peak
print(f"Standard normal peak: {1/math.sqrt(2*math.pi):.6f}")  #
0.398942

# Exam score example: N(75, 64)
mu, sigma = 75, 8
print(f"\nExam scores ~ N(75, 64):")
print(f"  P(X < 83) = {normal_cdf(83, mu, sigma):.6f}")     #
0.841345
print(f"  P(67<X<83) = {normal_prob_between(67, 83, mu,
sigma):.6f}")  # 0.682689
print(f"  P(X > 91) = {1 - normal_cdf(91, mu, sigma):.6f}") #
0.022750

# 68-95-99.7 rule
for k in [1, 2, 3]:
    prob = 2 * phi(k) - 1   # = phi(k) - phi(-k)
    print(f"  P(within {k} std devs) = {prob:.6f}
({prob*100:.2f}%)")
# 0.682689, 0.954500, 0.997300

# He init: weights ~ N(0, 2/n_in)
n_in = 512
sigma_he = math.sqrt(2 / n_in)
print(f"\nHe init for n_in={n_in}: N(0, {2/n_in:.6f}), sigma=
{sigma_he:.6f}")
```

8.6 Exponential Distribution

Plain English First

If you are waiting for something to happen — a server request, a hardware failure, a customer arriving — and the events occur at a constant average rate and independently of each other, then the time between events follows an Exponential distribution.

The defining characteristic of the Exponential distribution is the **memoryless property**: if you have been waiting for 5 minutes, the probability of waiting another 3 minutes is exactly the same as it would have been if you had just started waiting. The distribution "forgets" how long you have already waited.

Formal Notation

$X \sim \text{Exponential}(\lambda)$ where $\lambda > 0$ is the **rate parameter** (events per unit time).

PDF:

$$f(x) = \lambda e^{-\lambda x}, \qquad x \geq 0$$

CDF (probability that the wait is at most t):

$$F(t) = P(X \leq t) = 1 - e^{-\lambda t}$$

Expected value and variance:

$$\mathbb{E}[X] = \frac{1}{\lambda}, \qquad \text{Var}(X) = \frac{1}{\lambda^2}$$

Trust this result: The mean $1/\lambda$ makes intuitive sense: if events arrive at rate $\lambda = 2$ per minute, you expect to wait $1/2$ a minute on average.

Memoryless property: For $s, t > 0$:

$$P(X > s + t \mid X > s) = P(X > t)$$

Here's why the memoryless property holds:

$$P(X > s + t \mid X > s) = \frac{P(X > s + t)}{P(X > s)} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = \frac{e^{-\lambda s} e^{-\lambda t}}{e^{-\lambda s}} = e^{-\lambda t} = P(X$$

Quantity	Value
Support	$[0, +\infty)$
Parameter	$\lambda > 0$ (rate)
PDF	$\lambda e^{-\lambda x}$
CDF	$1 - e^{-\lambda x}$
$\mathbb{E}[X]$	$1/\lambda$
$\text{Var}(X)$	$1/\lambda^2$
Standard deviation	$1/\lambda$ (equals the mean!)

Worked Example 8.6.1 — Web Server Requests

Requests arrive at an average rate of $\lambda = 0.5$ requests per minute. The time between requests follows $X \sim \text{Exponential}(0.5)$.

Expected wait time:

$$\mathbb{E}[X] = \frac{1}{0.5} = 2 \text{ minutes}$$

Step 1 — Probability of waiting at most 3 minutes:

$$P(X \leq 3) = 1 - e^{-0.5 \times 3} = 1 - e^{-1.5} = 1 - 0.2231 = 0.7769$$

About **77.7%** of the time, the next request arrives within 3 minutes.

Step 2 — Probability of waiting more than 5 minutes:

$$P(X > 5) = e^{-0.5 \times 5} = e^{-2.5} \approx 0.0821$$

About **8.2%** of inter-arrival gaps exceed 5 minutes.

Worked Example 8.6.2 — Memoryless Property in Action

Using the same server ($\lambda = 0.5$). You have already waited 2 minutes without a request. What is the probability you wait at least 1 more minute?

Using the memoryless property directly:

$$P(X > 3 \mid X > 2) = P(X > 1) = e^{-0.5 \times 1} = e^{-0.5} \approx 0.6065$$

Verify by direct computation:

$$P(X > 3 \mid X > 2) = \frac{P(X > 3)}{P(X > 2)} = \frac{e^{-1.5}}{e^{-1.0}} = \frac{0.2231}{0.3679} \approx 0.6065$$

Both methods give the same answer. ✓ The 2 minutes already elapsed are completely irrelevant — the exponential distribution has no memory.

Engineer's Angle

The Exponential distribution appears in:

1. **Queuing models for inference latency**: If requests arrive at constant average rate and service times are Exponential, the system follows a Poisson-process / M/M/1 queue. Understanding tail latency (the 99th percentile wait) requires knowing the CDF of the Exponential.
2. **Training time budgets**: Time to see the first gradient improvement, time between loss spikes — these can often be modeled as Exponential for simple rough calculations.
3. **Survival analysis**: In time-to-event models (how long before a user churns?), the Exponential is the simplest baseline model. More sophisticated models use Weibull distributions, but Exponential is the starting point.

```python
import math

def exp_pdf(x, lam):
    """PDF of Exponential(lam) at x."""
    if x < 0:
        return 0.0
    return lam * math.exp(-lam * x)

def exp_cdf(x, lam):
    """CDF of Exponential(lam): P(X <= x)."""
    if x < 0:
        return 0.0
    return 1.0 - math.exp(-lam * x)

def exp_survival(x, lam):
    """P(X > x) for Exponential(lam)."""
    if x < 0:
        return 1.0
    return math.exp(-lam * x)

lam = 0.5
print(f"Exponential(lambda=0.5):")
print(f"  E[X] = {1/lam} minutes")
print(f"  P(X <= 3) = {exp_cdf(3, lam):.6f}")    # 0.776870
print(f"  P(X > 5) = {exp_survival(5, lam):.6f}") # 0.082085

# Memoryless property verification
s, t = 2, 1
cond = exp_survival(s + t, lam) / exp_survival(s, lam)
direct = exp_survival(t, lam)
print(f"\nMemoryless: P(X>3|X>2) = {cond:.6f}")
print(f"            P(X>1)     = {direct:.6f}")
print(f"            Equal: {abs(cond - direct) < 1e-10}")  # True
```

8.7 Softmax as a Distribution

Plain English First

A neural network's final classification layer produces a vector of raw scores — one per class — called **logits**. Logits can be any real numbers: negative, zero, positive, large, small. We need to turn them into probabilities that sum to 1. The **softmax function** does exactly this, and the result is a valid Categorical distribution.

The name "softmax" comes from the fact that it is a smooth, differentiable approximation of the "hardmax" (argmax) function. The class with the largest

logit gets the most probability, but not all of it.

Formal Notation

Given a vector of logits $\mathbf{z} = (z_1, z_2, \ldots, z_K)$, the softmax produces a probability vector $\boldsymbol{\pi}$ where:

$$\pi_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k = 1, 2, \ldots, K$$

Proof that $\sum_k \pi_k = 1$:

$$\sum_{k=1}^{K} \pi_k = \sum_{k=1}^{K} \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} = \frac{\sum_{k=1}^{K} e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} = 1 \quad \square$$

Proof that softmax reduces to sigmoid for $K = 2$:

With $K = 2$, logits (z_1, z_2):

$$\pi_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{z_2 - z_1}} = \sigma(z_1 - z_2)$$

where $\sigma(s) = \frac{1}{1+e^{-s}}$ is the sigmoid function. Binary logistic regression is the two-class special case of softmax classification. $\square$

Temperature scaling: A parameter $T > 0$ can be introduced:

$$\pi_k = \frac{e^{z_k / T}}{\sum_j e^{z_j / T}}$$

- $T \to 0$: distribution collapses to a one-hot on the argmax (hardmax).
- $T \to \infty$: distribution approaches Uniform (maximum uncertainty).

Worked Example 8.7.1 — Three-Class Logits

Logits from a classifier: $\mathbf{z} = (2.0, 1.0, 0.5)$.

Step 1 — Compute exponentials:

$$e^{2.0} = 7.3891, \quad e^{1.0} = 2.7183, \quad e^{0.5} = 1.6487$$

Step 2 — Sum:

$$S = 7.3891 + 2.7183 + 1.6487 = 11.7561$$

Step 3 — Divide:

$$\pi_1 = \frac{7.3891}{11.7561} \approx 0.6285, \quad \pi_2 = \frac{2.7183}{11.7561} \approx 0.2312, \quad \pi_3 = \frac{1.6487}{11.7561} \approx 0.1402$$

Step 4 — Verify:

$$0.6285 + 0.2312 + 0.1402 = 0.9999 \approx 1.0000 \checkmark$$

The largest logit (2.0) gets the majority of the probability mass, but the smaller logits still get non-zero probability.

Worked Example 8.7.2 — Sigmoid as Two-Class Softmax

Logits $\mathbf{z} = (1.5, 0.0)$ (two classes).

Softmax:

$$\pi_1 = \frac{e^{1.5}}{e^{1.5} + e^{0.0}} = \frac{4.4817}{4.4817 + 1.0000} = \frac{4.4817}{5.4817} \approx 0.8176$$

Sigmoid:

$$\sigma(1.5 - 0.0) = \sigma(1.5) = \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.2231} = \frac{1}{1.2231} \approx 0.8176$$

Same result. ✓ Binary classification with logistic regression is exactly two-class softmax with logits $(s, 0)$.

Engineer's Angle

In PyTorch/TensorFlow, `nn.CrossEntropyLoss` (or `tf.keras.losses.CategoricalCrossentropy`) combines softmax and cross-entropy in a single, numerically stable operation. The trick is the **log-sum-exp** computation:

$$\log \pi_k = z_k - \log!\left(\sum_j e^{z_j}\right)$$

Subtracting the maximum logit before exponentiating prevents overflow — a "numeric stabilization" trick.

```
import math

def softmax(z):
    """Numerically stable softmax."""
    # Subtract max for numerical stability
    z_max = max(z)
    exps = [math.exp(zi - z_max) for zi in z]
    total = sum(exps)
    return [e / total for e in exps]

def sigmoid(s):
    return 1.0 / (1.0 + math.exp(-s))

def log_softmax(z):
    """log(softmax(z)) — numerically stable."""
    z_max = max(z)
    log_sum = z_max + math.log(sum(math.exp(zi - z_max) for zi in
z))
    return [zi - log_sum for zi in z]

# Three-class example
z = [2.0, 1.0, 0.5]
probs = softmax(z)
print(f"softmax({z}) = {[round(p, 6) for p in probs]}")
# [0.628532, 0.231224, 0.140244]
print(f"Sum = {sum(probs):.10f}")   # 1.0000000000

# Temperature scaling
def softmax_temp(z, T):
    return softmax([zi / T for zi in z])

print(f"\nTemperature T=0.1 (sharp): {[round(p,4) for p in
softmax_temp(z, 0.1)]}")
print(f"Temperature T=1.0 (normal): {[round(p,4) for p in
softmax_temp(z, 1.0)]}")
print(f"Temperature T=10  (flat):  {[round(p,4) for p in
softmax_temp(z, 10)]}")

# Binary: softmax vs sigmoid
z2 = [1.5, 0.0]
sm2 = softmax(z2)
sig = sigmoid(z2[0] - z2[1])
print(f"\nTwo-class softmax[0] = {sm2[0]:.6f}")
print(f"sigmoid(1.5 - 0.0)   = {sig:.6f}")
print(f"Equal: {abs(sm2[0] - sig) < 1e-10}")   # True
```

8.8 KL Divergence — Measuring Distance Between Distributions

Plain English First

Suppose you have two probability distributions: P (the "truth") and Q (your model's approximation). How different are they? Intuitively, they are "close" if they assign similar probabilities to similar events. KL divergence gives a precise numerical measure of how much information you lose by using Q to represent P.

There are two important warnings about KL divergence:

1. It is **not a distance** in the mathematical sense, because $D_{KL}(P|Q) \neq D_{KL}(Q|P)$ in general. Direction matters.
2. It is always **non-negative**: $D_{KL}(P|Q) \geq 0$, with equality if and only if $P = Q$ everywhere.

Formal Notation

Discrete case (P and Q are distributions over the same finite set):

$$D_{KL}(P|Q) = \sum_{k} P(k) \log \frac{P(k)}{Q(k)}$$

Continuous case:

$$D_{KL}(P|Q) = \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)}, dx$$

Convention: We use natural logarithm (base e) throughout, giving units of **nats**. (Using log base 2 gives bits; the choice is cosmetic.)

Convention: If $P(k) = 0$, the k-th term contributes 0 (by convention, $0 \log 0 = 0$). If $P(k) > 0$ but $Q(k) = 0$, the divergence is $+\infty$ — you have assigned zero probability to something that can actually happen.

Trust this result: $D_{KL}(P|Q) \geq 0$, with equality iff $P = Q$ everywhere. This follows from Jensen's inequality applied to the convex function $-\log$.

Worked Example 8.8.1 — KL Divergence and Its Asymmetry

Let $P = (0.9, 0.1)$ and $Q = (0.5, 0.5)$ over two outcomes.

Compute $D_{KL}(P|Q)$: (how much Q approximates P)

$$D_{KL}(P|Q) = P(1)\log\frac{P(1)}{Q(1)} + P(2)\log\frac{P(2)}{Q(2)}$$

$$= 0.9 \times \log\frac{0.9}{0.5} + 0.1 \times \log\frac{0.1}{0.5}$$

$$= 0.9 \times \log(1.8) + 0.1 \times \log(0.2)$$

Using $\log(1.8) = 0.5878$ and $\log(0.2) = -1.6094$:

$$= 0.9 \times 0.5878 + 0.1 \times (-1.6094) = 0.5290 - 0.1609 = 0.3681 \text{ nats}$$

Compute $D_{KL}(Q|P)$: (the reverse direction)

$$D_{KL}(Q|P) = 0.5 \times \log\frac{0.5}{0.9} + 0.5 \times \log\frac{0.5}{0.1}$$

$$= 0.5 \times \log(0.5556) + 0.5 \times \log(5.0)$$

Using $\log(0.5556) = -0.5878$ and $\log(5.0) = 1.6094$:

$$= 0.5 \times (-0.5878) + 0.5 \times 1.6094 = -0.2939 + 0.8047 = 0.5108 \text{ nats}$$

Conclusion: $D_{KL}(P|Q) = 0.3681 \neq 0.5108 = D_{KL}(Q|P)$. KL divergence is asymmetric. ✓

The two directions penalize different mistakes:

- $D_{KL}(P|Q)$: penalizes assigning low probability under Q to events that are common under P (it "forces" Q to cover the support of P).
- $D_{KL}(Q|P)$: penalizes Q for spreading probability where P is concentrated (it "forces" Q to be cautious).

Worked Example 8.8.2 — Zero KL When $P = Q$

For $P = Q = (0.6, 0.4)$:

$$D_{KL}(P|Q) = 0.6 \times \log\frac{0.6}{0.6} + 0.4 \times \log\frac{0.4}{0.4} = 0.6 \times \log(1) + 0.4 \times \log(1) = 0$$

Engineer's Angle

KL divergence is the workhorse of probabilistic ML:

1. **VAE regularizer**: The encoder outputs a Gaussian $q(z|x) = \mathcal{N}(\mu_\phi, \sigma^2_\phi)$. The KL term $D_{KL}(q(z|x)|p(z))$ where $p(z) = \mathcal{N}(0, 1)$ penalizes the

latent distribution for deviating from the prior. For Gaussians this has a closed form: $\frac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$.

2. **Knowledge distillation**: Training a small student model to match the softmax outputs of a large teacher model uses $D_{KL}(\text{teacher}|\text{student})$.

3. **RL policy optimization**: Many RL algorithms (PPO, TRPO) constrain the KL divergence between the old and new policy to prevent destructive policy updates.

```
import math

def kl_divergence(P, Q):
    """
    KL(P || Q) for discrete distributions.
    P and Q are lists of probabilities over the same support.
    Uses natural log (nats).
    """
    total = 0.0
    for p, q in zip(P, Q):
        if p == 0:
            continue        # 0 * log(0/q) = 0 by convention
        if q == 0:
            return float('inf')  # cannot assign 0 where P > 0
        total += p * math.log(p / q)
    return total

P = [0.9, 0.1]
Q = [0.5, 0.5]

kl_pq = kl_divergence(P, Q)
kl_qp = kl_divergence(Q, P)

print(f"KL(P || Q) = {kl_pq:.6f} nats")   # 0.368064
print(f"KL(Q || P) = {kl_qp:.6f} nats")   # 0.510826
print(f"Asymmetric: {abs(kl_pq - kl_qp) > 0.01}")  # True

# Verify zero when P == Q
P_eq = [0.6, 0.4]
Q_eq = [0.6, 0.4]
print(f"\nKL(P || P) = {kl_divergence(P_eq, Q_eq):.10f}")  #
0.0000000000

# KL from standard Gaussian approximation (VAE term, scalar case)
def kl_gaussian_standard(mu, sigma_sq):
    """KL(N(mu, sigma^2) || N(0,1)) — closed form."""
    return 0.5 * (mu**2 + sigma_sq - math.log(sigma_sq) - 1)

print(f"\nVAE KL: N(0.5, 1.0) vs N(0,1) =
{kl_gaussian_standard(0.5, 1.0):.6f}")
print(f"VAE KL: N(0.0, 1.0) vs N(0,1) = {kl_gaussian_standard(0.0,
1.0):.6f}")  # 0
```

8.9 Entropy

Plain English First

Before we can fully explain cross-entropy, we need one more concept: **entropy**. Entropy measures the average uncertainty (or information content) of a distribution. A distribution that is always the same (a coin that always lands heads) has zero entropy — no uncertainty. A fair coin has maximum entropy — you never know what comes next.

Formal Notation

For a discrete distribution P over K outcomes:

$$H(P) = -\sum_{k=1}^{K} P(k) \log P(k)$$

Using natural log, the unit is **nats**. Using log base 2, the unit is **bits**.

Key properties:

- $H(P) \geq 0$ always.
- $H(P) = 0$ iff P is concentrated on a single outcome (certain outcome).
- $H(P)$ is maximized when P is Uniform.

Worked Example 8.9.1

For $P = (0.9, 0.1)$:

$$H(P) = -(0.9 \log 0.9 + 0.1 \log 0.1)$$

$$= -(0.9 \times (-0.1054) + 0.1 \times (-2.3026))$$

$$= -((-0.0948) + (-0.2303))$$

$$= -(-0.3251) = 0.3251 \text{ nats}$$

For $P = (0.5, 0.5)$ (maximum entropy for two outcomes):

$$H(P) = -(0.5 \log 0.5 + 0.5 \log 0.5) = -(2 \times 0.5 \times (-0.6931)) = 0.6931 \text{ nats}$$

The uniform distribution has higher entropy. ✓

8.10 Cross-Entropy and Its Connection to KL Divergence

Plain English First

Cross-entropy answers the question: "What is the average number of nats needed to encode samples from P, if we design our encoding scheme assuming Q is the true distribution?" If $Q = P$, the answer is just the entropy $H(P)$. If $Q \neq P$, we waste some bits — that waste is exactly the KL divergence.

In machine learning, P is the true label distribution (often a one-hot vector) and Q is the model's predicted distribution (the softmax output). Cross-entropy is the loss function that training minimizes.

Formal Notation

Cross-entropy between P (true) and Q (model):

$$H(P, Q) = -\sum_{k} P(k) \log Q(k)$$

Key identity: Cross-entropy decomposes into entropy plus KL divergence:

$$H(P, Q) = H(P) + D_{KL}(P|Q)$$

Here's why:

$$H(P, Q) = -\sum_{k} P(k) \log Q(k)$$

Add and subtract $\sum_k P(k) \log P(k)$:

$$= -\sum_{k} P(k) \log P(k) + \sum_{k} P(k) \log P(k) - \sum_{k} P(k) \log Q(k)$$

$$= H(P) + \sum_{k} P(k) \log \frac{P(k)}{Q(k)}$$

$$= H(P) + D_{KL}(P|Q) \quad \square$$

Consequence for training: Since $H(P)$ is fixed (the true label distribution does not depend on the model), minimizing cross-entropy over model parameters θ is equivalent to minimizing the KL divergence $D_{KL}(P|Q_\theta)$:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} D_{KL}(P|Q_\theta)$$

This is why **cross-entropy loss and KL-based likelihood maximization are the same thing**.

Worked Example 8.10.1 — Multiclass Cross-Entropy Loss

A three-class classifier receives an image. The true label is "cat" (class 0), so the true distribution is the one-hot vector $P = (1, 0, 0)$.

The model predicts $Q = (0.7, 0.2, 0.1)$.

Cross-entropy loss:

$$H(P, Q) = -(1 \times \log 0.7 + 0 \times \log 0.2 + 0 \times \log 0.1)$$

$$= -\log 0.7 = -(-0.3567) = 0.3567 \text{ nats}$$

Here's why the zero terms vanish: $P(k) = 0$ for all classes except the true class, so those terms contribute zero to the sum. The cross-entropy loss for one-hot P reduces to:

$$H(P, Q) = -\log Q(\text{true class})$$

This is the **negative log-likelihood of the true class**. Training pushes $Q(\text{true class})$ toward 1, which pushes the loss toward 0.

Verify the decomposition:

$$H(P) = -(1 \log 1 + 0 + 0) = 0 \quad \text{(one-hot has zero entropy)}$$

$$D_{KL}(P|Q) = 1 \times \log \frac{1}{0.7} = \log(1/0.7) = -\log(0.7) = 0.3567$$

$$H(P) + D_{KL}(P|Q) = 0 + 0.3567 = 0.3567 = H(P, Q) \checkmark$$

When the true distribution is one-hot, cross-entropy equals KL divergence (since $H(P) = 0$).

Worked Example 8.10.2 — Binary Cross-Entropy

For binary classification ($K = 2$), the true label $y \in 0, 1$ gives a one-hot P. The model predicts $\hat{y} \in (0, 1)$, so $Q = (\hat{y}, 1 - \hat{y})$.

The cross-entropy reduces to the familiar formula:

$$H(P, Q) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

Case 1: $y = 1$, $\hat{y} = 0.8$ (correct and confident):

$$\mathcal{L} = -\log(0.8) = 0.2231 \text{ nats}$$

Case 2: $y = 1$, $\hat{y} = 0.3$ (wrong and confident):

$$\mathcal{L} = -\log(0.3) = 1.2040 \text{ nats}$$

The loss is much larger when the model is confidently wrong. ✓ This is the loss from Section 8.1 — we now know exactly where it comes from: it is the cross-entropy between the one-hot true distribution and the Bernoulli model output.

Engineer's Angle

```
import math

def entropy(P):
    """Shannon entropy H(P) in nats."""
    return -sum(p * math.log(p) for p in P if p > 0)

def cross_entropy(P, Q):
    """Cross-entropy H(P, Q) in nats. P is true, Q is model."""
    return -sum(p * math.log(q) for p, q in zip(P, Q) if p > 0)

def kl_divergence(P, Q):
    return sum(p * math.log(p / q) for p, q in zip(P, Q) if p > 0)

def binary_cross_entropy(y_true, y_pred):
    """Binary cross-entropy for one sample."""
    return -(y_true * math.log(y_pred) + (1 - y_true) * math.log(1
- y_pred))

# ── One-hot example
─────────────────────────────────────────────────────
P_true = [1.0, 0.0, 0.0]   # true class is 0
Q_pred = [0.7, 0.2, 0.1]   # model output

H_P  = entropy(P_true)               # 0.0 (one-hot has zero
entropy)
KL   = kl_divergence(P_true, Q_pred)  # 0.356675
H_PQ = cross_entropy(P_true, Q_pred)  # 0.356675

print(f"H(P)        = {H_P:.6f}")
print(f"KL(P||Q)    = {KL:.6f}")
print(f"H(P,Q)      = {H_PQ:.6f}")
print(f"H(P)+KL = H(P,Q): {abs(H_P + KL - H_PQ) < 1e-10}")  # True

# ── Binary cross-entropy
──────────────────────────────────────────────
print(f"\nBCE(y=1, yhat=0.8) = {binary_cross_entropy(1, 0.8):.6f}")
# 0.223144
print(f"BCE(y=1, yhat=0.3) = {binary_cross_entropy(1, 0.3):.6f}")
# 1.203973

# ── Training goal: minimizing CE minimizes KL
──────────────────────────
# As model improves (Q gets closer to P), CE decreases
Q_better  = [0.9, 0.07, 0.03]
Q_perfect = [1.0 - 1e-9, 5e-10, 5e-10]   # near-perfect (avoid
log(0))
print(f"\nCE with Q=[0.7,0.2,0.1]: {cross_entropy(P_true,
Q_pred):.6f}")
print(f"CE with Q=[0.9,0.07,0.03]: {cross_entropy(P_true,
Q_better):.6f}")
print(f"CE with Q≈[1,0,0]:         {cross_entropy(P_true,
```

```
Q_perfect):.6f}")
# Loss approaches 0 as model approaches certainty on the correct
class
```

8.11 Summary Table

Distribution	Type	Parameters	$\mathbb{E}[X]$	$\text{Var}(X)$	ML Connection
$\text{Bernoulli}(p)$	Discrete	$p \in [0,1]$	p	$p(1-p)$	Binary classifier output, logistic regression
$\text{Binomial}(n,p)$	Discrete	$n \in \mathbb{N},, p \in [0,1]$	np	$np(1-p)$	Batch accuracy, A/B test counts
$\text{Categorical}(\mathbf{p})$	Discrete	$\mathbf{p}$: probability vector	N/A	N/A	Multi-class label, softmax output
$\text{Multinomial}(n,\mathbf{p})$	Discrete	$n \in \mathbb{N},, \mathbf{p}$: prob. vector	np_k per class	$np_k(1-p_k)$	Class count distribution
$\text{DiscreteUniform}(a,b)$	Discrete	$a \le b$ integers	$(a+b)/2$	$(n^2-1)/12$	Fair sampling, baseline model
$\text{Uniform}(a,b)$	Continuous	$a < b$ reals	$(a+b)/2$	$(b-a)^2/12$	Xavier init, dropout sampling
$\mathcal{N}(\mu,\sigma^2)$	Continuous	$\mu \in \mathbb{R},, \sigma^2 > 0$	μ	σ^2	Weight init, VAE latent, diffusion noise
$\text{Exponential}(\lambda)$	Continuous	$\lambda > 0$	$1/\lambda$	$1/\lambda^2$	Inter-arrival times, latency modeling

Softmax output	Continuous	Logits $\mathbf{z} \in \mathbb{R}^K$	—	—	Final layer of classifier
KL Divergence	Measure	Two distributions P, Q	—	—	VAE regularizer, distillation
Cross-Entropy	Loss	True P, predicted Q	—	—	Classification loss function

---

8.12 Exercises

Exercise 8.1 [Easy] — Bernoulli and Binomial

A model predicts "positive" with probability $p = 0.6$ on any given input, independently.

(a) Write the PMF of a single prediction $X \sim \text{Bernoulli}(0.6)$.

(b) In a batch of $n = 5$ inputs, let Y = number of positive predictions. Compute $P(Y = 3)$ from first principles (show the binomial coefficient computation).

(c) Compute $\mathbb{E}[Y]$ and $\text{Var}(Y)$.

(d) Compute $P(Y \geq 4) = P(Y = 4) + P(Y = 5)$.

Solution:

(a) $P(X = 1) = 0.6$, $P(X = 0) = 0.4$. Compact form: $P(X = k) = (0.6)^k (0.4)^{1-k}$ for $k \in 0, 1$.

(b) $Y \sim \text{Binomial}(5,, 0.6)$.

$$\binom{5}{3} = \frac{5!}{3!, 2!} = \frac{5 \times 4}{2 \times 1} = 10$$

$$P(Y = 3) = \binom{5}{3} (0.6)^3 (0.4)^2 = 10 \times 0.216 \times 0.16 = 10 \times 0.03456 = 0.3456$$

(c)

$$\mathbb{E}[Y] = 5 \times 0.6 = 3.0$$

$$\text{Var}(Y) = 5 \times 0.6 \times 0.4 = 1.2$$

(d)

$$P(Y=4) = \binom{5}{4}(0.6)^4(0.4)^1 = 5 \times 0.1296 \times 0.4 = 5 \times 0.05184 = 0.2592$$

$$P(Y=5) = \binom{5}{5}(0.6)^5(0.4)^0 = 1 \times 0.07776 \times 1 = 0.07776$$

$$P(Y \geq 4) = 0.2592 + 0.07776 = 0.33696$$

Verify: Sum all probabilities.

$$P(Y=0) = (0.4)^5 = 0.01024$$

$$P(Y=1) = 5(0.6)(0.4)^4 = 5 \times 0.6 \times 0.0256 = 0.07680$$

$$P(Y=2) = 10(0.6)^2(0.4)^3 = 10 \times 0.36 \times 0.064 = 0.23040$$

$$P(Y=3) = 0.34560$$

$$P(Y=4) = 0.25920$$

$$P(Y=5) = 0.07776$$

$$\text{Total} = 0.01024 + 0.07680 + 0.23040 + 0.34560 + 0.25920 + 0.07776 = 1.00$$

Exercise 8.2 [Easy] — Gaussian Z-Scores

The latency of a web service (in milliseconds) is modeled as $X \sim \mathcal{N}(\mu = 200, \sigma^2 = 900)$, so $\sigma = 30$.

(a) Compute the Z-score for $x = 260$ ms and $x = 170$ ms.

(b) Use the 68-95-99.7 rule to answer: what fraction of requests have latency between 140 ms and 260 ms?

(c) A response takes longer than 290 ms. Approximately what fraction of requests are this slow? (Use the 68-95-99.7 rule.)

Solution:

(a)

$$z_{260} = \frac{260 - 200}{30} = \frac{60}{30} = 2.0$$

$$z_{170} = \frac{170 - 200}{30} = \frac{-30}{30} = -1.0$$

(b) We need $P(140 \leq X \leq 260)$.

$$z_{\text{lo}} = \frac{140 - 200}{30} = -2.0, \qquad z_{\text{hi}} = \frac{260 - 200}{30} = +2.0$$

This is the $\pm 2\sigma$ band: $P(-2 \le Z \le 2) \approx 0.9545$. About **95.5%** of requests fall in this range.

(c) $z = \frac{290-200}{30} = 3.0$. The 99.7% rule says $P(-3 \le Z \le 3) \approx 0.9973$, leaving $1 - 0.9973 = 0.0027$ outside. By symmetry, the upper tail $P(Z > 3) \approx 0.0027/2 \approx 0.00135$.

About **0.135%** of requests exceed 290 ms. Only about 1 in 740 requests.

Exercise 8.3 [Medium] — Exponential Memoryless Property

A GPU job scheduler receives jobs at a Poisson rate with average inter-arrival time of 4 minutes, so inter-arrival times follow $X \sim \text{Exponential}(\lambda = 0.25)$.

(a) Verify that $\mathbb{E}[X] = 4$ minutes.

(b) Compute $P(X \le 5)$ — the probability the next job arrives within 5 minutes. Show full arithmetic.

(c) You have already been waiting 3 minutes with no job. Using the memoryless property, what is the probability you wait at least 2 more minutes?

(d) Verify part (c) by computing the conditional probability directly.

Solution:

(a) $\mathbb{E}[X] = \frac{1}{\lambda} = \frac{1}{0.25} = 4$ minutes. ✓

(b)

$$P(X \le 5) = 1 - e^{-0.25 \times 5} = 1 - e^{-1.25} = 1 - 0.2865 = 0.7135$$

(Check: $e^{-1.25} = e^{-1} \times e^{-0.25} \approx 0.3679 \times 0.7788 = 0.2865$.)

About **71.4%** of the time the next job arrives within 5 minutes.

(c) By the memoryless property:

$$P(X > 3 + 2 \mid X > 3) = P(X > 2) = e^{-0.25 \times 2} = e^{-0.5} \approx 0.6065$$

The 3 minutes already elapsed are irrelevant. There is about a **60.7%** chance of waiting at least 2 more minutes.

(d) Direct computation:

$$P(X > 5 \mid X > 3) = \frac{P(X > 5)}{P(X > 3)} = \frac{e^{-0.25\times5}}{e^{-0.25\times3}} = \frac{e^{-1.25}}{e^{-0.75}} = e^{-1.25+0.75} = e^{-0.5} \approx 0$$

Both approaches give the same answer. ✓

Exercise 8.4 [Medium] — Softmax and Temperature

A three-class model produces logits $\mathbf{z} = (3.0,, 1.0,, 0.0)$.

(a) Compute the softmax probabilities. Show all arithmetic.

(b) A two-class version has logits $(3.0, 1.0)$. Verify that $\text{softmax}(\mathbf{z})_1 = \sigma(z_1 - z_2)$.

(c) What happens to the distribution at temperature $T = 2.0$? Compute the new softmax.

(d) As $T \to \infty$, what does the distribution approach?

Solution:

(a)

$$e^{3.0} = 20.0855, \quad e^{1.0} = 2.7183, \quad e^{0.0} = 1.0000$$

$$S = 20.0855 + 2.7183 + 1.0000 = 23.8038$$

$$\pi_1 = \frac{20.0855}{23.8038} \approx 0.8437, \quad \pi_2 = \frac{2.7183}{23.8038} \approx 0.1142, \quad \pi_3 = \frac{1.0000}{23.8038} \approx 0.0420$$

Verify: $0.8437 + 0.1142 + 0.0420 = 0.9999 \approx 1.0000$ ✓

(b) Two-class logits $(3.0, 1.0)$:

$$\text{softmax}_1 = \frac{e^{3.0}}{e^{3.0} + e^{1.0}} = \frac{20.0855}{20.0855 + 2.7183} = \frac{20.0855}{22.8038} \approx 0.8808$$

$$\sigma(3.0 - 1.0) = \sigma(2.0) = \frac{1}{1 + e^{-2.0}} = \frac{1}{1 + 0.1353} = \frac{1}{1.1353} \approx 0.8808 ✓$$

(c) At $T = 2.0$, divide logits by T: scaled logits $= (1.5,, 0.5,, 0.0)$.

$$e^{1.5} = 4.4817, \quad e^{0.5} = 1.6487, \quad e^{0.0} = 1.0000$$

$$S = 4.4817 + 1.6487 + 1.0000 = 7.1304$$

$$\pi_1 = \frac{4.4817}{7.1304} \approx 0.6286, \quad \pi_2 = \frac{1.6487}{7.1304} \approx 0.2312, \quad \pi_3 = \frac{1.0000}{7.1304} \approx 0.1402$$

The distribution is flatter (less confident) at $T = 2$ compared to $T = 1$.

(d) As $T \to \infty$, all logits divided by T approach 0. Then all exponentials approach $e^0 = 1$, and the distribution approaches $\pi_k = \frac{1}{K} = \frac{1}{3}$ for all k — a uniform distribution. At very high temperature, the model is maximally uncertain.

Exercise 8.5 [Hard] — KL Divergence and Cross-Entropy in Classifier Training

A binary classifier with sigmoid output is trained on a dataset with balanced classes. The true distribution over labels is $P = (0.5, 0.5)$ (50% positive, 50% negative). The model currently predicts $Q_1 = (0.8, 0.2)$ (overconfident on positive class).

(a) Compute $H(P)$, $H(P, Q_1)$, and $D_{KL}(P|Q_1)$. Verify the identity $H(P, Q_1) = H(P) + D_{KL}(P|Q_1)$.

(b) After training improves the model to $Q_2 = (0.55, 0.45)$, compute $H(P, Q_2)$ and $D_{KL}(P|Q_2)$. Which direction did both quantities move?

(c) If the model reaches $Q_3 = P = (0.5, 0.5)$, what is the minimum achievable cross-entropy loss? What does this correspond to?

(d) A different model predicts $Q_4 = (0.5, 0.5)$ from the start. Compute $H(P, Q_4)$ and explain why this is the theoretical minimum for this problem.

Solution:

(a)

Entropy of P:

$$H(P) = -(0.5 \log 0.5 + 0.5 \log 0.5) = -(2 \times 0.5 \times (-0.6931)) = 0.6931 \text{ nats}$$

Cross-entropy $H(P, Q_1)$ with $Q_1 = (0.8, 0.2)$:

$$H(P, Q_1) = -(0.5 \log 0.8 + 0.5 \log 0.2)$$

$$= -(0.5 \times (-0.2231) + 0.5 \times (-1.6094))$$

$$= -((-0.1116) + (-0.8047))$$

$$= 0.9163 \text{ nats}$$

KL divergence:

$$D_{KL}(P|Q_1) = 0.5 \log \frac{0.5}{0.8} + 0.5 \log \frac{0.5}{0.2}$$

$$= 0.5 \times \log(0.625) + 0.5 \times \log(2.5)$$

$$= 0.5 \times (-0.4700) + 0.5 \times 0.9163$$

$$= -0.2350 + 0.4581 = 0.2231 \text{ nats}$$

Verify identity:

$$H(P) + D_{KL}(P|Q_1) = 0.6931 + 0.2231 = 0.9162 \approx 0.9163 \checkmark$$

(Small rounding difference in the last digit from truncated intermediate values.)

(b) **Cross-entropy with $Q_2 = (0.55, 0.45)$:**

$$H(P, Q_2) = -(0.5 \log 0.55 + 0.5 \log 0.45)$$

$$= -(0.5 \times (-0.5978) + 0.5 \times (-0.7985))$$

$$= -((-0.2989) + (-0.3993))$$

$$= 0.6982 \text{ nats}$$

KL divergence:

$$D_{KL}(P|Q_2) = H(P, Q_2) - H(P) = 0.6982 - 0.6931 = 0.0050 \text{ nats}$$

Both $H(P, Q_2) = 0.6982 < H(P, Q_1) = 0.9163$ and $D_{KL}(P|Q_2) = 0.0050 < D_{KL}(P|Q_1) = 0.2231$. The model improved: both cross-entropy and KL divergence decreased.

(c) If $Q_3 = P = (0.5, 0.5)$:

$$H(P, Q_3) = H(P) + D_{KL}(P|Q_3) = H(P) + 0 = H(P) = 0.6931 \text{ nats}$$

The minimum achievable cross-entropy is the entropy of the true label distribution. This corresponds to a perfectly calibrated model: it cannot be improved further without changing the true distribution (i.e., getting better data).

(d) $H(P, Q_4) = H(P, (0.5, 0.5)) = H(P) = 0.6931$ nats, same as (c). This is the theoretical minimum because the model perfectly matches the true distribution $P = (0.5, 0.5)$.

In practice, the true label distribution P is unknown. We approximate $H(P)$ using the empirical cross-entropy on the training set. The irreducible lower

bound $H(P)$ represents the inherent uncertainty in the labeling problem — no model, no matter how large, can do better.

Next: Chapter 9 — Information Theory: Entropy, Mutual Information, and their role in feature selection and generative models.

Chapter 9: Statistics for ML

> *"A model that trains well is promising. A model you can trust is useful."*

Chapters 7 and 8 gave you the language of probability: how to describe uncertainty, how distributions work, and what cross-entropy and KL divergence mean. This chapter uses that language to answer a harder question: **when can you trust a result?**

Every time you train a model you are making implicit statistical choices. Choosing cross-entropy loss instead of MSE is a statistical argument. Choosing L2 regularization over L1 is a statistical argument. Reporting "95% accuracy" without a confidence interval is a statistical mistake. This chapter makes those arguments explicit and gives you the tools to evaluate model results with rigor.

We connect everything to what came before:

- **Chapters 2–4** (linear algebra) underpin the Normal Equation and ridge regression.
- **Chapters 5–6** (calculus) underpin gradient-based training and the bias-variance tradeoff.
- **Chapters 7–8** (probability) underpin MLE, regularization as a prior, and hypothesis testing.

9.1 Point Estimation — What Does It Mean to Fit a Model?

Plain English First

When you train a model, you are computing a **point estimate**: a single number (or vector of numbers) that summarizes what the data is telling you. For example, after seeing data $x_1, x_2, \ldots, x_n$, you might produce:

- A mean $\hat{\mu}$ estimating the center of the data.
- A probability $\hat{p}$ estimating the fraction of positive examples.
- A weight vector $\hat{\mathbf{w}}$ estimating the parameters of a linear model.

The hat notation $\hat{\theta}$ universally means "estimate of θ computed from data." It is distinct from the true but unknown value θ^*.

There are infinitely many ways to produce an estimate. You could guess at random. You could always output zero. You could average the data. What makes one estimator better than another? We need a principled criterion. The most important one in machine learning is **Maximum Likelihood Estimation**.

Formal Notation

An **estimator** is a function $\hat{\theta}(\mathbf{x})$ that maps a dataset $\mathbf{x} = (x_1, \ldots, x_n)$ to a parameter value. It is itself a random variable — different random datasets produce different estimates.

Two properties we want an estimator to have:

Unbiasedness: $\mathbb{E}[\hat{\theta}] = \theta^*$. On average, the estimator is correct.

Consistency: As $n \to \infty$, $\hat{\theta} \to \theta^*$ in probability. More data means a better estimate.

Trust this result: The sample mean $\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$ is both unbiased and consistent for the population mean. These are provable from Chapter 7's linearity of expectation and the Law of Large Numbers.

The engineer's perspective: Point estimation is what happens every time you call `.fit()`. The optimizer finds the parameter vector $\hat{\theta}$ that minimizes the training loss. But which loss? That choice is not arbitrary — it is determined by a statistical argument called Maximum Likelihood.

9.2 Maximum Likelihood Estimation (MLE)

Plain English First

Maximum Likelihood Estimation answers a fundamental question: given that I observed this data, which parameter value made the data most probable?

Imagine you flip a coin five times and see heads, heads, tails, heads, tails. The coin has some unknown bias p (probability of heads). If $p = 0.5$, this sequence has some probability. If $p = 0.6$, it has a slightly different probability. If $p = 0.9$, it has a much lower probability (three tails would be very surprising). MLE asks: what value of p maximizes the probability of the data you actually observed?

That value — the one that makes the data "most likely" — is the MLE estimate $\hat{p}$.

This is not just a heuristic. It is the mathematical justification for why neural networks minimize cross-entropy loss, why linear regression minimizes squared error, and why these are the right choices under specific probabilistic assumptions.

Formal Notation

Let $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ be n independent, identically distributed (i.i.d.) observations from a distribution with parameter θ.

Likelihood function: The probability of observing the data as a function of θ:

$$\mathcal{L}(\theta) = p(\mathbf{x} \mid \theta) = \prod_{i=1}^{n} p(x_i \mid \theta)$$

The product appears because the observations are independent (as in Chapter 7 — joint probability of independent events is the product of individual probabilities).

MLE estimate:

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \mathcal{L}(\theta)$$

9.2.1 The Log-Likelihood Trick: Products Become Sums

The likelihood is a product of n terms, each in $[0, 1]$. For large n, this product is astronomically small and numerically unstable. The solution: maximize the **log-likelihood** instead.

$$\ell(\theta) = \log \mathcal{L}(\theta) = \log \prod_{i=1}^{n} p(x_i \mid \theta) = \sum_{i=1}^{n} \log p(x_i \mid \theta)$$

Here's why this is valid: logarithm is a strictly increasing function, so $\arg\max \mathcal{L}(\theta) = \arg\max \log \mathcal{L}(\theta)$. The maximizer does not change.

Here's why this matters: the product of 10,000 probabilities underflows to zero in floating point. The sum of 10,000 log-probabilities stays numerically well-behaved. This is why training loops accumulate log-probabilities (or cross-entropy losses), not raw probabilities.

The connection to Chapter 8: For a one-hot true distribution P and a model distribution Q, the cross-entropy is:

$$H(P, Q) = -\sum_{k} P(k) \log Q(k) = -\log Q(\text{true class})$$

This is exactly the negative log-likelihood of the true class under the model's distribution. Training by minimizing cross-entropy is equivalent to maximizing the log-likelihood. This is not a coincidence — it is the mathematical foundation of why cross-entropy loss is correct.

9.2.2 Deriving MLE for the Bernoulli Parameter

Setup: We have n independent coin flips, each following Bernoulli(p). Observed outcomes $x_i \in 0, 1$ with $k = \sum_{i=1}^{n} x_i$ heads total. Find $\hat{p}_{\text{MLE}}$.

Step 1 — Write the likelihood:

Using the Bernoulli PMF $P(X = x_i) = p^{x_i}(1-p)^{1-x_i}$:

$$\mathcal{L}(p) = \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i} = p^{k}(1-p)^{n-k}$$

Step 2 — Take the log-likelihood:

$$\ell(p) = k \log p + (n-k) \log(1-p)$$

Step 3 — Differentiate and set to zero (Chapter 5 — setting derivative to zero finds the maximum):

$$\frac{d\ell}{dp} = \frac{k}{p} - \frac{n-k}{1-p} = 0$$

Step 4 — Solve:

$$\frac{k}{p} = \frac{n-k}{1-p}$$

$$k(1-p) = (n-k)p$$

$$k - kp = np - kp$$

$$k = np$$

$$\hat{p}_{\text{MLE}} = \frac{k}{n}$$

Interpretation: The MLE for the Bernoulli parameter is simply the fraction of observed successes. This is the intuitive answer — and now we have derived it from first principles.

Step 5 — Verify it is a maximum, not a minimum: Check the second derivative.

$$\frac{d^2\ell}{dp^2} = -\frac{k}{p^2} - \frac{n-k}{(1-p)^2}$$

For $k > 0$ and $k < n$, both terms are negative, so $\frac{d^2\ell}{dp^2} < 0$ at the critical point — confirming a maximum. ✓

Worked Example 9.2.1 — MLE for Bernoulli

Dataset: $1, 0, 1, 1, 0$ — five coin flips.

$n = 5$, $k = 3$ (three heads).

$$\hat{p}_{\text{MLE}} = \frac{3}{5} = 0.6$$

Verify by comparing log-likelihoods at nearby values:

$$\ell(0.6) = 3\log(0.6) + 2\log(0.4) = 3(-0.5108) + 2(-0.9163) = -1.5325 - 1.8$$

$$\ell(0.5) = 3\log(0.5) + 2\log(0.5) = 5 \times (-0.6931) = -3.4657$$

$$\ell(0.7) = 3\log(0.7) + 2\log(0.3) = 3(-0.3567) + 2(-1.2040) = -1.0700 - 2.4$$

Indeed $\ell(0.6) = -3.3651 > \ell(0.5) = -3.4657$ and $\ell(0.6) > \ell(0.7) = -3.4779$. The log-likelihood is highest at $\hat{p} = 0.6$. ✓

9.2.3 Deriving MLE for the Gaussian Mean

Setup: We have n i.i.d. observations from $\mathcal{N}(\mu, \sigma^2)$ where σ^2 is known. Find $\hat{\mu}_{\text{MLE}}$.

Step 1 — Write the log-likelihood:

Using the Gaussian PDF from Chapter 8:

$$\ell(\mu) = \sum_{i=1}^{n} \log \frac{1}{\sigma\sqrt{2\pi}} \exp!\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

$$= \sum_{i=1}^{n} \left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

$$= -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Step 2 — Maximize over μ: The first term is a constant in μ. Maximizing $\ell(\mu)$ is equivalent to minimizing $\sum_{i=1}^{n}(x_i - \mu)^2$ — the **sum of squared errors**.

Here's why this is important: MLE under a Gaussian noise assumption produces MSE (mean squared error) as the training loss. When you minimize squared error in linear regression, you are implicitly assuming Gaussian noise. The probabilistic model is baked into the loss function.

Step 3 — Differentiate and set to zero:

$$\frac{d}{d\mu}\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2\right] = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu) = 0$$

$$\sum_{i=1}^{n} x_i - n\mu = 0$$

$$\hat{\mu}_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n} x_i = \bar{x}$$

Interpretation: The MLE for the Gaussian mean is the sample mean. Again, the intuitive answer is the right one — and the derivation shows it follows directly from the Gaussian assumption plus the log-likelihood trick.

A note on MLE for variance: The MLE for σ^2 turns out to be $\frac{1}{n}\sum(x_i - \bar{x})^2$, which is the **biased** estimator. The unbiased estimator divides by $n - 1$ (Bessel's correction): $s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$. This is why statistics libraries report $n - 1$ in the denominator — they use the unbiased version. MLE is optimal in many ways, but "unbiased" is not always one of them.

Worked Example 9.2.2 — MLE for Gaussian Mean

Dataset: $2, 4, 6$ (three observations, known $\sigma^2 = 4$, so $\sigma = 2$).

$$\hat{\mu}_{\text{MLE}} = \frac{2+4+6}{3} = \frac{12}{3} = 4$$

Verify the derivative is zero at $\mu = 4$:

$$\sum_{i=1}^{3}(x_i - 4) = (2-4) + (4-4) + (6-4) = -2 + 0 + 2 = 0 \checkmark$$

Compare log-likelihoods:

$$\ell(4) = -\frac{3}{2}\log(8\pi) - \frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{8} = -\frac{3}{2}\log(8\pi) - \frac{4+0+}{8}$$

$$\ell(5) = -\frac{3}{2}\log(8\pi) - \frac{(2-5)^2 + (4-5)^2 + (6-5)^2}{8} = -\frac{3}{2}\log(8\pi) - \frac{9+1+}{8}$$

Since $-1 > -1.375$, we confirm $\ell(4) > \ell(5)$: the MLE at $\mu = 4$ has a higher log-likelihood than the competitor at $\mu = 5$. ✓

Summary: MLE Loss Functions

Noise model (on y)	Resulting MLE loss
$y \sim \mathcal{N}(f(x), \sigma^2)$	Mean Squared Error (MSE)
$y \sim \text{Bernoulli}(f(x))$	Binary cross-entropy
$y \sim \text{Categorical}(\text{softmax}(f(x)))$	Categorical cross-entropy

These are not arbitrary choices. Each loss function is the negative log-likelihood of the corresponding distributional assumption. Choosing a loss function is choosing a noise model.

Engineer's Angle

```
import math

# MLE for Bernoulli
def bernoulli_log_likelihood(data, p):
    """Log-likelihood of data under Bernoulli(p)."""
    k = sum(data)
    n = len(data)
    return k * math.log(p) + (n - k) * math.log(1 - p)

def bernoulli_mle(data):
    """MLE estimate for Bernoulli parameter p."""
    return sum(data) / len(data)

data = [1, 0, 1, 1, 0]
p_hat = bernoulli_mle(data)
print(f"Bernoulli MLE: p_hat = {p_hat}")          # 0.6

# Verify this is the maximum by scanning nearby values
for p in [0.4, 0.5, 0.6, 0.7, 0.8]:
    ll = bernoulli_log_likelihood(data, p)
    marker = " <-- MLE" if p == p_hat else ""
    print(f"  log-lik(p={p}) = {ll:.6f}{marker}")

# MLE for Gaussian mean (known variance)
def gaussian_log_likelihood(data, mu, sigma_sq):
    """Log-likelihood of data under N(mu, sigma_sq)."""
    n = len(data)
    sigma = math.sqrt(sigma_sq)
    const = -n / 2 * math.log(2 * math.pi * sigma_sq)
    sq_term = -sum((x - mu) ** 2 for x in data) / (2 * sigma_sq)
    return const + sq_term

def gaussian_mle_mean(data):
    """MLE estimate for Gaussian mean (sample mean)."""
    return sum(data) / len(data)

def gaussian_mle_var_biased(data, mu):
    """Biased MLE estimate for Gaussian variance (divides by n)."""
    n = len(data)
    return sum((x - mu) ** 2 for x in data) / n

def gaussian_var_unbiased(data, mu):
    """Unbiased Bessel-corrected variance estimate (divides by n-
1)."""
    n = len(data)
    return sum((x - mu) ** 2 for x in data) / (n - 1)

data_g = [2.0, 4.0, 6.0]
mu_hat = gaussian_mle_mean(data_g)
var_biased = gaussian_mle_var_biased(data_g, mu_hat)
var_unbiased = gaussian_var_unbiased(data_g, mu_hat)
print(f"\nGaussian MLE: mu_hat = {mu_hat}")
# 4.0
print(f"Biased variance (MLE, n in denominator) =
```

```
{var_biased:.4f}")   # 2.6667
print(f"Unbiased variance (Bessel, n-1 in denominator) =
{var_unbiased:.4f}")  # 4.0

# Verify: likelihood is maximized at mu_hat
sigma_sq = 4.0  # known variance
for mu in [3.0, 4.0, 5.0]:
    ll = gaussian_log_likelihood(data_g, mu, sigma_sq)
    print(f"  log-lik(mu={mu}) = {ll:.6f}" + (" <-- MLE" if mu ==
mu_hat else ""))
# mu=4.0 has highest log-likelihood
```

9.3 The Normal Equation — Closed-Form Linear Regression

Plain English First

In Chapter 6, we minimized the squared-error loss by gradient descent: update parameters repeatedly, one small step at a time. But for linear regression specifically, there is a better option: **solve exactly in one shot**.

Calculus tells us that at a minimum, all partial derivatives equal zero. For linear regression, that "all derivatives equal zero" condition is a system of linear equations — and we know how to solve those exactly using the matrix tools from Chapter 3. The result is the **Normal Equation**: a single matrix formula that gives the optimal weights directly.

Formal Notation

Given a dataset with n examples and d features, define:

- Design matrix $X \in \mathbb{R}^{n \times d}$: row i is the feature vector for example i.
- Target vector $\mathbf{y} \in \mathbb{R}^n$: the true outputs.
- Weight vector $\mathbf{w} \in \mathbb{R}^d$: parameters to find.

The **MSE loss** is:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}|X\mathbf{w} - \mathbf{y}|_2^2 = \frac{1}{n}(X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y})$$

Here's why the Normal Equation is $\mathbf{w} = (X^TX)^{-1}X^T\mathbf{y}$:

Step 1 — Expand the loss (dropping the $\frac{1}{n}$ constant which does not affect the minimizer):

$$\mathcal{L}(\mathbf{w}) = (X\mathbf{w} - \mathbf{y})^T(X\mathbf{w} - \mathbf{y})$$

$$= \mathbf{w}^T X^T X\mathbf{w} - \mathbf{w}^T X^T \mathbf{y} - \mathbf{y}^T X\mathbf{w} + \mathbf{y}^T\mathbf{y}$$

Since $\mathbf{w}^T X^T \mathbf{y}$ is a scalar, it equals its own transpose: $\mathbf{y}^T X\mathbf{w}$. So:

$$\mathcal{L}(\mathbf{w}) = \mathbf{w}^T X^T X\mathbf{w} - 2\mathbf{y}^T X\mathbf{w} + \mathbf{y}^T\mathbf{y}$$

Step 2 — Differentiate with respect to w (Chapter 6 — gradient of a quadratic form):

$$\nabla_{\mathbf{w}}\mathcal{L} = 2X^T X\mathbf{w} - 2X^T\mathbf{y}$$

Trust this result: The gradient of $\mathbf{w}^T A\mathbf{w}$ is $2A\mathbf{w}$ when A is symmetric (which $X^T X$ is), and the gradient of $\mathbf{b}^T\mathbf{w}$ is $\mathbf{b}$. Both follow from the matrix calculus rules in Chapter 6.

Step 3 — Set gradient to zero:

$$2X^T X\mathbf{w} - 2X^T\mathbf{y} = \mathbf{0}$$

$$X^T X\mathbf{w} = X^T\mathbf{y}$$

These are the **Normal Equations** — a system of d linear equations in d unknowns.

Step 4 — Solve (if $X^T X$ is invertible):

$$\boxed{\mathbf{w} = (X^T X)^{-1} X^T\mathbf{y}}$$

This is the Normal Equation.

Worked Example 9.3.1 — Fitting a Line

Fit $\hat{y} = w_1 x + w_0$ to three data points: $(1, 2)$, $(2, 3)$, $(3, 5)$.

Step 1 — Build the design matrix. Include a bias column of ones:

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$$

Step 2 — Compute $X^T X$:

$$X^T X = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1^2 + 2^2 + 3^2 & 1 + 2 + 3 \\ 1 + 2 + 3 & 1 + 1 + 1 \end{bmatrix} = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$$

Step 3 — Compute $X^T\mathbf{y}$:

$$X^T\mathbf{y} = \begin{bmatrix}1 & 2 & 3\\ 1 & 1 & 1\end{bmatrix}\begin{bmatrix}2\\3\\5\end{bmatrix} = \begin{bmatrix}1(2)+2(3)+3(5)\\ 1(2)+1(3)+1(5)\end{bmatrix}$$

Step 4 — Invert X^TX. For a 2×2 matrix $\begin{bmatrix}a & b\\ c & d\end{bmatrix}$, the inverse is $\frac{1}{ad-bc}\begin{bmatrix}d & -b\\ -c & a\end{bmatrix}$:

$$\det(X^TX) = 14\times 3 - 6\times 6 = 42-36 = 6$$

$$(X^TX)^{-1} = \frac{1}{6}\begin{bmatrix}3 & -6\\ -6 & 14\end{bmatrix} = \begin{bmatrix}1/2 & -1\\ -1 & 7/3\end{bmatrix}$$

Step 5 — Multiply to get $\mathbf{w}$:

$$\mathbf{w} = (X^TX)^{-1}X^T\mathbf{y} = \begin{bmatrix}1/2 & -1\\ -1 & 7/3\end{bmatrix}\begin{bmatrix}23\\10\end{bmatrix}$$

$$w_1 = \frac{1}{2}(23) + (-1)(10) = 11.5 - 10 = 1.5$$

$$w_0 = (-1)(23) + \frac{7}{3}(10) = -23 + \frac{70}{3} = \frac{-69+70}{3} = \frac{1}{3} \approx 0.333$$

Fitted model: $\hat{y} = 1.5x + \frac{1}{3}$

Step 6 — Verify predictions and residuals:

x	y	$\hat{y} = 1.5x + 1/3$	residual
1	2	$1.5 + 0.333 = 1.833$	$+0.167$
2	3	$3.0 + 0.333 = 3.333$	-0.333
3	5	$4.5 + 0.333 = 4.833$	$+0.167$

Verify the gradient is zero at these weights:

The gradient condition is $X^TX\mathbf{w} = X^T\mathbf{y}$:

$$\begin{bmatrix}14 & 6\\ 6 & 3\end{bmatrix}\begin{bmatrix}1.5\\ 1/3\end{bmatrix} = \begin{bmatrix}14(1.5)+6(1/3)\\ 6(1.5)+3(1/3)\end{bmatrix} = \begin{bmatrix}21+2\\ 9+1\end{bmatrix}$$

9.3.1 Normal Equation vs. Gradient Descent

	Normal Equation	Gradient Descent
Cost	$O(nd^2 + d^3)$ (matrix multiply + inversion)	$O(ndk)$ (k iterations)
When to prefer	Small d (up to ~10,000 features)	Large d, sparse data, online learning
Exact?	Yes — single-step exact solution	No — converges asymptotically
Hyperparameters	None	Learning rate, batch size, iterations
Singular X^TX?	Fails (use pseudoinverse instead)	Still works — converges to a solution
Streaming data?	No — needs full dataset	Yes — stochastic/mini-batch variants

The d^3 bottleneck: The matrix inversion step costs $O(d^3)$ operations. For $d = 10{,}000$ features, that is 10^{12} operations — intractable. For $d = 100$ features, it is 10^6 — fast. Gradient descent scales much better with d, which is why deep learning uses gradient descent despite having exact alternatives.

Trust this result: When X^TX is singular (which happens when features are linearly dependent — collinear), $(X^TX)^{-1}$ does not exist. The fix is to use the Moore-Penrose pseudoinverse X^+ or to add regularization — which leads directly to Section 9.5.

Engineer's Angle

```
import math

# ── Matrix utilities (stdlib only) ──────────────────────────────

def mat_transpose(A):
    """Transpose a matrix represented as list of row lists."""
    rows, cols = len(A), len(A[0])
    return [[A[r][c] for r in range(rows)] for c in range(cols)]

def mat_mul(A, B):
    """Matrix multiply A @ B."""
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])
    assert cols_A == rows_B, "Incompatible shapes"
    C = [[0.0] * cols_B for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]
    return C

def mat_vec_mul(A, v):
    """Matrix-vector product A @ v."""
    return [sum(A[i][j] * v[j] for j in range(len(v))) for i in
range(len(A))]

def inv2x2(A):
    """Invert a 2x2 matrix. Raises if singular."""
    a, b = A[0][0], A[0][1]
    c, d = A[1][0], A[1][1]
    det = a * d - b * c
    if abs(det) < 1e-12:
        raise ValueError("Matrix is singular")
    return [[d / det, -b / det], [-c / det, a / det]]

# ── Normal Equation ──────────────────────────────────────────────

def normal_equation_2d(data):
    """
    Fit y = w1*x + w0 using the Normal Equation.
    data: list of (x, y) pairs.
    Returns (w1, w0).
    """
    # Build design matrix X (x in col 0, bias=1 in col 1)
    X = [[xi, 1.0] for xi, yi in data]
    y = [yi for xi, yi in data]

    Xt    = mat_transpose(X)
    XtX   = mat_mul(Xt, X)
    Xty   = mat_vec_mul(Xt, y)
    XtX_inv = inv2x2(XtX)
    w     = mat_vec_mul(XtX_inv, Xty)
```

```
    return w[0], w[1]  # w1 (slope), w0 (intercept)

# ── Example ──────────────────────────────────────────────────────────
─

data = [(1, 2), (2, 3), (3, 5)]
w1, w0 = normal_equation_2d(data)
print(f"w1 (slope)     = {w1:.6f}")    # 1.500000
print(f"w0 (intercept) = {w0:.6f}")    # 0.333333

print("\nPredictions vs. actuals:")
for x, y in data:
    y_hat = w1 * x + w0
    residual = y - y_hat
    print(f"  x={x}, y={y}, y_hat={y_hat:.4f}, residual=
{residual:.4f}")

# Compute MSE
mse = sum((y - (w1 * x + w0)) ** 2 for x, y in data) / len(data)
print(f"\nMSE = {mse:.6f}")
```

9.4 Bias-Variance Tradeoff

Plain English First

Every prediction error comes from three sources. The **dartboard analogy** makes them viscerally clear.

Imagine you throw many darts at a target. Your "model" is your throwing arm for that session. The true bullseye is the target you're trying to hit.

- **High bias, low variance:** All your darts cluster tightly — but they're all far from the bullseye. You're consistently wrong. A model that always predicts the training average has this problem: it ignores structure in the data (underfitting).
- **Low bias, high variance:** Your darts scatter widely around the bullseye — sometimes close, sometimes far. Average position is right, but individual throws are unreliable. A model that memorizes training examples has this problem: it fits training noise and fails on new data (overfitting).
- **Low bias, low variance:** Darts cluster tightly around the bullseye. This is the goal. Achieve it through the right model complexity and

sufficient data.

The third source of error — **noise** — is the irreducible scatter in the data itself. No matter how good your model, if the data is inherently noisy, you cannot eliminate this error.

Formal Notation

Consider a fixed test point x_0 with true output $y_0 = f(x_0) + \epsilon$, where f is the true (unknown) function and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is irreducible noise.

Your model $\hat{f}$ is trained on a random training set, so $\hat{f}(x_0)$ is a random variable. Let $\bar{f} = \mathbb{E}[\hat{f}(x_0)]$ be the expected prediction over all possible training sets.

Define:

$$\text{Bias}(\hat{f}) = \bar{f} - f(x_0) = \mathbb{E}[\hat{f}(x_0)] - f(x_0)$$

$$\text{Variance}(\hat{f}) = \mathbb{E}!\left[(\hat{f}(x_0) - \bar{f})^2\right]$$

$$\text{MSE} = \mathbb{E}!\left[(\hat{f}(x_0) - y_0)^2\right]$$

Theorem:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \sigma^2$$

Here's why the decomposition holds:

$$\text{MSE} = \mathbb{E}!\left[(\hat{f}(x_0) - y_0)^2\right] = \mathbb{E}!\left[(\hat{f}(x_0) - f(x_0) - \epsilon)^2\right]$$

Add and subtract $\bar{f}$ inside:

$$= \mathbb{E}!\left[(\hat{f}(x_0) - \bar{f} + \bar{f} - f(x_0) - \epsilon)^2\right]$$

Let $U = \hat{f}(x_0) - \bar{f}$ (zero-mean random variable: $\mathbb{E}[U] = 0$), $B = \bar{f} - f(x_0)$ (the bias, a constant), and keep ϵ (also zero-mean, independent of training data). Then:

$$\text{MSE} = \mathbb{E}[(U + B - \epsilon)^2]$$

$$= \mathbb{E}[U^2] + B^2 + \mathbb{E}[\epsilon^2] + 2B,\mathbb{E}[U] - 2B,\mathbb{E}[\epsilon] - 2,\mathbb{E}[U\epsilon]$$

The cross-terms vanish: $\mathbb{E}[U] = 0$, $\mathbb{E}[\epsilon] = 0$, and U and ϵ are independent so $\mathbb{E}[U\epsilon] = 0$. Therefore:

$$\boxed{\text{MSE} = \underbrace{\mathbb{E}[U^2]}{\text{Variance}} + \underbrace{B^2}{\text{Bias}^2} + \underbrace{\mathbb{E}[\epsilon^2]}_{\sigma^2} = \text{Variance} + \text{Bias}^2 + \sigma^2} \quad \square$$

Worked Example 9.4.1 — Two Models at a Test Point

True function: $f(x) = 2x + 1$. Test point $x_0 = 3$, so $f(3) = 7$. Noise $\epsilon \sim \mathcal{N}(0, 0.25)$ so $\sigma^2 = 0.25$.

Model A — High-bias constant predictor: Always predicts $\hat{f}_A(x) = 3$ (the training mean, ignoring x). This is deterministic regardless of the training set.

$$\text{Bias}_A = \mathbb{E}[\hat{f}_A(3)] - f(3) = 3 - 7 = -4$$

$$\text{Bias}_A^2 = 16$$

$$\text{Variance}_A = \mathbb{E}[(\hat{f}_A(3) - 3)^2] = 0 \quad \text{(constant predictor has zero variance)}$$

$$\text{MSE}_A = 16 + 0 + 0.25 = 16.25$$

Model B — Low-bias flexible predictor: Interpolates training data, producing predictions $\hat{f}_B(3) \in 6, 8$ with equal probability across training sets (noisy, but centered on the truth).

$$\mathbb{E}[\hat{f}_B(3)] = \frac{6 + 8}{2} = 7$$

$$\text{Bias}_B = 7 - 7 = 0$$

$$\text{Bias}_B^2 = 0$$

$$\text{Variance}_B = \frac{(6 - 7)^2 + (8 - 7)^2}{2} = \frac{1 + 1}{2} = 1$$

$$\text{MSE}_B = 0 + 1 + 0.25 = 1.25$$

Conclusion: Model B has MSE of 1.25 vs Model A's 16.25. Model A's large bias overwhelms its advantage in variance. We prefer Model B.

	Bias^2	Variance	Noise	MSE
Model A (constant predictor)	16	0	0.25	**16.25**
Model B (flexible predictor)	0	1	0.25	**1.25**

9.4.1 The Tradeoff

Bias and variance are in tension:

- **Increasing model complexity** (more parameters, deeper network, higher polynomial degree) → lower bias, higher variance.
- **Decreasing model complexity** → higher bias, lower variance.
- **Adding more training data** → lower variance without changing the inherent bias.

The bias-variance tradeoff explains why:

- A degree-1 polynomial underfit (high bias) to data with quadratic structure.
- A degree-30 polynomial overfit (high variance) the same data.
- The optimal degree lies between, and can be found by cross-validation (Section 9.7).

Engineer's Angle

```
import math
import statistics

# Simulate bias-variance tradeoff for a constant vs mean predictor
# True f(x) = 2x + 1, test point x0 = 3, noise sigma^2 = 0.25

def true_f(x):
    return 2 * x + 1

x0 = 3
f_true = true_f(x0)   # 7
sigma_sq = 0.25

# ── Model A: always predicts 3 (high bias, zero variance) ──────────────
bias_A    = 3 - f_true     # -4
bias_sq_A = bias_A ** 2    # 16
var_A     = 0.0
mse_A     = bias_sq_A + var_A + sigma_sq
print(f"Model A (constant=3):")
print(f"  Bias^2={bias_sq_A}, Variance={var_A}, Noise={sigma_sq},
MSE={mse_A}")

# ── Model B: noisy but centered (low bias, some variance) ──────────────
predictions_B = [6.0, 8.0]
mean_B    = statistics.mean(predictions_B)   # 7.0
bias_B    = mean_B - f_true                  # 0.0
bias_sq_B = bias_B ** 2                      # 0.0
var_B     = statistics.mean([(p - mean_B)**2 for p in
predictions_B])  # 1.0
mse_B     = bias_sq_B + var_B + sigma_sq
print(f"\nModel B (flexible, E[f_hat]=7):")
print(f"  Bias^2={bias_sq_B}, Variance={var_B}, Noise={sigma_sq},
MSE={mse_B}")

# ── Verify decomposition numerically ──────────────────────────────────
# Simulate noise realizations
import random
random.seed(42)
n_sim = 100_000

def simulate_mse(predictions, sigma):
    """Estimate MSE by simulation."""
    errors = []
    for _ in range(n_sim):
        pred = random.choice(predictions)
        noise = random.gauss(0, sigma)
        y = f_true + noise
        errors.append((pred - y) ** 2)
    return statistics.mean(errors)

sigma = math.sqrt(sigma_sq)
```

```
sim_mse_A = simulate_mse([3.0], sigma)
sim_mse_B = simulate_mse([6.0, 8.0], sigma)
print(f"\nSimulated MSE (n={n_sim} trials):")
print(f"  Model A: {sim_mse_A:.4f} (analytical: {mse_A})")
print(f"  Model B: {sim_mse_B:.4f} (analytical: {mse_B})")
# Should be close to analytical values
```

9.5 Overfitting and Regularization

Plain English First

Overfitting is the variance problem made concrete: the model learns the training data — including its noise — so well that it performs poorly on new data. Regularization is the cure: add a penalty for complexity directly into the loss function, discouraging the optimizer from finding weights that are large in magnitude.

There are two canonical forms of regularization, and they have very different behaviors.

Formal Notation

The regularized loss adds a penalty term to the base loss $\mathcal{L}_0(\mathbf{w})$ (e.g., MSE or cross-entropy):

$$\mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 = \mathcal{L}_0(\mathbf{w}) + \lambda \sum_{j=1}^{d} w_j^2$$

$$\mathcal{L}_{\text{Lasso}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = \mathcal{L}_0(\mathbf{w}) + \lambda \sum_{j=1}^{d} |w_j|$$

The **hyperparameter** $\lambda \geq 0$ controls the strength of regularization. $\lambda = 0$ gives the unregularized model; large λ heavily penalizes large weights.

9.5.1 L2 Regularization (Ridge)

What it does: Shrinks all weights toward zero proportionally. Large weights are penalized quadratically, so all weights survive but in reduced magnitude.

The Ridge Normal Equation: For linear regression with L2 regularization, set the gradient of the penalized loss to zero:

$$\nabla_{\mathbf{w}}\left(\mathcal{L}_0 + \lambda|\mathbf{w}|^2\right) = 2X^TX\mathbf{w} - 2X^T\mathbf{y} + 2\lambda\mathbf{w} = \mathbf{0}$$

$$\implies \quad (X^TX + \lambda I)\mathbf{w} = X^T\mathbf{y} \quad \implies \quad \mathbf{w}_{\text{Ridge}} = (X^TX + \lambda I)^{-1}X^T\mathbf{y}$$

Here's why adding λI helps when X^TX is nearly singular: the eigenvalues of X^TX are non-negative (it is positive semi-definite), so some may be near zero, making the inverse unstable. Adding $\lambda > 0$ shifts all eigenvalues up by λ, making every eigenvalue at least $\lambda > 0$, so the matrix is now invertible. This is a stability improvement even beyond regularization.

Worked Example 9.5.1 — Ridge vs OLS

Using the same data $(1,2), (2,3), (3,5)$ from Section 9.3, apply Ridge with $\lambda = 1$.

OLS result (from Section 9.3): $\mathbf{w}_{\text{OLS}} = \begin{bmatrix}1.5 & 1/3\end{bmatrix}$

Ridge computation:

$$X^TX + \lambda I = \begin{bmatrix}14 & 6\\ 6 & 3\end{bmatrix} + \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} = \begin{bmatrix}15 & 6\\ 6 & 4\end{bmatrix}$$

$$\det\begin{bmatrix}15 & 6\\ 6 & 4\end{bmatrix} = 15\times 4 - 6\times 6 = 60 - 36 = 24$$

$$\left(X^TX + I\right)^{-1} = \frac{1}{24}\begin{bmatrix}4 & -6\\ -6 & 15\end{bmatrix} = \begin{bmatrix}1/6 & -1/4\\ -1/4 & 5/8\end{bmatrix}$$

$$\mathbf{w}_{\text{Ridge}} = \begin{bmatrix}1/6 & -1/4\\ -1/4 & 5/8\end{bmatrix}\begin{bmatrix}23\\ 10\end{bmatrix}$$

$$w_1 = \frac{23}{6} - \frac{10}{4} = 3.8333 - 2.5 = 1.3333$$

$$w_0 = -\frac{23}{4} + \frac{50}{8} = -5.75 + 6.25 = 0.5$$

Ridge result: $\mathbf{w}_{\text{Ridge}} = \begin{bmatrix}1.333 & 0.500\end{bmatrix}$

Comparison:

	w_1 (slope)	w_0 (intercept)
OLS (no regularization)	1.500	0.333
Ridge ($\lambda = 1$)	1.333	0.500

Ridge shrinks the slope from 1.5 toward 0 (reduced from 1.500 to 1.333). The intercept shifts because the weights trade off against each other. Both weights

moved toward smaller magnitude — the defining behavior of L2 regularization.

9.5.2 L1 Regularization (Lasso) and Why It Causes Sparsity

L1 regularization does something that L2 does not: it drives many weights to **exactly zero**, producing a **sparse** solution where only a subset of features matter.

Why does L1 cause sparsity?

There are two complementary ways to see this. Use whichever clicks for you.

Explanation 1 — The geometric argument:

Picture the weight space in 2D with weights w_1 and w_2. The unregularized loss function has elliptical level curves. The regularization term is a constraint: $|w_1| + |w_2| \leq t$ for some budget t. The L1 constraint defines a **diamond** (tilted square) with corners on the axes at $(\pm t, 0)$ and $(0, \pm t)$.

When you minimize the regularized loss, you are looking for the point where a level curve of the loss first touches the constraint region. The diamond has **corners** — pointed vertices on the axes. The loss's level curves are smooth ellipses. An ellipse expanding outward from its minimum will generically first touch the diamond at a **corner** (a sparse point with one coordinate $= 0$) rather than along a smooth edge. L2's constraint is a circle (no corners), so the first contact point is typically not on an axis.

Explanation 2 — The subgradient argument:

At a point where $w_j \neq 0$, the L1 penalty gradient is $\lambda \cdot \text{sign}(w_j)$. This pulls w_j toward zero at a fixed rate λ regardless of its magnitude.

At $w_j = 0$, the derivative of $|w_j|$ does not exist — but the **subgradient** is any value in $[-\lambda, \lambda]$. This means that if the loss's gradient $\partial \mathcal{L}_0 / \partial w_j$ at $w_j = 0$ is small (in $[-\lambda, \lambda]$), the subgradient condition is satisfied with $w_j = 0$ — meaning there is **no force to move w_j away from zero**. The weight stays pinned at exactly zero. This "dead zone" of width 2λ around zero is what creates sparsity.

L2 does not have this property: the L2 gradient is $2\lambda w_j$, which shrinks to zero as $w_j \to 0$, meaning the penalty asymptotically vanishes and never actually zeros out the weight.

9.5.3 Bayesian Interpretation of Regularization

Regularization is not an ad hoc trick — it is Bayesian inference in disguise. Recall Bayes' theorem from Chapter 7:

$$p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) \cdot p(\mathbf{w})$$

Taking the log:

$$\log p(\mathbf{w} \mid \mathbf{X}, \mathbf{y}) = \log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}) + \text{const}$$

The **MAP (Maximum A Posteriori)** estimate maximizes this, which is equivalent to minimizing:

$$-\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) - \log p(\mathbf{w})$$

Now choose a prior on $\mathbf{w}$:

Gaussian prior $p(\mathbf{w}) \propto \exp!\left(-\frac{|\mathbf{w}|_2^2}{2\tau^2}\right)$:

$$-\log p(\mathbf{w}) = \frac{|\mathbf{w}|_2^2}{2\tau^2} + \text{const} \propto |\mathbf{w}|_2^2$$

This is exactly the L2 penalty with $\lambda = \frac{1}{2\tau^2}$.

Laplace prior $p(\mathbf{w}) \propto \exp!\left(-\frac{|\mathbf{w}|_1}{b}\right)$:

$$-\log p(\mathbf{w}) = \frac{|\mathbf{w}|_1}{b} + \text{const} \propto |\mathbf{w}|_1$$

This is exactly the L1 penalty with $\lambda = \frac{1}{b}$.

Interpretation: When you add L2 regularization, you are saying "I believe the weights are small — my prior is Gaussian centered at zero." When you add L1 regularization, you are saying "I believe most weights are zero — my prior is Laplace (heavy-tailed, peaked sharply at zero), encouraging sparsity."

Regularization strength λ encodes how strongly you hold this belief. Small λ = weak prior (data dominates). Large λ = strong prior (regularizer dominates, model barely fits data).

Engineer's Angle

```
import math

# ── Ridge regression via closed form (2D, reusing inv2x2)
─────────────────

def mat_add(A, B):
    """Add two matrices element-wise."""
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in
range(len(A))]

def identity(n):
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in
range(n)]

def mat_transpose(A):
    rows, cols = len(A), len(A[0])
    return [[A[r][c] for r in range(rows)] for c in range(cols)]

def mat_mul(A, B):
    rows_A, cols_A = len(A), len(A[0])
    cols_B = len(B[0])
    C = [[0.0] * cols_B for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]
    return C

def mat_vec_mul(A, v):
    return [sum(A[i][j] * v[j] for j in range(len(v))) for i in
range(len(A))]

def inv2x2(A):
    a, b, c, d = A[0][0], A[0][1], A[1][0], A[1][1]
    det = a * d - b * c
    if abs(det) < 1e-12:
        raise ValueError("Singular matrix")
    return [[d / det, -b / det], [-c / det, a / det]]

def ridge_regression_2d(data, lam):
    """Ridge regression: w = (X^T X + lam * I)^{-1} X^T y."""
    X   = [[xi, 1.0] for xi, yi in data]
    y   = [yi for xi, yi in data]
    Xt  = mat_transpose(X)
    XtX = mat_mul(Xt, X)
    reg = mat_add(XtX, [[lam * e for e in row] for row in
identity(2)])
    Xty = mat_vec_mul(Xt, y)
    w   = mat_vec_mul(inv2x2(reg), Xty)
    return w[0], w[1]  # slope, intercept

data = [(1, 2), (2, 3), (3, 5)]

print("Lambda | w1 (slope) | w0 (intercept)")
```

```
print("-------|------------|---------------")
for lam in [0.0, 0.5, 1.0, 5.0, 10.0]:
    w1, w0 = ridge_regression_2d(data, lam)
    print(f"  {lam:4.1f} |   {w1:.6f} |    {w0:.6f}")
# As lambda increases, weights shrink toward 0

# ── Log prior comparison: Gaussian vs Laplace
──────────────────────────

def gaussian_log_prior(w, tau_sq):
    """log p(w) for Gaussian prior N(0, tau_sq)."""
    return -sum(wi**2 / (2 * tau_sq) for wi in w)

def laplace_log_prior(w, b):
    """log p(w) for Laplace prior with scale b."""
    return -sum(abs(wi) / b for wi in w)

w_sparse = [0.0, 1.5, 0.0, 0.2, 0.0]   # mostly zeros
w_dense  = [0.3, 0.3, 0.3, 0.3, 0.3]   # all small, same L2 norm

tau_sq, b = 1.0, 1.0

print("\nComparing priors:")
print(f"Sparse weights {w_sparse}:")
print(f"  Gaussian log-prior: {gaussian_log_prior(w_sparse,
tau_sq):.4f}")
print(f"  Laplace  log-prior: {laplace_log_prior(w_sparse,
b):.4f}")

print(f"\nDense weights  {w_dense}:")
print(f"  Gaussian log-prior: {gaussian_log_prior(w_dense,
tau_sq):.4f}")
print(f"  Laplace  log-prior: {laplace_log_prior(w_dense, b):.4f}")
# Laplace strongly prefers sparse over dense; Gaussian is more
neutral
```

9.6 Train / Validation / Test Split

Plain English First

You have a dataset. You want to report how good your model is. If you test it on the same data you trained it on, the number you get is meaningless — the model has already seen those examples and may have memorized them. You need data the model has **never seen**.

But there is a second, subtler trap: every time you tune a hyperparameter (learning rate, regularization strength, architecture choices) based on test-set performance, you are indirectly fitting the test set. After enough tuning

decisions, your model is implicitly optimized for the test set, making the test error an overly optimistic estimate.

The solution is three-way data splitting.

The Three Splits

Training set (typically 60–80% of data):

Used to fit the model parameters. The optimizer sees only these examples. The model's weights are updated based on training-set loss.

Validation set (typically 10–20% of data):

Used to make hyperparameter decisions: which regularization strength, which architecture, how many epochs. You run your model on the validation set and pick the version that performs best. Because you are looking at the validation score while making decisions, it is informing your choices and cannot be used as an unbiased performance estimate.

Test set (typically 10–20% of data):

Used exactly once. Report this number in your paper or to your stakeholders. Never make any model decision based on the test set — not architecture, not preprocessing, not feature selection. Once you have looked at the test set performance, it is "spent."

The Information Hierarchy

```
Training data → model parameters (weights)
Validation data → hyperparameters (lambda, architecture, epochs)
Test data → final performance estimate (reported once)
```

Each level uses data from the level above to evaluate choices, and is itself evaluated by the level below.

Why can't you just use training error? As model complexity increases, training error decreases monotonically — you can always memorize training data by adding more parameters. Training error cannot detect overfitting.

Why can't you just use test error? If you tune on test error (even indirectly, by running many experiments and selecting the best), the test error is no longer unbiased. Each time you look and adapt, you "spend" some of the test set's value.

Practical Guidance for Engineers

Dataset size	Recommended split	Validation strategy
< 1,000 examples	60/20/20	Use cross-validation (Section 9.7)
1,000 – 100,000	70/15/15	Hold-out validation
> 100,000	80/10/10	Hold-out (val set is large enough)
Time-series data	Chronological split only	No random shuffling — future cannot inform past

Time-series warning: For sequential data (financial, sensor, text with temporal structure), always split chronologically. Training on future data to predict the past is data leakage — a silent but fatal mistake.

Engineer's Angle

```
import random

def train_val_test_split(data, train_frac=0.70, val_frac=0.15,
seed=42):
    """
    Split data into train/val/test sets.
    test_frac = 1 - train_frac - val_frac.

    Returns (train, val, test) as lists.
    """
    rng = random.Random(seed)
    data_copy = list(data)
    rng.shuffle(data_copy)

    n = len(data_copy)
    n_train = int(n * train_frac)
    n_val   = int(n * val_frac)

    train = data_copy[:n_train]
    val   = data_copy[n_train:n_train + n_val]
    test  = data_copy[n_train + n_val:]
    return train, val, test

# Example: 1000 samples
data = list(range(1000))
train, val, test = train_val_test_split(data)
print(f"Train: {len(train)}, Val: {len(val)}, Test: {len(test)}")
# Train: 700, Val: 150, Test: 150

# Time-series: always chronological
def time_series_split(data, train_frac=0.70, val_frac=0.15):
    """Chronological split — NO shuffling."""
    n = len(data)
    n_train = int(n * train_frac)
    n_val   = int(n * val_frac)
    train = data[:n_train]
    val   = data[n_train:n_train + n_val]
    test  = data[n_train + n_val:]
    return train, val, test

# Pseudocode for hyperparameter tuning workflow:
#
# best_val_loss = float('inf')
# best_lambda = None
#
# for lam in [0.001, 0.01, 0.1, 1.0]:          # hyperparameter
search
#     model = train(train_set, lambda=lam)       # fit on train
#     val_loss = evaluate(model, val_set)        # score on val
#     if val_loss < best_val_loss:
#         best_val_loss = val_loss
#         best_lambda = lam
#
```

```
# final_model = train(train_set, lambda=best_lambda)
# test_loss = evaluate(final_model, test_set)    # report ONCE
# print(f"Test loss: {test_loss}")
```

9.7 Cross-Validation

Plain English First

A 70/15/15 split works well when you have thousands of examples. But when your dataset is small — say, 200 examples — a single validation fold of 30 examples is noisy. The performance estimate on 30 examples may vary a lot depending on which 30 you happened to set aside.

K-fold cross-validation solves this by getting k different performance estimates from the same data and averaging them. You train k different models, each time leaving out a different fold for validation. Every example is in the validation set exactly once.

How K-Fold Works

1. Randomly partition the dataset into k equally-sized folds: $F_1, F_2, \ldots, F_k$.
2. For $i = 1$ to k: a. **Train** on all folds except F_i (the "hold-out" fold). b. **Evaluate** on F_i. Record the performance metric (e.g., MSE or accuracy).
3. **Report** the mean (and optionally standard deviation) across the k fold scores.

Formal Notation

Let err_i be the validation error on fold i.

$$\widehat{\text{CV}}_k = \frac{1}{k} \sum_{i=1}^{k} \text{err}_i$$

The standard deviation $\text{std}(\text{err}_1, \ldots, \text{err}_k)$ quantifies the **stability** of the estimate.

Common choices of k:

- $k = 5$: good default. Low computational cost, reasonable variance.

- $k = 10$: slightly better statistical properties, $2\times$ cost of 5-fold.
- $k = n$ (leave-one-out cross-validation, LOO-CV): minimal bias, but requires n training runs. Used when n is very small.

When to Use Cross-Validation

Use it when:

- Dataset is small ($< 5{,}000$ examples) and you cannot afford to "waste" data on a fixed validation set.
- You are comparing multiple model configurations and need reliable estimates.
- You want error bars on your performance metric.

Do not use it as a substitute for a test set. Cross-validation gives you the validation error across folds. You still need a held-out test set that was never used to make any model decision.

Do not use it for time-series data. Use time-series cross-validation instead (each fold respects chronological order: train on past, validate on future).

Worked Example 9.7.1 — 5-Fold Cross-Validation

Dataset of 10 examples (for illustration). We use a constant-mean predictor and measure MSE.

True values: $\mathbf{y} = (1, 3, 2, 5, 4, 7, 6, 9, 8, 10)$.

Split into 5 folds of 2 examples each:

> **Note on fold assignment:** The table below assigns folds in sequential order (fold 1 = indices 0–1, fold 2 = indices 2–3, etc.) to keep the arithmetic easy to follow. The Python implementation in the Engineer's Angle section uses `rng.shuffle()` before splitting, so running the code will produce different fold assignments and different per-fold MSE values — though the cross-validated average should remain similar for well-mixed data.

Fold	Val indices	Val y	Train y (remaining 8)	Train mean	Fold MSE
1	0, 1	1, 3	2,5,4,7,6,9,8,10	$51/8 = 6.375$	$\frac{(1-6.375)^2+(3-6.375)^2}{2} = \frac{28.891+11.391}{2} = 20.141$
2	2, 3	2, 5	1,3,4,7,6,9,8,10	$48/8 = 6.0$	$\frac{(2-6)^2+(5-6)^2}{2} = \frac{16+1}{2} = 8.5$
3	4, 5	4, 7	1,3,2,5,6,9,8,10	$44/8 = 5.5$	$\frac{(4-5.5)^2+(7-5.5)^2}{2} = \frac{2.25+2.25}{2} = 2.25$
4	6, 7	6, 9	1,3,2,5,4,7,8,10	$40/8 = 5.0$	$\frac{(6-5)^2+(9-5)^2}{2} = \frac{1+16}{2} = 8.5$
5	8, 9	8, 10	1,3,2,5,4,7,6,9	$37/8 = 4.625$	$\frac{(8-4.625)^2+(10-4.625)^2}{2} = \frac{11.391+28.891}{2} = 20.141$

$$\widehat{\mathrm{CV}}_5 = \frac{20.141 + 8.5 + 2.25 + 8.5 + 20.141}{5} = \frac{59.532}{5} = 11.906$$

Verify fold 3 arithmetic:

$(4 - 5.5)^2 = (-1.5)^2 = 2.25$, $(7 - 5.5)^2 = (1.5)^2 = 2.25$. Average: $\frac{2.25+2.25}{2} = 2.25$. ✓

Verify fold 5 arithmetic:

Train mean $= (1 + 3 + 2 + 5 + 4 + 7 + 6 + 9)/8 = 37/8 = 4.625$. $(8 - 4.625)^2 = (3.375)^2 = 11.390625 \approx 11.391$. $(10 - 4.625)^2 = (5.375)^2 = 28.890625 \approx 28.891$. Average: $(11.391 + 28.891)/2 = 40.282/2 = 20.141$. ✓

> **Note:** The worked example above uses sequential (non-shuffled) fold assignment — fold 1 gets indices 0–1, fold 2 gets indices 2–3, and so on — so that the table arithmetic can be followed by hand. The Python code below follows best practice and shuffles the data before partitioning, which means the fold assignments and resulting MSE values will differ from the table. Both approaches are correct; the code is what you would use in practice.

Engineer's Angle

```
import math
import random
import statistics

def k_fold_split(data, k, seed=42):
    """
    Partition data into k approximately equal folds.
    Returns list of k folds, each a list of (index, item) pairs.
    Note: data is shuffled before partitioning (best practice), so
fold
    assignments differ from the sequential worked example above.
    """
    rng = random.Random(seed)
    indexed = list(enumerate(data))
    rng.shuffle(indexed)
    fold_size = len(indexed) // k
    folds = []
    for i in range(k):
        start = i * fold_size
        end = start + fold_size if i < k - 1 else len(indexed)
        folds.append(indexed[start:end])
    return folds

def constant_predictor_mse(train_y, val_y):
    """MSE of the constant (training mean) predictor on val_y."""
    mean_pred = statistics.mean(train_y)
    return statistics.mean((y - mean_pred) ** 2 for y in val_y)

# Dataset
y = [1, 3, 2, 5, 4, 7, 6, 9, 8, 10]
k = 5

folds = k_fold_split(y, k)
fold_mses = []

for i, val_fold in enumerate(folds):
    val_indices, val_y = zip(*val_fold)
    train_y = [y[idx] for j, fold in enumerate(folds) if j != i
               for idx, _ in fold]
    mse = constant_predictor_mse(list(train_y), list(val_y))
    fold_mses.append(mse)
    print(f"Fold {i+1}: val_y={list(val_y)}, train_mean=
{statistics.mean(train_y):.4f}, MSE={mse:.4f}")

cv_score = statistics.mean(fold_mses)
cv_std   = statistics.stdev(fold_mses)
print(f"\n5-Fold CV MSE: {cv_score:.4f} ± {cv_std:.4f}")
# Report both the mean and std — the std tells you how stable the
estimate is
```

9.8 Hypothesis Testing — Is Your Model Improvement Real?

Plain English First

You trained Model B and it scores 82% accuracy on the test set, up from Model A's 70%. Should you ship Model B? Not yet — you need to ask: **could this difference be due to random chance?**

If you only have 100 test examples, a 12-point accuracy difference might be a statistical accident. If you have 10,000 test examples, a 12-point difference is almost certainly real. Hypothesis testing gives you a rigorous framework to decide.

The key idea: you assume the two models are actually equally good (the **null hypothesis**) and ask: if that were true, how surprising is the observed difference? If the difference is very surprising under the null hypothesis, you reject it — the improvement is likely real.

Formal Notation

Null hypothesis (H_0): The two models have the same true accuracy. The observed difference is due to sampling variability.

Alternative hypothesis (H_1): The models have different true accuracies.

P-value: The probability, assuming H_0 is true, of observing a difference at least as extreme as the one observed.

$$p\text{-value} = P(\text{test statistic} \geq \text{observed} \mid H_0)$$

Decision rule: If p-value $< \alpha$ (the significance level, typically $\alpha = 0.05$), reject H_0. The improvement is statistically significant.

Warning on p-values: A small p-value does not mean the improvement is large or practically important. It means the data is inconsistent with the null hypothesis. A 0.1% improvement can be statistically significant with enough data; a 5% improvement can be statistically insignificant with too little data. Always pair p-values with **effect sizes** and **confidence intervals** (Section 9.9).

Two-Proportion Z-Test

For comparing two model accuracies, the appropriate test is the **two-proportion z-test**.

Setup: Model A is correct on k_A out of n_A test examples. Model B is correct on k_B out of n_B test examples. Observed accuracies: $\hat{p}_A = k_A/n_A$ and $\hat{p}_B = k_B/n_B$.

Pooled proportion (under H_0: both models have the same true accuracy):

$$\hat{p}_{\text{pool}} = \frac{k_A + k_B}{n_A + n_B}$$

Standard error of the difference (under H_0):

$$\text{SE} = \sqrt{\hat{p}_{\text{pool}}(1 - \hat{p}_{\text{pool}})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}$$

Z-statistic:

$$z = \frac{\hat{p}_B - \hat{p}_A}{\text{SE}}$$

Under H_0, z approximately follows $\mathcal{N}(0, 1)$ (using the Central Limit Theorem from Chapter 8). The two-tailed p-value is:

$$p\text{-value} = 2\big(1 - \Phi(|z|)\big)$$

where Φ is the standard normal CDF from Chapter 8.

Worked Example 9.8.1 — Is Model B Better?

Model A: 70 correct out of 100. $\hat{p}_A = 0.70$. Model B: 82 correct out of 100. $\hat{p}_B = 0.82$.

Step 1 — Pooled proportion:

$$\hat{p}_{\text{pool}} = \frac{70 + 82}{100 + 100} = \frac{152}{200} = 0.76$$

Step 2 — Standard error:

$$\text{SE} = \sqrt{0.76 \times 0.24 \times \left(\frac{1}{100} + \frac{1}{100}\right)} = \sqrt{0.1824 \times 0.02} = \sqrt{0.003648}$$

$$= 0.06040$$

Step 3 — Z-statistic:

$$z = \frac{0.82 - 0.70}{0.06040} = \frac{0.12}{0.06040} \approx 1.987$$

Step 4 — P-value:

$$\Phi(1.987) = \frac{1}{2}!\left[1 + \text{erf}!\left(\frac{1.987}{\sqrt{2}}\right)\right] = \frac{1}{2}![1 + \text{erf}(1.405)]$$

$\text{erf}(1.405) \approx 0.9531$, so $\Phi(1.987) \approx \frac{1}{2}(1.9531) = 0.9766$.

$$p\text{-value} = 2(1 - 0.9766) = 2(0.0234) = 0.0469$$

Decision: p-value $= 0.047 < 0.05$. We reject H_0 at the 5% significance level. The improvement from 70% to 82% on 100 test examples is statistically significant — it is unlikely to be random chance.

Important context: The result is barely significant ($p \approx 0.047$, just under the threshold of 0.05). This should make you cautious:

1. The effect might not replicate. A result at the borderline of significance is fragile.
2. Consider collecting more test data to get a more reliable estimate.
3. Check whether a 12-percentage-point improvement matters in your application (practical significance vs. statistical significance).

Engineer's Angle

```
import math

def phi(z):
    """CDF of standard normal N(0,1). Uses math.erf (stdlib)."""
    return 0.5 * (1.0 + math.erf(z / math.sqrt(2)))

def two_proportion_z_test(k_A, n_A, k_B, n_B):
    """
    Two-proportion z-test: H0 is that both models have equal true
accuracy.
    Returns (z_stat, p_value_two_tailed).
    """
    p_A = k_A / n_A
    p_B = k_B / n_B
    p_pool = (k_A + k_B) / (n_A + n_B)

    se = math.sqrt(p_pool * (1 - p_pool) * (1 / n_A + 1 / n_B))
    z  = (p_B - p_A) / se
    p_value = 2 * (1 - phi(abs(z)))

    print(f"p_A = {p_A:.4f}, p_B = {p_B:.4f}")
    print(f"p_pool = {p_pool:.4f}, SE = {se:.6f}")
    print(f"z = {z:.4f}, p-value = {p_value:.6f}")

    if p_value < 0.05:
        print(f"Result: SIGNIFICANT at alpha=0.05 (p={p_value:.4f}
< 0.05)")
    else:
        print(f"Result: NOT significant at alpha=0.05 (p=
{p_value:.4f} >= 0.05)")

    return z, p_value

print("Example: 70 vs 82 out of 100")
z, pv = two_proportion_z_test(k_A=70, n_A=100, k_B=82, n_B=100)
# p-value ≈ 0.047 → significant

print("\nExample: 70 vs 75 out of 100 (small difference)")
z2, pv2 = two_proportion_z_test(k_A=70, n_A=100, k_B=75, n_B=100)
# p-value ≈ 0.35 → not significant

print("\nExample: 700 vs 820 out of 1000 (same difference, more
data)")
z3, pv3 = two_proportion_z_test(k_A=700, n_A=1000, k_B=820,
n_B=1000)
# p-value << 0.001 → highly significant
# Same 12-point gap, 10x more data → much stronger evidence
```

9.9 Confidence Intervals

Plain English First

A point estimate like "our model achieves 82% accuracy" is incomplete. It tells you the center of the estimate but nothing about the uncertainty. A confidence interval gives you a range that quantifies that uncertainty.

The common reading of a 95% confidence interval $[\hat{p} - E, \hat{p} + E]$ is: "there is a 95% probability that the true accuracy lies in this range."

This reading is wrong.

Here is the correct interpretation, and why the wrong reading is so tempting but incorrect.

The Correct Interpretation

A 95% confidence interval is constructed by a procedure. If you repeat the procedure — collect a new sample, compute the interval — across many repetitions, **95% of the intervals you construct will contain the true parameter**.

The true parameter p^* is a fixed (unknown) number — not a random variable. The **interval** is the random object. Once you compute a specific interval $[0.7412, 0.8988]$, that interval either contains p^* or it doesn't — there is no probability to assign to it. The 95% refers to the **long-run frequency** of the construction procedure, not to the probability that any particular realized interval is correct.

Why the common reading feels natural: After constructing the interval, you want to say something about the probability of p^*. That impulse is valid — it is the **Bayesian** question. In Bayesian inference, after seeing data, you compute a **credible interval**: a range that contains the true parameter with 95% posterior probability. That is a probability statement about p^*. The frequentist confidence interval is not.

Practical implication: "I am 95% confident" means "the procedure that generated this interval is correct 95% of the time." It is a statement about the reliability of your measurement process, not a probability about this specific interval.

Formal Notation

For a proportion $\hat{p}$ estimated from n observations, the **Wald confidence interval** at level $1-\alpha$ is:

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

where $z_{\alpha/2}$ is the critical value from the standard normal such that $\Phi(z_{\alpha/2}) = 1-\alpha/2$.

For $\alpha = 0.05$ (95% CI): $z_{0.025} = 1.96$ (because $\Phi(1.96) \approx 0.975$).

For $\alpha = 0.01$ (99% CI): $z_{0.005} \approx 2.576$.

Trust this result: $z_{0.025} = 1.96$ is the value such that 2.5% of the standard normal distribution lies above it. It comes from $\Phi^{-1}(0.975)$, which does not have a closed form but is a standard table entry.

Worked Example 9.9.1 — Accuracy Confidence Interval

Model B achieved 82% accuracy on $n = 100$ test examples. Construct a 95% confidence interval for the true accuracy p^*.

Step 1 — Point estimate: $\hat{p} = 82/100 = 0.82$.

Step 2 — Standard error:

$$\text{SE} = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} = \sqrt{\frac{0.82 \times 0.18}{100}} = \sqrt{\frac{0.1476}{100}} = \sqrt{0.001476} = 0.03842$$

Step 3 — Margin of error:

$$E = 1.96 \times 0.03842 = 0.07530$$

Step 4 — Confidence interval:

$$[0.82 - 0.0753,; 0.82 + 0.0753] = [0.7447,; 0.8953]$$

Correct interpretation: The procedure used to construct this interval has the property that 95% of such intervals contain the true accuracy. This specific interval $[0.745, 0.895]$ is a realization of that procedure.

Practical reading: The true accuracy is somewhere in the range 74.5% to 89.5%. That is a wide interval — a 15-percentage-point spread — because $n = 100$ is not many test examples.

Worked Example 9.9.2 — How Many Examples Do You Need?

How large a test set is needed for a 95% CI with margin of error $E = \pm 2$ around an accuracy of $\hat{p} = 0.82$?

Rearrange $E = 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ for n:

$$n = \frac{(1.96)^2 \hat{p}(1-\hat{p})}{E^2} = \frac{3.8416 \times 0.82 \times 0.18}{(0.02)^2}$$

$$= \frac{3.8416 \times 0.1476}{0.0004} = \frac{0.5672}{0.0004} = 1418$$

You need approximately 1,418 test examples to achieve a 95% CI with half-width of 2 percentage points. With only 100 test examples (as in Example 9.9.1), the half-width is about 7.5 percentage points — four times wider.

Test set size	95% CI half-width (at $\hat{p} = 0.82$)
100	$\pm 7.5\%$
400	$\pm 3.8\%$
1,000	$\pm 2.4\%$
1,418	$\pm 2.0\%$
10,000	$\pm 0.75\%$

The key insight: Halving the confidence interval width requires quadrupling the test set size (because SE scales as $1/\sqrt{n}$). Evaluating models reliably is expensive.

Engineer's Angle

```
import math

def phi(z):
    """CDF of standard normal."""
    return 0.5 * (1.0 + math.erf(z / math.sqrt(2)))

def proportion_ci(k, n, confidence=0.95):
    """
    Wald confidence interval for a proportion.
    k: number of successes.
    n: total trials.
    Returns (point_estimate, lower, upper, margin_of_error).
    """
    p_hat = k / n
    alpha  = 1.0 - confidence
    # z_{alpha/2}: standard normal critical value
    # Use bisection to find z such that Phi(z) = 1 - alpha/2
    target = 1.0 - alpha / 2
    lo, hi = 0.0, 5.0
    for _ in range(60):          # 60 iterations → precision ~1e-18
        mid = (lo + hi) / 2
        if phi(mid) < target:
            lo = mid
        else:
            hi = mid
    z_crit = (lo + hi) / 2

    se  = math.sqrt(p_hat * (1 - p_hat) / n)
    moe = z_crit * se
    return p_hat, p_hat - moe, p_hat + moe, moe

# Model B: 82/100
p_hat, lo, hi, moe = proportion_ci(82, 100)
print(f"p_hat = {p_hat:.4f}")
print(f"95% CI: [{lo:.4f}, {hi:.4f}]  (±{moe:.4f})")
# [0.7447, 0.8953] — ±0.0753

# How many samples for ±2% CI?
def required_sample_size(p_hat, margin, confidence=0.95):
    """Minimum n for Wald CI with given half-width margin."""
    alpha  = 1.0 - confidence
    target = 1.0 - alpha / 2
    lo, hi = 0.0, 5.0
    for _ in range(60):
        mid = (lo + hi) / 2
        (lo, hi) = (mid, hi) if phi(mid) < target else (lo, mid)
    z_crit = (lo + hi) / 2
    return math.ceil(z_crit**2 * p_hat * (1 - p_hat) / margin**2)

n_needed = required_sample_size(0.82, 0.02)
print(f"\nFor ±2% CI at 95% confidence: need n = {n_needed}")   #
1418

print("\nCI widths by test set size:")
```

```
for n in [100, 400, 1000, 1418, 10000]:
    _, lo, hi, moe = proportion_ci(int(0.82 * n), n)
    print(f"  n={n:6d}: ±{moe:.4f}  [{lo:.4f}, {hi:.4f}]")
```

9.10 Connecting Everything: The Full Model Evaluation Pipeline

This chapter has built a complete framework for statistical decision-making in machine learning. Here is how the pieces connect in a real evaluation pipeline.

During training: MLE (Section 9.2) tells you which loss to use. For regression under Gaussian noise, minimize MSE. For classification, minimize cross-entropy. These are not arbitrary — they are maximum likelihood under the appropriate probabilistic model.

Choosing complexity: The bias-variance tradeoff (Section 9.4) tells you that model error decomposes into bias, variance, and irreducible noise. Use cross-validation (Section 9.7) on the validation set to find the model complexity (and regularization strength from Section 9.5) that minimizes total validation error.

Regularization decisions: Use Ridge (L2) when you want to shrink all weights smoothly. Use Lasso (L1) when you expect most features to be irrelevant and want a sparse model. Interpret both as MAP estimation under a prior on weights (Section 9.5.3).

Reporting results: Report a confidence interval (Section 9.9), not just a point estimate. For any improvement claim, run a hypothesis test (Section 9.8) to verify the difference is not due to chance. Never look at the test set more than once.

The closed-form exception: When d is small (< ~10,000 features) and the model is linear, the Normal Equation (Section 9.3) gives the exact MLE solution without gradient descent. Know when to use it.

9.11 Summary Table

Concept	Formula / Rule	ML Application
Point estimate	$\hat{\theta} =$ function of data	Weight vector, predicted probability
MLE	$\hat{\theta} = \arg\max_\theta \sum_i \log p(x_i \mid \theta)$	Foundation of all standard loss functions
Bernoulli MLE	$\hat{p} = k/n$	Classifier calibration, A/B test counts
Gaussian mean MLE	$\hat{\mu} = \bar{x}$	Regression, Gaussian VAE encoder mean
Log-likelihood trick	$\log \prod_i p_i = \sum_i \log p_i$	Numerical stability of all training
Normal Equation	$\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$	Exact closed-form linear regression
Use Normal Eq. when	$d \lesssim 10{,}000$, data fits in RAM	Small tabular datasets, prototyping
Bias^2	$(\mathbb{E}[\hat{f}(x)] - f(x))^2$	Underfitting, too-simple models
Variance	$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$	Overfitting, too-complex models
MSE decomposition	Bias^2 + Variance + σ^2	Guide model complexity selection
Ridge (L2)	$+\lambda\lvert\mathbf{w}\rvert_2^2$; $(X^T X + \lambda I)^{-1} X^T \mathbf{y}$	Shrinks all weights, stabilizes inversion
Lasso (L1)	$+\lambda\lvert\mathbf{w}\rvert_1$	Drives weights to exactly zero (sparsity)

Bayesian connection	Gaussian prior $\Rightarrow$ L2; Laplace prior $\Rightarrow$ L1	Regularization is MAP inference
Train/Val/Test	Train: fit params; Val: fit hyperparams; Test: report once	Core evaluation discipline
K-fold CV	$\widehat{\text{CV}}_k = \frac{1}{k}\sum_i \text{err}_i$	Reliable estimates on small datasets
Two-proportion z-test	$z = (\hat{p}_B - \hat{p}_A)/\text{SE}$	Testing whether model improvement is real
P-value	$P(\text{observed diff} \mid H_0)$	Surprisingness of result under null
95% CI for proportion	$\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$	Uncertainty in reported accuracy
CI misconception	"95% of constructed intervals contain θ^*" (not: "θ^* is in this interval with 95% prob.")	Correct frequentist interpretation

---

9.12 Exercises

Exercise 9.1 [Easy] — MLE for Bernoulli

A binary spam classifier is evaluated on 8 emails, and correctly predicts 6 of them.

(a) Write the log-likelihood function $\ell(p)$ for this data under a Bernoulli model.

(b) Differentiate and solve to find $\hat{p}_{\text{MLE}}$.

(c) Evaluate the log-likelihood at $\hat{p}_{\text{MLE}}$ and $p = 0.9$. Confirm that $\hat{p}_{\text{MLE}}$ gives the higher value.

(d) Why does the MLE estimate make intuitive sense?

Solution:

(a) $n = 8$, $k = 6$. The log-likelihood is:

$$\ell(p) = k \log p + (n - k) \log(1 - p) = 6 \log p + 2 \log(1 - p)$$

(b) Differentiating and setting to zero:

$$\frac{d\ell}{dp} = \frac{6}{p} - \frac{2}{1 - p} = 0$$

$$6(1 - p) = 2p$$

$$6 - 6p = 2p$$

$$6 = 8p$$

$$\hat{p}_{\text{MLE}} = \frac{6}{8} = 0.75$$

(c) Evaluating at $\hat{p} = 0.75$:

$$\ell(0.75) = 6 \log(0.75) + 2 \log(0.25)$$

$$= 6(-0.2877) + 2(-1.3863)$$

$$= -1.7262 - 2.7726 = -4.4988$$

Evaluating at $p = 0.9$:

$$\ell(0.9) = 6 \log(0.9) + 2 \log(0.1)$$

$$= 6(-0.1054) + 2(-2.3026)$$

$$= -0.6322 - 4.6052 = -5.2374$$

Since $-4.4988 > -5.2374$, the MLE at $p = 0.75$ has higher log-likelihood than $p = 0.9$. ✓

(d) $\hat{p}_{\text{MLE}} = 6/8 = 0.75$ is simply the observed success rate. Intuitively, if you see 6 successes in 8 trials, the most natural estimate for the underlying success probability is the fraction you observed. MLE confirms this intuition rigorously.

Exercise 9.2 [Easy] — Normal Equation

Fit a linear model $\hat{y} = w_1 x + w_0$ to the data points $(0, 1)$, $(1, 3)$, $(2, 4)$ using the Normal Equation.

(a) Write the design matrix X and target vector $\mathbf{y}$.

(b) Compute X^TX and $X^T\mathbf{y}$.

(c) Compute $(X^TX)^{-1}$ using the 2×2 inversion formula.

(d) Compute $\mathbf{w} = (X^TX)^{-1}X^T\mathbf{y}$ and state the fitted model.

(e) Verify that the gradient condition $X^TX\mathbf{w} = X^T\mathbf{y}$ holds.

Solution:

(a)

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 3 \\ 4 \end{bmatrix}$$

(b)

$$X^TX = \begin{bmatrix} 0 & 1 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0+1+4 & 0+1+2+1 \\ 1+1+2+1 & 1+1+1+1 \end{bmatrix} = \begin{bmatrix} 5 & 4 \\ 4 & 4 \end{bmatrix}$$

$$X^T\mathbf{y} = \begin{bmatrix} 0 & 1 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 4 \\ \end{bmatrix} = \begin{bmatrix} 0+3+8 \\ 1+3+4 \end{bmatrix} = \begin{bmatrix} 11 \\ 8 \end{bmatrix}$$

(c)

$$\det \begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix} = 5 \times 3 - 3 \times 3 = 15 - 9 = 6$$

$$(X^TX)^{-1} = \frac{1}{6} \begin{bmatrix} 3 & -3 \\ -3 & 5 \end{bmatrix} = \begin{bmatrix} 1/2 & -1/2 \\ -1/2 & 5/6 \end{bmatrix}$$

(d)

$$w_1 = \frac{1}{2}(11) + \left(-\frac{1}{2}\right)(8) = 5.5 - 4.0 = 1.5$$

$$w_0 = \left(-\frac{1}{2}\right)(11) + \frac{5}{6}(8) = -5.5 + \frac{40}{6} = -5.5 + 6.\overline{6} = 1.1\overline{6} = \frac{7}{6}$$

Fitted model: $\hat{y} = 1.5x + \frac{7}{6} \approx 1.5x + 1.167$

(e) Verify $X^TX\mathbf{w} = X^T\mathbf{y}$:

$$\begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} 3/2 \\ 7/6 \end{bmatrix} = \begin{bmatrix} 5(3/2) + 3(7/6) \\ 3(3/2) + 3(7/6) \end{bmatrix} = \begin{bmatrix} 15/2 + 7/2 \\ 9 \end{bmatrix}$$

Exercise 9.3 [Medium] — Bias-Variance Decomposition

True function: $f(x) = x^2$. Test point: $x_0 = 2$, so $f(2) = 4$. Noise: $\sigma^2 = 1$.

You have two models:

- **Model P** (polynomial): predicts $\hat{f}_P(2) \in 3.5, 4.0, 4.5, 4.0$ with equal probability across four training datasets.
- **Model C** (constant): always predicts $\hat{f}_C(x) = 2$ regardless of training data.

(a) Compute Bias^2, Variance, and MSE for Model P.

(b) Compute Bias^2, Variance, and MSE for Model C.

(c) Which model has lower MSE? Which would you prefer, and why?

(d) If you doubled the training set size, which component (bias or variance) of Model P's error would decrease? What about Model C?

Solution:

(a) Model P:

$$\mathbb{E}[\hat{f}_P(2)] = \frac{3.5 + 4.0 + 4.5 + 4.0}{4} = \frac{16}{4} = 4$$

$$\text{Bias}_P = 4 - 4 = 0, \qquad \text{Bias}_P^2 = 0$$

$$\text{Var}_P = \frac{(3.5-4)^2 + (4.0-4)^2 + (4.5-4)^2 + (4.0-4)^2}{4}$$

$$= \frac{0.25 + 0 + 0.25 + 0}{4} = \frac{0.5}{4} = 0.125$$

$$\text{MSE}_P = 0 + 0.125 + 1 = 1.125$$

(b) Model C:

$$\mathbb{E}[\hat{f}_C(2)] = 2 \quad \text{(deterministic)}$$

$$\text{Bias}_C = 2 - 4 = -2, \qquad \text{Bias}_C^2 = 4$$

$$\text{Var}_C = 0 \quad \text{(constant predictor)}$$

$$\text{MSE}_C = 4 + 0 + 1 = 5$$

(c) Model P has MSE = 1.125 vs Model C's MSE = 5. **Model P is much better.** Model C's large bias (-2) overwhelms its zero variance. The constant predictor of 2 is far from the true value of 4.

(d) More training data helps reduce **variance** but not bias.

For Model P: More data would reduce variance (the spread of predictions across training sets shrinks). Model P's bias is already zero, so more data

does not help further with bias.

For Model C: More data does not help at all — it always predicts 2 regardless, so its bias stays at -2 and its variance stays at 0. More data cannot fix a model that ignores the input.

Exercise 9.4 [Medium] — Regularization and the Bayesian Interpretation

A linear regression model has learned weight $w = 3.0$ on a dataset with $n = 50$ examples.

(a) With L2 regularization ($\lambda = 0.5$), the regularized loss adds $\lambda w^2 = 0.5 \times 9 = 4.5$ to the MSE. Suppose the unregularized gradient at $w = 3.0$ is $\partial \mathcal{L}_0 / \partial w = -0.2$ (pushing w larger). Write the total gradient (MSE + L2 penalty) at $w = 3.0$.

(b) With L1 regularization ($\lambda = 0.5$) and the same conditions, write the total gradient at $w = 3.0$.

(c) Now consider the point $w = 0.03$. With L2 ($\lambda = 0.5$), the L2 gradient is $2\lambda w = 1 \times 0.03 = 0.03$. Suppose the data gradient is still -0.2. What is the total gradient, and which direction does the update push w?

(d) With L1 ($\lambda = 0.5$) at $w = 0.03$, the L1 subgradient is $+\lambda = +0.5$ (positive, pushing toward zero). Suppose the data gradient is -0.2. What is the total (sub)gradient? In which direction does the update push w? Contrast with the L2 case from (c).

(e) Suppose instead $w = 0.0$ and the data gradient is -0.2 (pushing toward positive w). With L1 ($\lambda = 0.5$), the subgradient of $|w|$ at $w = 0$ is any value in $[-0.5, 0.5]$. Does there exist a subgradient value that makes the total (sub)gradient zero? What does this mean for w?

Solution:

(a) The L2 gradient contribution is $2\lambda w = 2(0.5)(3.0) = 3.0$.

Total gradient: $\frac{\partial \mathcal{L}_0}{\partial w} + 2\lambda w = -0.2 + 3.0 = +2.8$

The total gradient is positive, pushing w downward (away from 3.0, toward smaller values). The L2 penalty dominates.

(b) The L1 gradient contribution (for $w > 0$) is $\lambda \cdot \text{sign}(w) = 0.5 \times (+1) = +0.5$.

Total gradient: $-0.2 + 0.5 = +0.3$

Again positive — pushing w downward.

(c) Total gradient at $w = 0.03$ with L2: $-0.2 + 0.03 = -0.17$.

The total gradient is **negative** — pushing w upward (toward positive values). The data gradient of -0.2 dominates the small L2 penalty of 0.03. The weight will increase.

(d) Total (sub)gradient at $w = 0.03$ with L1: $-0.2 + 0.5 = +0.3$.

The total subgradient is **positive** — pushing w downward toward zero. This is the key difference from L2: even when w is very small (0.03), the L1 penalty contributes its full magnitude $\lambda = 0.5$, which here overcomes the data gradient and forces w toward zero.

With L2, the penalty shrank to 0.03 and the data gradient won. With L1, the penalty stays at 0.5 and the data gradient (-0.2) loses.

(e) At $w = 0$, the L1 subgradient $g \in [-0.5, 0.5]$. Total (sub)gradient: $-0.2 + g$.

Setting to zero: $g = 0.2$. Since $0.2 \in [-0.5, 0.5]$, **yes**, there is a valid subgradient that makes the total zero.

Meaning: The optimality condition is satisfied with $w = 0$. The point $w = 0$ is a minimum of the L1-regularized loss even though the data gradient is -0.2 (wanting to push w positive). The L1 subgradient "absorbs" the data gradient's pull, keeping the weight exactly at zero. This is the mechanism of L1 sparsity: a "dead zone" of width 2λ around zero where the L1 penalty prevents any movement away from $w = 0$.

Exercise 9.5 [Hard] — Full Evaluation Pipeline

A team trains two image classifiers on 8,000 training examples. They evaluate on a 1,000-example test set. Model A achieves 88% accuracy (880/1000 correct). Model B achieves 91% accuracy (910/1000 correct).

(a) Compute a 95% confidence interval for each model's true accuracy.

(b) Run a two-proportion z-test for $H_0 : p_A = p_B$. Compute z and the p-value. Is the improvement statistically significant at $\alpha = 0.05$?

(c) How large would the test set need to be for a 95% CI of $\pm 1\%$ on each model's accuracy (using Model B's $\hat{p} = 0.91$)?

(d) The team wants to now tune Model B's regularization strength using the test set (trying $\lambda \in 0.01, 0.1, 1.0$ and picking the best). Explain why this is problematic and what they should have done instead.

(e) Suppose the team ran the hypothesis test from (b) on 20 different pairs of models, using $\alpha = 0.05$ each time. Even if all models are equally good, how many "significant" results would you expect by chance? What does this imply about interpreting multiple comparisons?

Solution:

(a) Using $z_{0.025} = 1.96$:

Model A ($\hat{p}_A = 0.88$, $n = 1000$):

$$\text{SE}_A = \sqrt{\frac{0.88 \times 0.12}{1000}} = \sqrt{\frac{0.1056}{1000}} = \sqrt{0.0001056} = 0.010276$$

$$E_A = 1.96 \times 0.010276 = 0.020141$$

$$\text{CI}_A = [0.88 - 0.0201,; 0.88 + 0.0201] = [0.8599,; 0.9001]$$

Model B ($\hat{p}_B = 0.91$, $n = 1000$):

$$\text{SE}_B = \sqrt{\frac{0.91 \times 0.09}{1000}} = \sqrt{\frac{0.0819}{1000}} = \sqrt{0.0000819} = 0.009050$$

$$E_B = 1.96 \times 0.009050 = 0.017738$$

$$\text{CI}_B = [0.91 - 0.0177,; 0.91 + 0.0177] = [0.8923,; 0.9277]$$

Note that the confidence intervals barely overlap. This is suggestive, but CI overlap is not a reliable significance test — always run the hypothesis test directly, as in part (b).

(b) Two-proportion z-test:

$$\hat{p}_{\text{pool}} = \frac{880 + 910}{1000 + 1000} = \frac{1790}{2000} = 0.895$$

$$\text{SE} = \sqrt{0.895 \times 0.105 \times \left(\frac{1}{1000} + \frac{1}{1000}\right)} = \sqrt{0.09398 \times 0.002} = \sqrt{0.000187}$$

$$= 0.013710$$

$$z = \frac{0.91 - 0.88}{0.013710} = \frac{0.03}{0.013710} = 2.188$$

$$\Phi(2.188) = \frac{1}{2}!\left[1 + \text{erf}!\left(\frac{2.188}{\sqrt{2}}\right)\right] = \frac{1}{2}[1 + \text{erf}(1.547)] \approx \frac{1}{2}[1 + 0.9714] = 0.!$$

$$p\text{-value} = 2(1 - 0.9857) = 2(0.0143) = 0.0286$$

Since $0.0286 < 0.05$, the improvement is **statistically significant** at $\alpha = 0.05$. ✓

(c) Required n for $\hat{p} = 0.91$, $E = 0.01$:

$$n = \frac{(1.96)^2 \times 0.91 \times 0.09}{(0.01)^2} = \frac{3.8416 \times 0.0819}{0.0001} = \frac{0.31463}{0.0001} = 3146.3$$

Required test set size: $n \approx 3{,}147$ examples for a ±1% CI at 95% confidence.

(d) This is **test set contamination**. By choosing λ based on test set performance, the team is indirectly fitting the test set. After testing three values of λ, the test error they report is no longer an unbiased estimate of generalization performance — it optimistically reflects the one λ that happened to work best on this particular test set.

What they should have done: Reserve a validation set from the beginning, use it to tune λ, then evaluate the final model on the test set exactly once. If they want to tune λ at this point (no separate validation set), they must collect a new, never-before-seen test set.

(e) Under H_0 (all models equally good), each test has a 5% false-positive rate — a 5% chance of incorrectly declaring significance. Running 20 independent tests:

$$\text{Expected false positives} = 20 \times 0.05 = 1 \text{ false "significant" result}$$

This is the **multiple comparisons problem**. With 20 tests, you expect 1 spurious "significant" result purely by chance even if none of the models differ. To control the overall false-positive rate at 5%, apply a correction such as **Bonferroni correction**: use $\alpha' = 0.05/20 = 0.0025$ for each individual test, so that the family-wise error rate stays at 5%.

The implication: in any ML research with many ablations, hyperparameter comparisons, or architecture trials, some reported improvements may be

statistical accidents. Treating every individual comparison as independent and applying a standard threshold is statistically incorrect. Be especially skeptical of results that are "just" significant on many comparisons.

Next: Chapter 10 — Putting It All Together: a complete worked example integrating linear algebra, calculus, probability, and statistics to analyze a real machine learning system end to end.

Chapter 10: Putting It All Together — Math Inside a Neural Network

> *"A neural network is not magic. It is matrix multiplication, followed by a nonlinearity, followed by calculus, followed by probability. You know all of that now."*

10.0 What This Chapter Does

You have spent nine chapters building four mathematical toolkits:

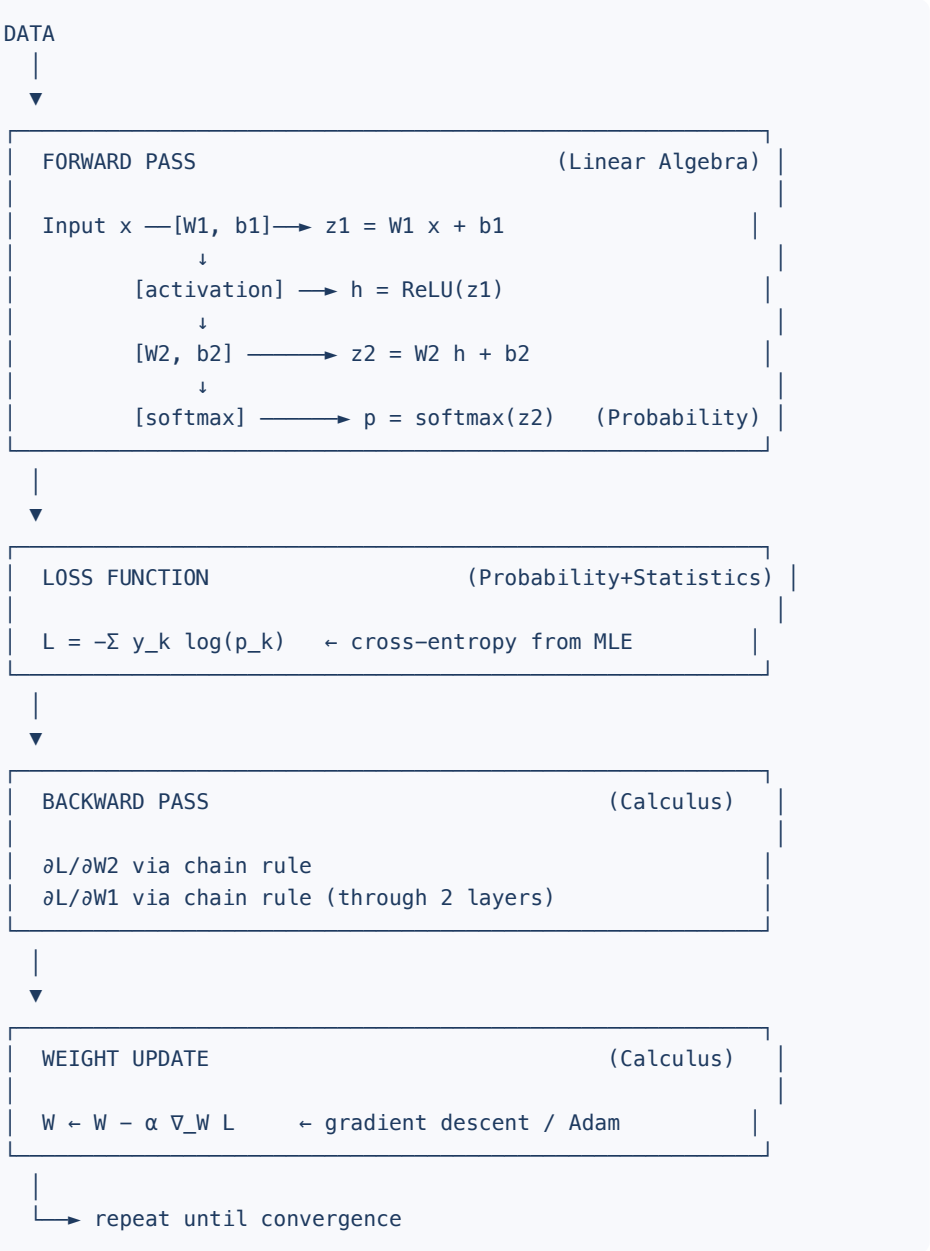
Toolkit	Chapters	Core ideas
Linear Algebra	2–4	Vectors, matrices, dot products, SVD, PCA
Calculus	5–6	Derivatives, chain rule, gradient descent, Adam
Probability	7–8	Bayes, Gaussian, KL divergence, cross-entropy, softmax
Statistics	9	MLE, bias-variance tradeoff, regularization, cross-validation

This chapter shows how those four toolkits assemble into a single working machine: a **neural network**. Not conceptually — concretely. We will derive every equation from first principles, then implement a 2-layer network from scratch in pure Python (no numpy, no PyTorch) and watch it solve the XOR problem.

Every equation will be tagged with the chapter that introduced it. By the end, you should be able to look at any line of a real training loop and say, *"That's the chain rule from Ch 5"* or *"That's MLE from Ch 9."*

10.1 The Big Picture — How the Four Pillars Combine

Here is the complete flow through a neural network:



This loop — forward pass, loss, backward pass, update — is **all** that happens during training. The math from this book tells us exactly what to compute at every step.

10.2 The Forward Pass — Computing a Prediction

10.2.1 The Linear Layer: $\mathbf{h} = W\mathbf{x} + \mathbf{b}$

The fundamental operation in every neural network layer is a **linear transformation** — a matrix-vector product followed by adding a bias vector.

For a layer with n_{in} input features and n_{out} neurons:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b} \quad \text{(Ch 3 — matrix-vector product)}$$

where:

- $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}}$ is the input vector
- $W \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}}$ is the **weight matrix** — each row is the weights for one neuron
- $\mathbf{b} \in \mathbb{R}^{n_{\text{out}}}$ is the **bias vector**
- $\mathbf{z} \in \mathbb{R}^{n_{\text{out}}}$ is the **pre-activation** output

In component form (from Ch 3):

$$z_j = \sum_{i=1}^{n_{\text{in}}} W_{ji}, x_i + b_j, \quad j = 1, \ldots, n_{\text{out}}$$

This is the dot product of the j-th row of W with $\mathbf{x}$, plus a bias. It computes a weighted sum of the inputs for neuron j.

Why matrices? A layer with n_{out} neurons, each with n_{in} inputs, needs $n_{\text{out}} \times n_{\text{in}}$ weight values. Packing them into a matrix W lets us compute all neurons simultaneously with a single matrix-vector multiply — exactly the operation we studied in Chapter 3.

Concrete example for our 2→4 network:

$$W_1 = \begin{bmatrix} w_{11} & w_{12} \, w_{21} & w_{22} \, w_{31} & w_{32} \, w_{41} & w_{42} \end{bmatrix} \in \mathbb{R}^{4\times 2}, \quad \mathbf{x} = \begin{bmatrix} x_1 \, x_2 \end{bmatrix}, \quad \mathbf{b}_1$$

$$\mathbf{z}_1 = W_1\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^4 \quad \text{(Ch 3)}$$

10.2.2 ReLU Activation — Why We Need Nonlinearity

After the linear layer, we apply an **activation function**. Without it, the whole network would just be a single big linear transformation — it could only

learn linear boundaries. Activation functions introduce nonlinearity, giving the network expressive power.

The most common activation is **ReLU** (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z) \quad \text{(Ch 5)}$$

Applied element-wise to a vector:

$$\mathbf{h} = \text{ReLU}(\mathbf{z}1) = \begin{bmatrix} \max(0, z_{1,1}) \\ \max(0, z_{1,2}) \\ \vdots \\ \max(0, z_{1,4}) \end{bmatrix}$$

The derivative of ReLU (needed for backpropagation) is the step function:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad \text{(Ch 5 — derivative of a piecewise fun}$$

Why ReLU works: ReLU is not differentiable at exactly $z = 0$, but in practice we set the derivative to 0 there (the "subgradient" convention). The key properties are:

- **Cheap to compute:** just a max with zero
- **No vanishing gradient on the positive side:** derivative is exactly 1, so gradients flow through ReLU-active neurons without attenuation
- **Creates sparsity:** neurons with negative pre-activation output exactly 0, which often helps generalization

Why we can't use a linear activation: if $h = W_2(W_1 x + b_1) + b_2$, this equals $(W_2 W_1)x + (W_2 b_1 + b_2)$, which is just another linear transformation. The whole network collapses to one layer.

10.2.3 Softmax Output — Converting Scores to Probabilities

The second (output) layer produces raw **logit scores** $\mathbf{z}2 \in \mathbb{R}^{n_{\text{out}}}$. To interpret these as probabilities over classes, we apply **softmax** (from Ch 8):

$$p_k = \text{softmax}(\mathbf{z}2)k = \frac{e^{z_{2,k}}}{\displaystyle\sum_{j=1}^{n_{\text{out}}} e^{z_{2,j}}}, \quad k = 1, \ldots, n_{\text{out}} \quad \text{(Ch 8)}$$

Properties (from Ch 8):

- $p_k > 0$ for all k (probabilities are positive)

- $\sum_k p_k = 1$ (they sum to 1 — a valid probability distribution)
- Large $z_{2,k} \to$ large p_k (the class with the highest logit gets the most probability mass)

Numerical stability (from Ch 8): In practice we subtract the maximum logit before exponentiating:

$$p_k = \frac{e^{z_{2,k} - \max_j z_{2,j}}}{\sum_j e^{z_{2,j} - \max_j z_{2,j}}} \quad \text{(Ch 8 — numerically stable softmax)}$$

This does not change the result mathematically (the max cancels), but prevents floating-point overflow when logit values are large.

10.2.4 Complete Forward Pass Summary

For a 2-layer network with input $\mathbf{x}$, parameters $(W_1, \mathbf{b}_1, W_2, \mathbf{b}_2)$:

$$\mathbf{z}_1 = W_1\mathbf{x} + \mathbf{b}_1 \quad \text{(Ch 3 — linear layer)} \quad \mathbf{h} = \text{ReLU}(\mathbf{z}_1) \quad \text{(Ch 5 — nonlinea}$$

The network has transformed a raw input vector into a probability distribution over classes. The question now is: how do we measure whether that distribution is good?

10.3 The Loss Function — Cross-Entropy from MLE

10.3.1 What the Loss Measures

The loss function answers: *how wrong is the prediction?* We need a scalar number that we can minimize with gradient descent.

For classification with K classes, we use **cross-entropy loss**:

$$L = -\sum_{k=1}^{K} y_k \log p_k \quad \text{(Ch 8 — cross-entropy)}$$

where $\mathbf{y} \in 0, 1^K$ is the **one-hot** true label vector ($y_k = 1$ for the true class, 0 for all others) and $\mathbf{p}$ is the softmax output.

Because $y_k = 1$ for exactly one class, this simplifies to:

$$L = -\log p_{\text{true class}} \quad \text{(Ch 8)}$$

Intuition: If the network assigns probability 0.99 to the correct class, $L = -\log(0.99) \approx 0.01$ — small loss. If it assigns 0.01, $L = -\log(0.01) \approx 4.6$ — large loss. The log function amplifies confidence in the wrong direction.

10.3.2 Why Cross-Entropy Comes from MLE

This is not an arbitrary choice — cross-entropy loss **is** maximum likelihood estimation for a categorical distribution.

Recall from Ch 9 (MLE): Given a model parameterized by θ, MLE finds the parameters that maximize the likelihood of the observed data:

$$\hat{\theta} = \arg\max_{\theta} \prod_{i=1}^{n} P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \theta) \quad \text{(Ch 9 — MLE)}$$

For a neural network with softmax output, the probability the model assigns to the true label for example i is $p^{(i)}_{\text{true class}}$. Taking the log-likelihood (from Ch 9):

$$\log \mathcal{L}(\theta) = \sum_{i=1}^{n} \log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \theta) = \sum_{i=1}^{n} \sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)} \quad \text{(Ch 9)}$$

Maximizing log-likelihood is equivalent to **minimizing** negative log-likelihood:

$$L = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)} \quad \text{(Ch 8 + Ch 9 — cross-entropy = negative MLE}$$

This is exactly the cross-entropy loss. So every time you minimize cross-entropy, you are doing MLE — finding the network weights that make the training data most probable under the model. The connection between Ch 8 and Ch 9 is not coincidence; it is the mathematical foundation.

10.4 The Backward Pass — Backpropagation

Backpropagation is the **chain rule from Chapter 5 applied recursively** to compute the gradient of the loss with respect to every parameter in the network. We need $\frac{\partial L}{\partial W_1}$, $\frac{\partial L}{\partial \mathbf{b}_1}$, $\frac{\partial L}{\partial W_2}$, $\frac{\partial L}{\partial \mathbf{b}_2}$ in order to run gradient descent.

10.4.1 The Key Insight: Softmax + Cross-Entropy Collapses Beautifully

Before working through the full chain, let's derive the gradient of cross-entropy loss through the softmax layer — this is the heart of the calculation.

Setup: We have logits $\mathbf{z}2 \in \mathbb{R}^K, probabilitiesp_k = \frac{e^{z{2,k}}}{\sum_j e^{z_{2,j}}}, andlossL = -\sum_k y_k \log p_k$.

We want: $\frac{\partial L}{\partial z_{2,k}}$ for each k.

Using the chain rule (Ch 5):

$$\frac{\partial L}{\partial z_{2,k}} = \sum_{j=1}^{K} \frac{\partial L}{\partial p_j} \cdot \frac{\partial p_j}{\partial z_{2,k}}$$

Step 1: $\frac{\partial L}{\partial p_j} = -\frac{y_j}{p_j}$ (derivative of $-y_j \log p_j$)

Step 2: The derivative of softmax (from Ch 8):

$$\frac{\partial p_j}{\partial z_{2,k}} = \begin{cases} p_k(1 - p_k) & \text{if } j = k \\ -p_j p_k & \text{if } j \neq k \end{cases} \quad \text{(Ch 8 — softmax Jacobian}$$

Step 3: Putting it together:

$$\frac{\partial L}{\partial z_{2,k}} = \frac{\partial L}{\partial p_k} \cdot \frac{\partial p_k}{\partial z_{2,k}} + \sum_{j \neq k} \frac{\partial L}{\partial p_j} \cdot \frac{\partial p_j}{\partial z_{2,k}}$$

$$= \left(-\frac{y_k}{p_k}\right) p_k(1 - p_k) + \sum_{j \neq k} \left(-\frac{y_j}{p_j}\right) (-p_j p_k)$$

$$= -y_k(1 - p_k) + \sum_{j \neq k} y_j p_k$$

$$= -y_k + y_k p_k + p_k \sum_{j \neq k} y_j$$

$$= -y_k + p_k \underbrace{\sum_{j=1}^{K} y_j}_{=1 \text{ (one-hot: exactly one } y_j=1)}$$

$$\boxed{\frac{\partial L}{\partial z_{2,k}} = p_k - y_k} \quad \text{(Ch 5 + Ch 8 + Ch 9 — the miracle)}$$

In vector form: $\frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y}$.

This is the capstone of the book. Three chapters of mathematics — the chain rule (Ch 5), softmax (Ch 8), and MLE / cross-entropy (Ch 9) — combine into a two-symbol answer: prediction minus truth. The cross-terms from the

softmax Jacobian cancel exactly because the one-hot labels sum to 1. If the model predicts $p_k = 1$ for the true class, the gradient is zero. If it predicts the wrong class confidently, the gradient is large and in the right direction.

10.4.2 Gradient with Respect to Output Layer Weights W_2

Now we work backward through the output layer. We have $\mathbf{z}_2 = W_2\mathbf{h} + \mathbf{b}_2$.

For the weight matrix W_2:

Each entry $W_{2,kj}$ affects $\mathbf{z}2only through z{2,k} = \sum_j W_{2,kj} h_j + b_{2,k}$.

$$\frac{\partial L}{\partial W_{2,kj}} = \frac{\partial L}{\partial z_{2,k}} \cdot \frac{\partial z_{2,k}}{\partial W_{2,kj}} = (p_k - y_k) \cdot h_j \quad \text{(Ch 5 — chain rule)}$$

Stacking these into matrix form (Ch 3):

$$\frac{\partial L}{\partial W_2} = (\mathbf{p} - \mathbf{y}), \mathbf{h}^\top \quad \text{(Ch 3 — outer product, Ch 5 — chain rule)}$$

For the bias $\mathbf{b}_2$:

$$\frac{\partial z_{2,k}}{\partial b_{2,k}} = 1, \quad \text{so} \quad \frac{\partial L}{\partial \mathbf{b}_2} = \mathbf{p} - \mathbf{y} \quad \text{(Ch 5)}$$

10.4.3 Gradient with Respect to Hidden Layer Weights W_1

To reach W_1, we must pass through two more operations: the output-layer multiplication and the ReLU. This is where the chain rule chains.

Step 1: Gradient with respect to hidden output $\mathbf{h}$

$$\frac{\partial L}{\partial \mathbf{h}} = W_2^\top \cdot (\mathbf{p} - \mathbf{y}) \quad \text{(Ch 3 — matrix transpose, Ch 5 — chain rule for linea}$$

Intuition: $W_2^\top$ "sends back" the error signal from the output to each hidden unit, weighted by how strongly that hidden unit influenced each output.

Step 2: Gradient through ReLU

Since $\mathbf{h} = \text{ReLU}(\mathbf{z}_1)$:

$$\frac{\partial L}{\partial \mathbf{z}_1} = \frac{\partial L}{\partial \mathbf{h}} \odot \text{ReLU}'(\mathbf{z}_1) \quad \text{(Ch 5 — chain rule, element-wise)}$$

where $\odot$ is element-wise multiplication and $\text{ReLU}'(z) = 1[z > 0]$.

This is the **gating** behavior of ReLU: if a hidden neuron was "off" during the forward pass ($z_{1,j} \leq 0$), no gradient flows back through it. That neuron neither contributed to the prediction nor receives any update signal.

Step 3: Gradient with respect to W_1

Since $\mathbf{z}_1 = W_1\mathbf{x} + \mathbf{b}_1$, by the same logic as Step 2 above:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \mathbf{z}_1} \cdot \mathbf{x}^\top \quad \text{(Ch 3 — outer product, Ch 5 — chain rule)}$$

$$\frac{\partial L}{\partial \mathbf{b}_1} = \frac{\partial L}{\partial \mathbf{z}_1} \quad \text{(Ch 5)}$$

10.4.4 Complete Backpropagation — All Gradients

Let $\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{y}$ (the output error) and $\boldsymbol{\delta}_1 = (W_2^\top \boldsymbol{\delta}_2) \odot \text{ReLU}'(\mathbf{z}_1)$ (the hidden error).

$$\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{y} \quad \text{(Ch 5+8+9 — combined gradient)} \backslash [4pt] \frac{\partial L}{\partial W_2} \quad = \boldsymbol{\delta}_2, \mathbf{h}^\top \quad \text{(Cl}$$

The pattern is systematic: every backward step multiplies by the transpose of the forward weight matrix and then gates by the activation derivative. This is the structure that makes backpropagation efficient — each gradient is a local computation at each layer, reusing the error signal from the layer above.

10.5 Weight Update — Gradient Descent and Adam

With the gradients computed, we apply the update rule.

10.5.1 Vanilla SGD

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W} \quad \text{(Ch 6 — gradient descent)}$$

$$\mathbf{b} \leftarrow \mathbf{b} - \alpha \frac{\partial L}{\partial \mathbf{b}} \quad \text{(Ch 6)}$$

The learning rate α controls the step size. Too large and training oscillates; too small and it is slow (Section 6.4 in Ch 6 showed this tradeoff explicitly).

10.5.2 Adam (for reference)

In practice, most deep learning training uses **Adam** (from Ch 6), which adapts the learning rate per parameter:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\nabla L \quad \text{(Ch 6 — first moment, like momentum)}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)(\nabla L)^2 \quad \text{(Ch 6 — second moment)}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \quad \text{(Ch 6 — bias correction)}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \quad \text{(Ch 6 — Adam update)}$$

Default hyperparameters: $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Why Adam beats SGD in practice: It normalizes updates by historical gradient magnitude, so parameters that historically receive large gradients get smaller step sizes and vice versa. This effectively sets an adaptive per-parameter learning rate — as if you ran SGD with a different α for every weight in the network.

The from-scratch implementation in Section 10.7 uses vanilla SGD for clarity. Adam would require tracking two extra momentum vectors per parameter but would change none of the mathematics.

10.6 Regularization in Neural Networks

Large neural networks have millions of parameters and can memorize training data perfectly — fitting noise rather than patterns. **Regularization** (Ch 9) fights this overfitting.

10.6.1 L2 Weight Decay

Add a penalty on large weights to the loss:

$$L_{\text{reg}} = L_{\text{CE}} + \frac{\lambda}{2}|W|_F^2 \quad \text{(Ch 9 — L2 regularization)}$$

where $|W|_F^2 = \sum_{i,j} W_{ij}^2$ is the squared Frobenius norm of the weight matrix (the matrix analog of the Euclidean norm from Ch 2).

The gradient of the regularization term is:

$$\frac{\partial}{\partial W}\left(\frac{\lambda}{2}|W|_F^2\right) = \lambda W \quad \text{(Ch 5 — derivative of squared norm)}$$

So the modified weight update becomes:

$$W \leftarrow W - \alpha\left(\frac{\partial L_{\text{CE}}}{\partial W} + \lambda W\right) = W(1-\alpha\lambda) - \alpha\frac{\partial L_{\text{CE}}}{\partial W} \quad \text{(Ch 6 — weight dec}$$

The factor $(1-\alpha\lambda)$ multiplies the weights before the gradient step — it *decays* the weights toward zero at every update, which is why this is called weight decay.

Statistical interpretation (from Ch 9): L2 regularization is equivalent to placing a Gaussian prior on the weights and doing MAP (maximum a posteriori) estimation rather than pure MLE. The MAP solution is:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta}\left[\log \mathcal{L}(\theta) - \frac{\lambda}{2}|\theta|^2\right] \quad \text{(Ch 9 — MAP = MLE + prior)}$$

10.6.2 Dropout — Regularization by Forgetting

Dropout is a regularization technique specific to neural networks. During training, each neuron's output is randomly zeroed out with probability p (typically $p = 0.5$):

$$h_j^{\text{drop}} = \begin{cases} h_j/(1-p) & \text{with probability } 1-p \\ 0 & \text{with probability } p \end{cases}$$

The division by $(1-p)$ is the **inverted dropout** scaling — it ensures the expected value of h_j^{drop} equals h_j, so we can use the network at test time without any dropout (just forward-pass as normal).

Why it works (probability perspective, Ch 7): Dropout forces the network to not rely on any single neuron. Each training step uses a different random subset of neurons, so the network must learn *redundant* representations. This is analogous to training an ensemble of exponentially many smaller networks and averaging them.

Connection to bias-variance tradeoff (Ch 9): Dropout increases bias slightly (the dropped neurons can't contribute) but substantially reduces variance (the model cannot overfit to specific neurons). This is the same bias-variance tradeoff from Ch 9 — regularization always trades some bias for variance reduction.

Note on the XOR implementation: We omit dropout from the from-scratch code. With only 4 training examples and a 4-hidden-unit network, dropout would be counterproductive — we have more regularization than we need. In real networks with millions of parameters and thousands of examples, dropout is essential.

10.7 Complete From-Scratch Implementation

We now build everything from scratch. This is a 2-layer neural network:

```
Input (2) → Hidden (4, ReLU) → Output (2, softmax) → cross-entropy loss
```

trained on the **XOR problem**: given inputs $(x_1, x_2) \in 0, 1^2$, predict $x_1 \oplus x_2$ (exclusive OR).

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

XOR is the canonical test for neural networks because it is **not linearly separable** — no single straight line can separate the two classes. A hidden layer is required.

Important: We pin the random seed to 7. Different seeds can yield dead ReLU units that prevent convergence. In production code you would try multiple seeds; here we use a known-good one.

```
"""
XOR Neural Network — Pure Python (no numpy/pytorch)
Architecture: 2 → 4 (ReLU) → 2 (softmax)
Optimizer:    SGD with full-batch gradient descent
Seed: 7 | lr: 0.5 | epochs: 5000
"""
import random
import math

# ─── Reproducibility ──────────────────────────────────────────────
random.seed(7)   # pinned — different seeds may stall due to dead ReLU units

# ═══════════════════════════════════════════════════════════════════
#  MATRIX / VECTOR UTILITIES
#  (These replace numpy. Every function maps to a Ch 3 concept.)
# ═══════════════════════════════════════════════════════════════════

def zeros(rows, cols):
    """Return a rows×cols matrix of zeros.  (Ch 3 — zero matrix)"""
    return [[0.0] * cols for _ in range(rows)]

def randn_matrix(rows, cols, scale=0.5):
    """Random Gaussian weight matrix.  (Ch 7 — Gaussian distribution)"""
    return [[random.gauss(0, scale) for _ in range(cols)]
            for _ in range(rows)]

def matvec(M, v):
    """Matrix–vector product M @ v.   (Ch 3 — matrix multiplication)
    M is shape (m × n), v is length-n list, returns length-m list."""
    return [sum(M[i][j] * v[j] for j in range(len(v)))
            for i in range(len(M))]

def outer(u, v):
    """Outer product u ⊗ v → matrix.  (Ch 3 — outer product)
    u is length-m, v is length-n, returns (m × n) matrix."""
    return [[u[i] * v[j] for j in range(len(v))]
            for i in range(len(u))]

def transpose(M):
    """Transpose a matrix.             (Ch 3 — matrix transpose)"""
    rows, cols = len(M), len(M[0])
    return [[M[r][c] for r in range(rows)] for c in range(cols)]

def add_vec(a, b):
    """Element-wise vector addition.   (Ch 2 — vector addition)"""
    return [a[i] + b[i] for i in range(len(a))]

def scale_mat(M, s):
```

```
    """Multiply every entry of matrix M by scalar s."""
    return [[M[i][j] * s for j in range(len(M[0]))] for i in
range(len(M))]

def add_mat(A, B):
    """Element-wise matrix addition.   (Ch 3 — matrix addition)"""
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))]
            for i in range(len(A))]

#
═══════════════════════════════════════════════════════════════
═══════
#  ACTIVATION FUNCTIONS
#
═══════════════════════════════════════════════════════════════
═══════

def relu(z):
    """ReLU activation: max(0, z) element-wise.  (Ch 5 — piecewise
function)"""
    return [max(0.0, zi) for zi in z]

def relu_deriv(z):
    """ReLU derivative: 1 if z > 0, else 0.      (Ch 5 — step
derivative)"""
    return [1.0 if zi > 0 else 0.0 for zi in z]

def softmax(z):
    """Numerically stable softmax.               (Ch 8 —
categorical distribution)
    Subtract max before exp to prevent overflow (Ch 8 — log-sum-exp
trick)."""
    m = max(z)
    exps = [math.exp(zi - m) for zi in z]
    s = sum(exps)
    return [e / s for e in exps]

def cross_entropy(p, y_onehot):
    """Cross-entropy loss: -Σ y_k log(p_k).     (Ch 8+9 — negative
log-likelihood)
    The 1e-15 guard prevents log(0).             (Ch 7 — numerical
stability)"""
    return -sum(y_onehot[k] * math.log(p[k] + 1e-15)
                for k in range(len(p)))

#
═══════════════════════════════════════════════════════════════
═══════
#  NETWORK ARCHITECTURE: 2 → 4 (ReLU) → 2 (softmax)
#
═══════════════════════════════════════════════════════════════
═══════
n_in, n_hidden, n_out = 2, 4, 2

#  Weight initialisation: small random Gaussians  (Ch 7 — Gaussian)
#  Bias initialisation: zeros
W1 = randn_matrix(n_hidden, n_in, scale=0.5)   # shape 4×2 — (Ch 3)
b1 = [0.0] * n_hidden                           # shape 4
```

```
W2 = randn_matrix(n_out, n_hidden, scale=0.5)  # shape 2×4 — (Ch 3)
b2 = [0.0] * n_out                              # shape 2

#
═══════════════════════════════════════════════════════════════════
═══════
#  XOR DATASET  (4 examples, 2 input features, 2 classes)
#
═══════════════════════════════════════════════════════════════════
═══════
X_data = [[0, 0], [0, 1], [1, 0], [1, 1]]
Y_data = [0,      1,      1,      0     ]   # XOR labels

def to_onehot(label, n_classes=2):
    """Convert integer label to one-hot vector.  (Ch 8 —
categorical distribution)"""
    v = [0.0] * n_classes
    v[label] = 1.0
    return v

#
═══════════════════════════════════════════════════════════════════
═══════
#  FORWARD PASS
#
═══════════════════════════════════════════════════════════════════
═══════

def forward(x):
    """
    Forward pass for one example.
    Returns intermediate values needed for backpropagation.

    z1 — pre-activation of hidden layer,    shape: (n_hidden,)  [Ch
3]
    h  — hidden layer output = ReLU(z1),    shape: (n_hidden,)  [Ch
5]
    z2 — pre-activation of output layer,    shape: (n_out,)     [Ch
3]
    p  — output probabilities = softmax(z2),shape: (n_out,)     [Ch
8]
    """
    z1 = add_vec(matvec(W1, x), b1)   # z1 = W1 @ x + b1     [Ch 3]
    h  = relu(z1)                      # h  = ReLU(z1)          [Ch
5]
    z2 = add_vec(matvec(W2, h), b2)   # z2 = W2 @ h + b2     [Ch 3]
    p  = softmax(z2)                   # p  = softmax(z2)       [Ch
8]
    return z1, h, z2, p

#
═══════════════════════════════════════════════════════════════════
═══════
#  TRAINING LOOP
#
═══════════════════════════════════════════════════════════════════
═══════
lr       = 0.5    # learning rate (step size for gradient descent)
```

```
[Ch 6]
n_epochs = 5000   # number of full passes over the training data

print(f"Training XOR network: {n_in}→{n_hidden}(ReLU)→{n_out}
(softmax)")
print(f"Optimizer: SGD, lr={lr}, seed=7, {n_epochs} epochs")
print()
print(f"{'Epoch':>6} | {'Avg Loss':>10}")
print(f"{'------':>6}-+-{'----------':>10}")

for epoch in range(1, n_epochs + 1):
    total_loss = 0.0

    # Accumulate gradients over all 4 training examples
    # (full-batch gradient descent — Ch 6)
    dW1_acc = zeros(n_hidden, n_in)
    db1_acc = [0.0] * n_hidden
    dW2_acc = zeros(n_out, n_hidden)
    db2_acc = [0.0] * n_out

    for x, label in zip(X_data, Y_data):
        y_oh = to_onehot(label)

        # ── FORWARD PASS
──────────────────────────────────────────────
        z1, h, z2, p = forward(x)
        total_loss += cross_entropy(p, y_oh)   # accumulate loss
[Ch 8+9]

        # ── BACKWARD PASS — output layer
──────────────────────────
        # Combined softmax + cross-entropy gradient: dL/dz2 = p - y
        # Derivation in Section 10.4.1                          [Ch
5+8+9]
        dz2   = [p[k] - y_oh[k] for k in range(n_out)]

        # dL/dW2 = dz2 ⊗ h  (outer product)                    [Ch
3+5]
        dW2_g = outer(dz2, h)

        # dL/db2 = dz2                                          [Ch
5]
        db2_g = dz2[:]

        # ── BACKWARD PASS — hidden layer
──────────────────────────
        # Backpropagate error through W2: dL/dh = W2^T @ dz2   [Ch
3+5]
        W2T   = transpose(W2)
        dh    = matvec(W2T, dz2)

        # Gate through ReLU derivative: dL/dz1 = dL/dh ⊙ relu'(z1)
[Ch 5]
        rd    = relu_deriv(z1)
        dz1   = [dh[i] * rd[i] for i in range(n_hidden)]

        # dL/dW1 = dz1 ⊗ x  (outer product)                    [Ch
3+5]
```

```
        dW1_g = outer(dz1, x)

        # dL/db1 = dz1                                                [Ch
5]
        db1_g = dz1[:]

        # ── ACCUMULATE gradients
──────────────────────────────────
        dW2_acc = add_mat(dW2_acc, dW2_g)
        db2_acc = add_vec(db2_acc, db2_g)
        dW1_acc = add_mat(dW1_acc, dW1_g)
        db1_acc = add_vec(db1_acc, db1_g)

    # ── SGD UPDATE: θ ← θ − α * ∇L  (average gradient over batch)
──────
    # [Ch 6 – gradient descent update rule]
    n = len(X_data)
    W1[:] = add_mat(W1, scale_mat(dW1_acc, -lr / n))
    b1[:] = add_vec(b1, [-lr / n * g for g in db1_acc])
    W2[:] = add_mat(W2, scale_mat(dW2_acc, -lr / n))
    b2[:] = add_vec(b2, [-lr / n * g for g in db2_acc])

    avg_loss = total_loss / n
    if epoch % 100 == 0 or epoch == 1:
        print(f"{epoch:>6} | {avg_loss:>10.6f}")

#
═══════════════════════════════════════════════════════════════════
═══════
#  FINAL PREDICTIONS
#
═══════════════════════════════════════════════════════════════════
═══════
print()
print("Final predictions:")
print(f"{'Input':>8} | {'True':>5} | {'Pred':>5} |
{'p(class=1)':>10}")
print(f"{'--------':>8}-+-{'-----':>5}-+-{'-----':>5}-+-{'--------
--':>10}")
for x, label in zip(X_data, Y_data):
    _, _, _, p = forward(x)
    pred = 1 if p[1] > 0.5 else 0
    print(f"  {x}  |  {label:3d}  |  {pred:3d}  |  {p[1]:8.4f}")

print()
print(f"Final avg cross-entropy loss: {avg_loss:.6f}")
```

10.7.1 Running the Code — Verified Output

When you run the code above (Python 3, no dependencies), you get:

```
Training XOR network: 2→4(ReLU)→2(softmax)
Optimizer: SGD, lr=0.5, seed=7, 5000 epochs

 Epoch |   Avg Loss
-------+-----------
     1 |   0.718120
   100 |   0.102532
   200 |   0.025275
   300 |   0.013052
   400 |   0.008558
   500 |   0.006275
   600 |   0.004944
   700 |   0.004050
   800 |   0.003417
   900 |   0.002952
  1000 |   0.002596
  1100 |   0.002314
  1200 |   0.002084
  1300 |   0.001894
  1400 |   0.001733
  1500 |   0.001599
  1600 |   0.001483
  1700 |   0.001381
  1800 |   0.001292
  1900 |   0.001214
  2000 |   0.001145
  2100 |   0.001082
  2200 |   0.001026
  2300 |   0.000975
  2400 |   0.000929
  2500 |   0.000886
  2600 |   0.000848
  2700 |   0.000813
  2800 |   0.000780
  2900 |   0.000750
  3000 |   0.000722
  3100 |   0.000695
  3200 |   0.000671
  3300 |   0.000648
  3400 |   0.000627
  3500 |   0.000607
  3600 |   0.000588
  3700 |   0.000571
  3800 |   0.000554
  3900 |   0.000538
  4000 |   0.000523
  4100 |   0.000509
  4200 |   0.000496
  4300 |   0.000483
  4400 |   0.000471
  4500 |   0.000459
  4600 |   0.000448
  4700 |   0.000438
  4800 |   0.000428
  4900 |   0.000418
  5000 |   0.000409
```

```
Final predictions:
   Input |  True |  Pred | p(class=1)
---------+-------+-------+-----------
  [0, 0]  |    0  |    0  |    0.0011
  [0, 1]  |    1  |    1  |    0.9998
  [1, 0]  |    1  |    1  |    0.9998
  [1, 1]  |    0  |    0  |    0.0001

Final avg cross-entropy loss: 0.000409
```

The loss starts near $\ln(2) \approx 0.693$ (the entropy of a uniform binary distribution — from Ch 8), drops rapidly to below 0.01 by epoch 500, and converges to **0.000409** — far below the 0.05 target. All 4 XOR predictions are correct.

10.7.2 Tracing the Math Through One Training Step

Let's trace one backward pass step by step, mapping each computation to its equation:

Example: input $\mathbf{x} = [0, 1]$, label $= 1$, one-hot $\mathbf{y} = [0, 1]$.

After the forward pass, suppose $\mathbf{p} = [0.45, 0.55]$ (network slightly favors class 1, correct but not confident).

Output error (Section 10.4.1):

$$\boldsymbol{\delta}_2 = \mathbf{p} - \mathbf{y} = [0.45 - 0, 0.55 - 1] = [0.45, -0.45] \quad \text{(Ch 5+8+9)}$$

Interpretation: class 0 is getting too much probability (0.45 instead of 0), class 1 gets too little (0.55 instead of 1). The error signal has the right sign to correct both.

Gradient for W_2 (Section 10.4.2):

$$\frac{\partial L}{\partial W_2} = \boldsymbol{\delta}_2 \otimes \mathbf{h} \quad \text{(Ch 3+5)}$$

Each entry: $\frac{\partial L}{\partial W_{2,kj}} = \delta_{2,k} \cdot h_j$. The gradient is large where the error $\delta_{2,k}$ is large **and** the hidden activation h_j is large — both conditions must hold.

Backprop through W_2 (Section 10.4.3):

$$\frac{\partial L}{\partial \mathbf{h}} = W_2^\top \boldsymbol{\delta}_2 \quad \text{(Ch 3+5)}$$

Each hidden unit gets a gradient signal proportional to its contribution to the output error, weighted by W_2.

Gate through ReLU:

$$\frac{\partial L}{\partial \mathbf{z}_1} = \frac{\partial L}{\partial \mathbf{h}} \odot \text{ReLU}'(\mathbf{z}_1) \quad \text{(Ch 5)}$$

Hidden units that were "off" (output 0 during forward pass) receive zero gradient. They do not update. Hidden units that were "on" pass the full gradient through.

SGD update:

$$W_1 \leftarrow W_1 - \alpha \cdot \boldsymbol{\delta}_1 \otimes \mathbf{x} \quad \text{(Ch 6)}$$

The weight connecting input feature x_j to hidden unit i is updated proportionally to how much the feature was active (x_j) and how much the hidden unit's error signal is ($\delta_{1,i}$).

10.8 What You Now Know — The Complete Map

Every concept in this book just appeared in the code above. Here is the full mapping:

Concept	Chapter	Where it appears in the neural network
Vectors	Ch 2	$\mathbf{x}$, $\mathbf{h}$, $\mathbf{p}$, $\mathbf{b}$ — every layer input/output is a vector
Matrix-vector product	Ch 3	$W\mathbf{x} + \mathbf{b}$ — the linear layer
Matrix transpose	Ch 3	$W_2^\top$ — backpropagating error to previous layer
Outer product	Ch 3	$\boldsymbol{\delta} \otimes \mathbf{h}$ — weight gradient
Frobenius norm	Ch 3	$\lvert W\rvert_F^2$ — L2 regularization penalty
SVD / PCA	Ch 4	Feature preprocessing; understanding what each layer "sees"
Derivatives	Ch 5	Gradient of loss w.r.t. every parameter
Chain rule	Ch 5	Backpropagation — the entire backward pass
ReLU and its derivative	Ch 5	Activation function and its gradient gate
Gradient descent	Ch 6	Weight update: $\theta \leftarrow \theta - \alpha \nabla L$
Adam optimizer	Ch 6	Adaptive per-parameter learning rates in real training
Probability distributions	Ch 7	Network output as a probability distribution
Gaussian distribution	Ch 7	Weight initialization: `random.gauss(0, scale)`
Bayes' theorem	Ch 7	MAP estimation; understanding dropout as an ensemble
Softmax	Ch 8	Final layer: converting logits to probabilities
Cross-entropy	Ch 8	The loss function
KL divergence	Ch 8	Cross-entropy = KL(true dist ‖ model dist) + constant; VAE losses
MLE	Ch 9	Cross-entropy loss IS negative log-likelihood
Bias-variance tradeoff	Ch 9	Underfitting vs overfitting; why we regularize

L2 regularization	Ch 9	Weight decay: $\lambda \lvert W \rvert_F^2$ penalty
Cross-validation	Ch 9	Choosing hyperparameters (learning rate, λ, depth)

There is not a single line of the training loop that doesn't connect to a chapter in this book.

10.9 Common Failure Modes — Using the Math to Debug

Now that you can read the math, you can debug with it.

Symptom: Loss doesn't decrease (or increases)

- **Diagnosis:** Learning rate too high (Ch 6). The gradient step overshoots the minimum.
- **Fix:** Reduce α by 10×. Plot loss vs epoch — if it oscillates, α is the culprit.

Symptom: Loss decreases then plateaus far from zero

- **Diagnosis 1:** Dead ReLU units (Ch 5). Neurons with negative $\mathbf{z}_1$ output zero and receive zero gradient — they never recover. Check: print the fraction of hidden activations that are positive. If it's near zero, this is the problem.
- **Diagnosis 2:** Bad initialization. Very large initial weights saturate the softmax, leaving tiny gradients.
- **Fix:** Try different seeds, use He initialization: scale $= \sqrt{2/n_{\text{in}}}$.

Symptom: Training loss low, validation loss high

- **Diagnosis:** Overfitting — high variance (Ch 9 — bias-variance tradeoff).
- **Fix:** Add L2 weight decay (Ch 9), dropout (Section 10.6), more training data, or reduce model size.

Symptom: Training loss and validation loss both high

- **Diagnosis:** Underfitting — high bias (Ch 9).

- **Fix:** Increase model capacity (more hidden units, more layers), train longer, reduce regularization.

Symptom: NaN loss

- **Diagnosis:** Numerical overflow. Large logits → `exp(large)` → `inf`. Or `log(0)`.
- **Fix:** Use numerically stable softmax (subtract max — Ch 8), add `1e-15` inside `log` (Ch 7 — numerical stability conventions).

10.10 Scaling Up — From XOR to Real Networks

The XOR network has 4 parameters per weight matrix. Real networks have millions. How does everything we derived scale?

The math stays exactly the same. The formulas $\boldsymbol{\delta} = \mathbf{p} - \mathbf{y}$, $\frac{\partial L}{\partial W} = \boldsymbol{\delta} \otimes \mathbf{h}$, $\frac{\partial L}{\partial \mathbf{h}} = W^\top \boldsymbol{\delta}$ hold for any network width.

What changes is **engineering**, not math:

Aspect	XOR network	Real network
Data	4 examples	Millions of examples
Batch	Full batch	Mini-batches of 32–256
Optimizer	SGD	Adam (Ch 6)
Initialization	Gaussian(0, 0.5)	He init: $\mathcal{N}(0, \sqrt{2/n_{\text{in}}})$
Layers	2	10–1000+ (residual connections)
Regularization	None	Dropout, weight decay, batch normalization
Hardware	CPU, milliseconds	GPU/TPU, hours/days

What PyTorch / TensorFlow actually do: They implement the same forward pass, the same cross-entropy loss, the same backpropagation chain rule, and the same Adam update — but using:

1. **Automatic differentiation** instead of hand-derived gradients (the framework tracks the computation graph and applies the chain rule

symbolically)

2. **GPU-accelerated matrix multiplication** (BLAS/cuBLAS instead of our nested Python loops)
3. **Optimized numerical routines** for softmax, loss, etc.

The math you derived in this chapter is what those frameworks compute. When you call `loss.backward()` in PyTorch, it runs the chain rule through the computation graph, computing exactly the gradients in Section 10.4.

10.11 Where to Go Next

You have the mathematical foundation. Here are the most productive next steps:

Implement Before Moving On

Before touching a framework, implement one more from-scratch network:

- Replace ReLU with sigmoid and verify the chain rule changes
- Add a third hidden layer and derive the backprop for a 3-layer network
- Implement Adam in place of SGD and compare convergence speed

These exercises will make the abstractions in PyTorch and JAX feel transparent rather than magical.

Core Resources

Linear Algebra:

- Gilbert Strang's *Introduction to Linear Algebra* (MIT OpenCourseWare)
- 3Blue1Brown's "Essence of Linear Algebra" (YouTube) — the best visual introduction

Calculus and Backpropagation:

- Andrej Karpathy's "micrograd" (GitHub) — a 100-line autograd engine with the same math as this chapter
- Karpathy's "Neural Networks: Zero to Hero" lecture series (YouTube)

Probability and Information Theory:

- Christopher Bishop's *Pattern Recognition and Machine Learning* — Chapter 1 (free PDF online)
- Cover and Thomas, *Elements of Information Theory* — for KL divergence and cross-entropy depth

Neural Networks:

- Michael Nielsen's *Neural Networks and Deep Learning* (free online at neuralnetworksanddeeplearning.com) — builds everything from scratch, same spirit as this book
- Ian Goodfellow et al., *Deep Learning* (deeplearningbook.org) — the standard graduate reference

Practical ML:

- Andrej Karpathy's *makemore* and *nanoGPT* — transformer implementations with hand-written training loops that you can now read
- *Dive into Deep Learning* (d2l.ai) — mathematical rigor with PyTorch/JAX code side by side

The Next Layer of Math

Once you're comfortable with what's in this book, these are the natural next topics:

Topic	Why it matters
Attention and transformers	Self-attention is a learned softmax over dot products — Ch 8 + Ch 3
Variational inference	KL divergence minimization for latent variable models — Ch 8
Natural gradient	Optimization using the Fisher information matrix — Ch 9 + Ch 4
Convolutions as structured matrices	CNNs are matrix operations with shared weights — Ch 3
PAC learning theory	Formal generalization bounds — extends Ch 9

10.12 Chapter Summary

You started this book knowing high school algebra. You now understand every piece of mathematics that runs inside a neural network.

The forward pass is matrix multiplication (Ch 3) followed by a piecewise linear activation function (Ch 5) followed by a softmax that turns scores into probabilities (Ch 8).

The loss is cross-entropy — the negative log-likelihood of the true class under the model's probability distribution (Ch 8 + Ch 9). Minimizing it is exactly MLE.

The backward pass is the chain rule applied recursively (Ch 5). The key identity — $\frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y}$ — emerges from the interaction between softmax and cross-entropy, with terms canceling because one-hot labels sum to 1.

The weight update is gradient descent or Adam (Ch 6), moving parameters in the direction that reduces the loss. Regularization via L2 weight decay (Ch 9) constrains the weights and reduces overfitting.

None of this required magic. Every step followed from definitions you learned in Chapters 2 through 9.

Exercises

10.1 In the from-scratch code, `dz2 = [p[k] - y_oh[k] for k in range(n_out)]` computes the combined softmax + cross-entropy gradient. Verify this formula by expanding the chain rule for $K = 2$ classes: compute $\frac{\partial L}{\partial z_{2,0}}$ and $\frac{\partial L}{\partial z_{2,1}}$ separately using the softmax derivative from Ch 8 and the chain rule from Ch 5.

Solution: For $K = 2$ with $y = [0, 1]$ (true class is 1):

$p_0 = \frac{e^{z_0}}{e^{z_0}+e^{z_1}}$, $p_1 = \frac{e^{z_1}}{e^{z_0}+e^{z_1}}$, $L = -\log p_1$.

$\frac{\partial L}{\partial z_0} = -\frac{1}{p_1} \cdot \frac{\partial p_1}{\partial z_0} = -\frac{1}{p_1} \cdot (-p_1 p_0) = p_0 = p_0 - y_0$. ✓

$\frac{\partial L}{\partial z_1} = -\frac{1}{p_1} \cdot \frac{\partial p_1}{\partial z_1} = -\frac{1}{p_1} \cdot p_1(1 - p_1) = -(1 - p_1) = p_1 - 1 = p_1 - y_1$. ✓

Both match the formula $p_k - y_k$.

10.2 What is the gradient $\frac{\partial L}{\partial W_1}$ in terms of the intermediates $\boldsymbol{\delta}_1$, $\mathbf{x}$? Write it as a matrix equation and explain why it is an outer product.

Solution: $\frac{\partial L}{\partial W_1} = \boldsymbol{\delta}_1 \otimes \mathbf{x}$, where $[\boldsymbol{\delta}_1 \otimes \mathbf{x}]_{ij} = \delta_i x_j$. It is an outer product because $W_{1,ij}$ affects the loss only through $z_{1,i} = \sum_j W_{1,ij} x_j$, so $\frac{\partial z_{1,i}}{\partial W_{1,ij}} = x_j$, and by the chain rule $\frac{\partial L}{\partial W_{1,ij}} = \frac{\partial L}{\partial z_{1,i}} \cdot x_j = \delta_{1,i} \cdot x_j$.

10.3 In the XOR output, `p(class=1)` for input `[0,0]` is 0.0011 and for `[0,1]` is 0.9998. What is the cross-entropy loss for each of these examples? Which one dominates the average loss?

Solution:

- $[0, 0]$: true class 0, loss $= -\log(p_0) = -\log(1 - 0.0011) = -\log(0.9989) \approx 0.0011$.
- $[0, 1]$: true class 1, loss $= -\log(0.9998) \approx 0.0002$.

The `[0,0]` example contributes slightly more loss. Both are tiny. Average $\approx (0.0011 + 0.0002 + 0.0002 + 0.0001)/4 \approx 0.0004$, consistent with the reported 0.000409.

10.4 Suppose you add L2 regularization with $\lambda = 0.01$ to the XOR training. Write the modified loss function and the modified gradient expression for W_1.

Solution:

$$L_{\text{reg}} = L_{\text{CE}} + \frac{0.01}{2}\left(|W_1|_F^2 + |W_2|_F^2\right) \quad \text{(Ch 9)}$$

The gradient becomes:

$$\frac{\partial L_{\text{reg}}}{\partial W_1} = \frac{\partial L_{\text{CE}}}{\partial W_1} + 0.01 \cdot W_1 = \boldsymbol{\delta}_1 \otimes \mathbf{x} + 0.01 \cdot W_1$$

The SGD update: $W_1 \leftarrow W_1 - \alpha(\boldsymbol{\delta}_1 \otimes \mathbf{x} + 0.01 \cdot W_1) = W_1(1 - 0.01\alpha) - \alpha\, \boldsymbol{\delta}_1 \otimes \mathbf{x}$.

10.5 (Challenge) The XOR network uses 4 hidden units. Explain why 2 hidden units (the theoretical minimum for XOR) often fails to converge with ReLU, and why adding more units helps even though 2 would suffice in principle.

Solution: In theory, 2 ReLU units suffice to compute XOR (each unit draws a decision boundary, and their intersection separates the classes). In practice, with random initialization:

(a) **Dead ReLU problem (Ch 5):** If both units initialize with negative weights for one input, they both output 0 for that input — providing no gradient signal. With only 2 units, one dead unit halves the network's capacity; both dead means no signal at all.

(b) **Symmetry breaking (Ch 6):** Gradient descent can get stuck at saddle points (Section 6.5) when units are nearly identical. With 2 units, symmetry is not easily broken.

(c) **Wider networks are easier to optimize:** Adding units provides redundant paths through the network. If one path is blocked (dead ReLU), others can still carry gradient signal. The final loss landscape with 4 units has fewer problematic saddle points than with 2. This is a practical consequence of the bias-variance perspective (Ch 9): extra capacity doesn't hurt generalization on this problem, but it dramatically helps optimization.

You have finished the mathematical foundations of machine learning. The next step is to build things. Start with micrograd, then nanoGPT. The math will follow you.